Pontificia Universidad Javeriana Cali
Facultad de Ingeniería.
Ingeniería de Sistemas y Computación.
Proyecto de Grado.

Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente

Julio Alejandro Mazo Reyes

Director: MSc. Mario Julián Mora Cardona

27/05/2026



Santiago de Cali, 27/05/2026.

Señores
Pontificia Universidad Javeriana Cali.
Dr. Gerardo Mauricio Sarria
Director Carrera de Ingeniería de Sistemas y Computación.
Cali.

Cordial Saludo.

Por medio de la presente me permito informarle que el estudiante de Ingeniería de Sistemas y Computación Julio Alejandro Mazo Reyes (cod: 8962011) trabaja bajo mi dirección en el proyecto de grado titulado "**Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente**".

Atentamente,

MSc. Mario Julián Mora Cardona

Santiago de Cali, 27/05/2026.

Señores
Pontificia Universidad Javeriana Cali.
Dr. Gerardo Mauricio Sarria
Director Carrera de Ingeniería de Sistemas y Computación.
Cali.

Cordial Saludo.

Me permito presentar a su consideración el anteproyecto de grado titulado "**Prototipo de plataforma web para la identificación de falencias y la retroalimentación docente en procesos de aprendizaje asistido por un sistema de tutoría inteligente**" con el fin de cumplir con los requisitos exigidos por la Universidad para llevar a cabo el proyecto de grado y posteriormente optar al título de Ingeniero de Sistemas y Computación.

Al firmar aquí, doy fe que entiendo y conozco las directrices para la presentación de trabajos de grado de la Facultad de Ingeniería aprobadas el 26 de Noviembre de 2009, donde se establecen los plazos y normas para el desarrollo del anteproyecto y del trabajo de grado.

Atentamente,

Julio Alejandro Mazo Reyes
Código: 8962011

Resumen

Este proyecto presenta el desarrollo de un prototipo de plataforma web que integra un Sistema de Tutoría Inteligente (ITS) basado en Modelos de Lenguaje de Gran Escala (LLM), orientado a la identificación de falencias en el aprendizaje y la generación de retroalimentación en cursos de Programación. El sistema es configurado mediante técnicas de prompting para guiar al estudiante a través del método socrático, y ofrece al docente un panel de visualización con métricas sobre el proceso de aprendizaje del grupo. La plataforma es desarrollada en Java Spring Boot con Supabase como Base de datos, documentada bajo el modelo C4 y evaluada mediante encuestas de percepción basadas en los modelos SUS y TAM.

Palabras Clave: Sistema Inteligente de Tutoría(ITS), Inteligencia Artificial Generativa, Prompting, C4, Retroalimentación, Programación, Método Socrático.

Índice general

1. Descripción del Problema **11**
1.1. Planteamiento del Problema . . . 11
1.1.1. Formulación . . . 12
1.1.2. Sistematización . . . 12
1.2. Objetivos . . . 13
1.2.1. Objetivo General . . . 13
1.2.2. Objetivos Específicos . . . 13
1.3. Justificación . . . 13
1.4. Alcances . . . 14
1.4.1. Delimitaciones y Entregables . . . 15
1.5. Metodología . . . 16
1.5.1. Tipo de Estudio. . . . 16
1.5.2. Actividades . . . 16
1.6. Cronograma . . . 18

2. Marco Teórico y Trabajos Relacionados **19**
2.1. Marco Teórico . . . 19
2.2. Trabajos Relacionados . . . 28
2.3. Diferenciadores del Proyecto . . . 29

3. Desarrollo de la Plataforma **31**
3.1. Revisión de Literatura para la Definición de Tipo de Feedback Requerido . . . 31
3.2. Requerimientos de un Sistema de Tutoría Inteligente . . . 33
3.3. Definición de Requerimientos, Métricas, Módulos y Tecnologías de la Plataforma . . . 34
3.3.1. Tecnologías de la Plataforma . . . 34
3.3.2. Módulos de la Plataforma . . . 35
3.3.3. Requisitos Funcionales . . . 37
3.3.4. Requisitos No Funcionales . . . 39
3.3.5. Métricas de Seguimiento . . . 39
3.4. Decisiones de Diseño de la Arquitectura de Prompts y Persistencia . . . 40
3.4.1. Arquitectura de Prompts y Persistencia de Datos . . . 40
3.4.2. Definicion de API de IA a Utilizar . . . 45
3.4.3. Flujos de Datos y Persistencia . . . 48
3.5. Modelo C4 . . . 51
3.5.1. Diseño de Diagrama de Contexto . . . 51
3.5.2. Diagrama de Contenedores . . . 52
3.5.3. Diagrama de Componentes . . . 53

3.5.4. Diagrama de Código . . . 55
3.5.5. Esquema de Base de Datos . . . 56
3.6. Decisiones de Arquitectura y Desarrollo del Prototipo de la Plataforma . . . 57
3.6.1. Organización del Proyecto . . . 58
3.6.2. Proceso de Desarrollo Incremental . . . 61
3.6.3. Gestión de Usuarios y Acceso . . . 61
3.6.4. Portal del Profesor . . . 64
3.6.5. Funcionamiento Interno del Tutor Socrático . . . 71
3.6.6. Trazabilidad de los Requisitos . . . 73
3.6.7. Desarrollo del Prompt del Tutor Socrático . . . 75

4. Evaluación de la Plataforma **79**
4.1. Planificación de la Evaluación . . . 79
4.2. Diseño del Cuestionario . . . 79
4.2.1. Escala de Usabilidad SUS . . . 80
4.2.2. Ítems de Aceptación basados en el Modelo TAM . . . 81
4.2.3. Integración del Cuestionario en la Plataforma . . . 82
4.3. Selección de los Participantes . . . 83
4.4. Aplicación del Cuestionario . . . 84
4.5. Análisis de Resultados . . . 84
4.5.1. Usabilidad según la Escala SUS . . . 84
4.5.2. Aceptación según el Modelo TAM . . . 85
4.5.3. Comentarios de los Estudiantes . . . 86
4.5.4. Uso del Prototipo durante la Actividad . . . 87
4.6. Limitaciones de la Evaluación . . . 88
4.7. Síntesis del Capítulo . . . 88

5. Conclusiones y Trabajos Futuros **89**
5.1. Conclusiones . . . 89
5.2. Trabajos Futuros . . . 90

6. Anexos **93**
6.0.1. Prompt de la Capa Fija del Tutor Socrático . . . 93
6.0.2. Video de Explicación del uso de la Plataforma . . . 93
6.0.3. Repositorio del Código Fuente . . . 93

Introducción

En la actualidad, la enseñanza de la programación en la educación superior enfrenta desafíos significativos debido a la alta demanda de estudiantes y la complejidad de conceptos abstractos y altamente lógicos. Esta situación suele aparecer por la falta de capacidad de los docentes para ofrecer un seguimiento individualizado y adaptable al mismo tiempo genera una "brecha de retroalimentación" donde el estudiante no recibe guías oportunas y el profesor carece de visibilidad sobre los obstáculos específicos que enfrenta cada alumno durante su proceso de aprendizaje autónomo.

Como respuesta a esta problemática, los Sistemas de Tutoría Inteligente (ITS) han emergido como herramientas capaces de emular la labor de un tutor humano mediante el uso de Inteligencia Artificial. Sin embargo, muchos de estos sistemas se limitan a entregar respuestas directas, lo que puede reducir el esfuerzo cognitivo del estudiante. Por este motivo, el presente proyecto propone el desarrollo de un prototipo de plataforma web que utiliza un (ITS) de diálogo socrático mediado por IA, que en lugar de resolver el problema por el alumno, el sistema actúa como un guía que formula preguntas y analogías para fomentar el pensamiento crítico.

El proyecto se enfoca en integrar este asistente conversacional con un módulo de métricas para el docente, permitiéndole identificar falencias recurrentes y niveles de dominio de los temas abordados. Para ello, mediante una estructuración modular de la plataforma y el uso de técnicas de prompting, se busca no solo apoyar la labor del profesor, sino también transformar la interacción con la IA en una experiencia pedagógica estructurada que fortalezca las competencias lógicas de los estudiantes de programación.

Capítulo 1

Descripción del Problema

1.1. Planteamiento del Problema

Históricamente, la enseñanza en ciencias formales como la computación ha lidiado con la complejidad de transmitir conceptos abstractos y lógicos que no se pueden ver ni tocar pero que si enfrentan retos pedagógicos que persisten hasta la actualidad. En el presente, esta problemática se intensifica en carreras con un alto componente matemático y lógico, como lo puede ser la ingeniería de sistemas, o las propias ciencias de la computación, donde la abstracción necesaria para dominar la programación crea barreras cognitivas en algunas ocasiones críticas en los estudiantes. Sin embargo, con dificultades los estudiantes siguen luchando con el objetivo de comprender conceptos fundamentales como la lógica, la sintaxis, las estructuras y los tipos de datos (Abiodun & Isong, 2025).

Asimismo, en materias como la Programación Orientada a Objetos (POO), esta situación es especialmente crítica, dado que el alumno se enfrenta a múltiples conceptos en un periodo corto de tiempo. La complejidad no solo radica en la lógica, sino en la necesidad de integrar simultáneamente el paradigma, el lenguaje de programación, el entorno de desarrollo y la metodología. Conceptos pilares como la abstracción, la herencia y el polimorfismo se tornan difíciles de asimilar, creando barreras que los métodos tradicionales de enseñanza no logran derribar fácilmente (Almarales & Fernández, 2019).

Por otra parte, según algunos estudios, las problemáticas mencionadas anteriormente generan una alta tasa de reprobación y una falta de autoconfianza en los estudiantes lo que deriva en un rendimiento académico deficiente (Abiodun & Isong, 2025). Lo que se analiza a menudo, es que este tipo de problemas se genera gracias a que el modelo educativo actual ha dependido casi siempre de una perspectiva de transmisión "simple" de conocimientos en donde se ignora las necesidades individuales de los estudiantes. Esto a su vez crea una "brecha de retroalimentación" (feedback gap), donde el estudiante no logra cerrar la distancia entre su desempeño actual y las metas de aprendizaje deseadas debido a la falta de claridad en las guías o la insuficiencia de recursos personalizados (Evans, 2013).

Si bien estos obstáculos han sido altamente estudiados y documentados desde los inicios de la disciplina (Abiodun & Isong, 2025; Evans, 2013), hoy nos encontramos en un punto de inflexión con la llegada de la Inteligencia Artificial Generativa. Esta tecnología no solo se caracteriza por su facilidad de uso, sino que surge como una oportunidad sin precedentes para transformar la educación tradicional que está basada en un modelo de "talla única" y convertirlo en un modelo dinámico e inclusivo que permite la creación de entornos de aprendizaje adaptativos y personalizados que pueden identificar dónde se encuentran las dificultades específicas de un alumno, ofreciendo estrategias

alternativas y retroalimentación inmediata que el sistema convencional no ha logrado cubrir por sí solo (Rodríguez Chávez, 2021).

De este modo, herramientas como los sistemas inteligentes de tutoria (ITS) impulsados por IA, no solo facilitan la recuperación de información y la generación de ideas, sino que ayudan a los estudiantes a comprender mejor el contenido del estudio al adaptarse a su ritmo individual y generar retroalimentación inmediata al estudiante que no se caracterice por la simple transmisión de conocimiento (Rodríguez Chávez, 2021; Zhou, 2025). Sin embargo, el problema persiste en la falta de integración formal de estas herramientas dentro de las actividades pedagógicas actuales, lo que genera una desconexión entre el potencial tecnológico y la práctica educativa en la actualidad.

Por último, si bien en la literatura se encuentran propuestas que abordan parcialmente esta problemática, la mayoría no logra integrar de forma completa los elementos necesarios para un acompañamiento pedagógico. Sistemas como OATutor, aunque efectivos para el seguimiento del dominio cognitivo mediante algoritmos adaptativos como el Bayesian Knowledge Tracing, dependen de estructuras de conocimiento rígidas y predefinidas que no permiten al docente configurar el comportamiento del tutor según el contexto específico de su curso (Pardos et al., 2023). Por otro lado, plataformas como CodeEdu incorporan agentes LLM especializados con capacidades de tutoría en tiempo real e informes de aprendizaje (Zhao et al., 2025), pero carecen de un módulo de seguimiento para el docente, limitando así la visibilidad del proceso de aprendizaje tanto a nivel individual como grupal. Finalmente, propuestas como Classroom Simulacra se enfocan en la simulación de comportamientos estudiantiles para optimizar estrategias de enseñanza (Xu et al., 2025), pero no ofrecen una herramienta directa de interacción entre el estudiante real y el sistema inteligente, ni un panel de retroalimentación basado en interacciones reales. En la revisión de la literatura realizada no se encontró un sistema que combine simultáneamente un diálogo persistente, una configuración pedagógica flexible por parte del docente y visibilidad del proceso de aprendizaje individual y grupal, lo que evidencia la necesidad del presente proyecto.

1.1.1. Formulación

A partir de los elementos expuestos en la descripción del problema, se plantea la siguiente pregunta de investigación, ¿Cómo desarrollar un prototipo de plataforma web que permita identificar falencias en el aprendizaje y generar retroalimentación mediante un sistema de tutoría inteligente, facilitando el seguimiento de los procesos de aprendizaje de los estudiantes de programación?

1.1.2. Sistematización

Con el fin de descomponer la pregunta de investigación en elementos abordables, se presentan las siguientes subpreguntas que orientan el diseño, desarrollo y evaluación del sistema:

1. ¿Qué requisitos funcionales y métricas de uso son necesarios para desarrollar una plataforma web que permita a los docentes realizar el seguimiento al aprendizaje y al comportamiento de los estudiantes durante el proceso de aprendizaje?

2. ¿Cómo diseñar la plataforma web mediante la estructuración de sus módulos?

3. ¿Cómo implementar una plataforma web que integre un sistema de tutoría inteligente (ITS) capaz de guiar la enseñanza de conceptos de programación y gestionar hilos de conversación persistentes para adaptar el progreso educativo al ritmo del estudiante?

4. ¿Cómo evaluar la usabilidad y aceptación del prototipo mediante encuestas de percepción aplicadas a sus usuarios?

1.2. Objetivos

1.2.1. Objetivo General

- Desarrollar un prototipo de plataforma web para la identificación de falencias en el aprendizaje y generación de retroalimentación mediante un sistema de tutoría inteligente socrático, que facilite el seguimiento del proceso de aprendizaje de los estudiantes de programación por parte de los docentes.

1.2.2. Objetivos Específicos

- Identificar los requisitos funcionales y las métricas de uso de la plataforma que permitan a los docentes el seguimiento al aprendizaje de los estudiantes de programación y su comportamiento durante el proceso de aprendizaje.
- Diseñar la plataforma web, mediante la estructuración de sus módulos.
- Implementar el prototipo de la plataforma web con un sistema de tutoría inteligente socrático basado en IA, configurado mediante técnicas de prompting, para el apoyo de las funciones de seguimiento y generación de retroalimentación pedagógica.
- Evaluar la usabilidad y aceptación del prototipo mediante la realización de encuestas de percepción sobre la plataforma.

1.3. Justificación

La creciente demanda de formación en lógica y fundamentos de programación en la educación superior ha puesto en manifiesto la necesidad de herramientas que trasciendan de la simple automatización de respuestas. En carreras de ingeniería, donde la abstracción es una barrera crítica, los sistemas inteligentes de tutoría han demostrado un impacto positivo al ofrecer asistencia adaptada a las necesidades individuales (Rodríguez Chávez, 2021; Zhou, 2025). Sin embargo, la mayoría de las herramientas actuales se limitan a formatos de evaluación cerrados o a la corrección superficial de sintaxis, dejando de lado la construcción profunda del conocimiento.

El presente proyecto se justifica por la necesidad de implementar un modelo de tutoría socrático impulsado por Inteligencia Artificial Generativa. A diferencia de los sistemas tradicionales, esta propuesta no busca entregar soluciones directas, sino guiar al estudiante mediante preguntas y

analogías que fomenten el aprendizaje significativo y que buscan romper la falsa retensión de conocimiento a través de preguntas como lo propone el método socrático (Farieta-Barrera & Murillo, 2023). Al integrar tecnologías modernas como Java Spring Boot y Supabase, el sistema permite gestionar la persistencia del diálogo, permitiendo que la IA mantenga el contexto histórico de las dificultades del alumno y adapte su discurso de manera evolutiva.

Por otra parte, desde una perspectiva técnica, la relevancia de esta investigación radica en el diseño de una plataforma web bajo el modelo C4, que asegure para cada estudiante su retroalimentación. Además, la implementación de este prototipo ofrece una solución a la "brecha de retroalimentación" (feedback gap) en cursos masivos.

Finalmente, este trabajo beneficia tanto a estudiantes como a docentes. Los primeros reciben un acompañamiento personalizado y constante que reduce la frustración y el deficiente desempeño. Los segundos obtienen una herramienta capaz de identificar patrones de error comunes, optimizando la labor pedagógica. Así, el proyecto sienta las bases para el uso de sistemas inteligentes que no solo informan, sino que enseñan a pensar, alineándose con las exigencias de innovación educativa en la formación de futuros ingenieros.

1.4. Alcances

- El presente proyecto contempla el desarrollo de un prototipo funcional de una plataforma web orientada a la identificación de falencias académicas y la generación de retroalimentación, cuya evaluación se realizará en el contexto de temas de Programación Orientada a objetos por su alto componente lógico y abstracto. El sistema integrará un sistema de tutoría inteligente (ITS) basado en Modelos de Lenguaje de Gran Escala (LLM), configurado mediante técnicas de prompting para guiar al estudiante en la construcción de su propio conocimiento y generar reportes útiles para el docente. A continuación se describen los alcances del proyecto:

  - **Requisitos Funcionales y Métricas:** Para dar cumplimiento al primer objetivo, se planea identificar los requisitos funcionales y las métricas de uso de la plataforma. En ese sentido, se planea hacer una revisión de la literatura o una aplicación de encuestas a docentes, lo que permitirá definir las capacidades técnicas necesarias del sistema y los indicadores necesarios para el monitoreo del proceso de aprendizaje de temas de Programación Orientada a Objetos.
  - **Diseño de Plataforma:** Según el segundo objetivo, se procederá con el diseño de la plataforma web mediante la estructuración de sus módulos funcionales utilizando el modelo C4 como marco de visualización de arquitectura. Esta etapa permitirá representar de manera jerárquica la organización de la plataforma, facilitando la comprensión de las interacciones entre los componentes, el flujo de datos y la integración de los servicios de inteligencia artificial.
  - **Implementación del Prototipo:** En relación con el tercer objetivo, se llevará a cabo la construcción del prototipo de la plataforma web, integrando un sistema de tutoría inteligente basado en inteligencia artificial. El núcleo del sistema, o el chat con IA,

se planea configurar mediante técnicas de prompting para lograr un comportamiento socrático, permitiendo así el soporte técnico necesario para las funciones de seguimiento estudiantil y la generación de retroalimentación pedagógica automatizada.

- **Evaluación:** En cumplimiento del cuarto objetivo, se evaluará la usabilidad y aceptación de la plataforma mediante la aplicación de encuestas de percepción basadas en el instrumento de usabilidad SUS (System Usability Scale) y el modelo de aceptación tecnológica TAM, dirigidas a estudiantes de Programación Orientada a Objetos. Los resultados permitirán identificar fortalezas y oportunidades de mejora del prototipo.

1.4.1. Delimitaciones y Entregables

- Para confirmar la viabilidad y el enfoque del proyecto, se establecen las siguientes delimitaciones:

  - **Enfoque de Inteligencia Artificial:** El proyecto no contempla la creación ni el entrenamiento desde cero de un modelo de inteligencia artificial. El "cerebro" del sistema inteligente de tutoría se apoyará en la conexión vía API a modelos LLM comerciales y preentrenados (como GPT o Claude), centrando el esfuerzo en la parametrización de su comportamiento pedagógico por medio de prompting.
  - **Prompts de Prueba:** Con el objetivo de evaluar el funcionamiento del sistema de tutoría inteligente, se entregarán los prompts de prueba utilizados durante la evaluación de la plataforma. Dichos prompts estarán orientados a temas de Programación Orientada a Objetos, que corresponde al contexto académico seleccionado para la evaluación del prototipo, y serán entregados junto con el documento final del proyecto y demás entregables.
  - **Rol del Sistema:** El prototipo se delimita a ser una herramienta de apoyo formativo. No está diseñado para emitir calificaciones definitivas ni para sustituir el criterio evaluativo del docente en la asignatura.
  - **Población y Entorno de Evaluación:** La evaluación del sistema se limitará a pruebas de aceptación y usabilidad en un entorno académico controlado dentro de la Pontificia Universidad Javeriana Cali.
  - **Estructura de la Plataforma:** Se planea entregar la documentación de la plataforma siguiendo los cuatro niveles del modelo C4: diagrama de contexto, de contenedores, de componentes y de código.
  - **Prototipo de Plataforma Web:** Se plantea entregar la aplicación funcional desarrollada en Java Spring Boot y desplegada en línea. Esta a su vez cual incluye los siguientes módulos: la interfaz de interacción socrática para el estudiante, el panel de visualización y seguimiento para el docente, el panel de configuración de prompts según el tema y la materia, y el panel de ayuda para la construcción de prompts asistida por IA. Adicionalmente, podrán incluirse otros módulos que se consideren necesarios tras el análisis de requisitos.

- **Repositorio de Código Fuente:** El acceso al código organizado se dará en el sistema de control de versiones (GitHub), incluyendo la configuración de la base de datos en Supabase y las funciones de backend.
- **Guía de Gestión Pedagógica para el Docente:** Documento técnico e instructivo orientado al docente, donde se detalla y explica como utilizar la plataforma.

1.5. Metodología

Se ha seleccionado un modelo de desarrollo incremental y evolutivo, el cual permite integrar funcionalidades de manera progresiva y ajustar la lógica del sistema de tutoría inteligente de forma iterativa. Este enfoque asegura la evaluación técnica de cada componente antes de su integración final en el prototipo. Bajo esta premisa, el proyecto se estructura en cuatro fases principales alineadas con el ciclo de vida del diseño de ingeniería:

1. **Análisis de Requisitos:** Identificar los requisitos funcionales y las métricas de uso de la plataforma que permitan a los docentes el seguimiento al aprendizaje de los estudiantes y su comportamiento durante el proceso de aprendizaje.
2. **Diseño de Plataforma:** Diseñar la plataforma web, mediante la estructuración de sus módulos por medio de un marco de visualización de arquitectura.
3. **Desarrollo e Integración de IA:** Implementar el prototipo de la plataforma web con un sistema de tutoría inteligente basado en IA, configurado mediante técnicas de prompting, para el apoyo de las funciones de seguimiento y generación de retroalimentación pedagógica.
4. **Evaluación del Sistema:** Evaluar la usabilidad y aceptación del prototipo mediante la realización de encuestas de percepción sobre la plataforma.

1.5.1. Tipo de Estudio.

El presente trabajo se define como un proyecto aplicado con un enfoque de diseño de ingeniería, centrado en la dimensión teórico-práctica del desarrollo de software. El estudio se orienta a la construcción y ejecución de una solución tecnológica, materializándose en la implementación de un prototipo funcional que integra servicios Inteligencia Artificial. Por su finalidad, se categoriza como investigación aplicada, ya que abarca desde la definición de requisitos técnicos y la ingeniería de prompts hasta la evaluación de la plataforma con encuestas de percepción, consolidando un sistema de información funcional que responde de manera directa a una necesidad educativa específica mediante inteligencia artificial.

1.5.2. Actividades

Se llevaran a cabo las siguientes actividades, alineadas con cada uno de los objetivos específicos del proyecto:

1. **Identificar los requisitos funcionales y las métricas de monitoreo de la plataforma.**
   - 1.1 Identificar los requerimientos funcionales del sistema.
   - 1.2 Determinar las métricas de seguimiento para el monitoreo del uso de la plataforma por medio del docente.
   - 1.3 Establecer los flujos de información para la persistencia de datos entre el front-end el backend y la API.
2. **Definir la estructura de la plataforma bajo el modelo C4 y el diseño de la interfaz.**
   - 2.1 Diseñar diagramas de Contexto y Contenedores para la integración de la IA.
   - 2.2 Elaborar diagramas de Componentes y Código para definir la lógica de la aplicación.
   - 2.3 Diseñar el esquema de la base de datos para el historial y métricas.
   - 2.4 Crear prototipos de interfaz para el chat y el panel del docente.
3. **Implementar el prototipo de la plataforma web con el sistema de tutoría inteligente.**
   - 3.1 Evaluar y elegir la API de IA que mejor se ajuste a los requerimientos técnicos del proyecto.
   - 3.2 Preparar la infraestructura de desarrollo y el entorno de despliegue para el software.
   - 3.3 Verificar que el frontend, el backend y los servicios de la API de IA se comuniquen correctamente antes de iniciar la programación de funciones avanzadas.
   - 3.4 Desarrollar la interfaz para que el profesor configure las instrucciones de la IA según el tema de programación a abordar.
   - 3.5 Construir el panel de visualización y monitoreo de estudiantes, en donde el profesor podrá observar de manera colectiva o individual como van avanzando los estudiantes por medio del uso de la plataforma.
   - 3.6 Construir el panel docente de prompting que ayude al profesor a estructurar el prompt que va a parametrizar el chat con IA para que brinde enseñanza socrática.
   - 3.7 Desarrollar las interfaces y la lógica del servidor necesaria para gestionar las conversaciones entre el estudiante y la IA.
4. **Evaluar el sistema.**
   - 4.1 Aplicar encuestas a estudiantes para medir qué tan fácil es usar la plataforma y qué tan útil la perciben para el aprendizaje.
   - 4.2 Procesar los datos obtenidos en las encuestas para identificar posibles mejoras en el prototipo final.

1.6. Cronograma

Mes	**1**				**2**				**3**				**4**			
Actividades / Semanas	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Fase 1: Requisitos.																
1.1 Identificación de requerimientos																
1.2 Determinación de métricas																
1.3 Flujos de datos y persistencia																
Fase 2: Arquitectura C4.																
2.1 Diseño Contexto y Contenedores																
2.2 Diseño Componentes y Código																
2.3 Esquema de base de datos																
2.4 Prototipado de interfaces																
Fase 3: Implementar el prototipo.																
3.1 Evaluar y elegir la API de IA																
3.2 Preparar infraestructura y entorno de despliegue																
3.3 Verificar conectividad front, back, API																
3.4 Implementación de Panel Prompting Docente																
3.5 Implementación de Panel Monitoreo Docente																
3.6 Implementación de Panel Ayuda Prompting Docente																
3.7 Implementación del Chat Socrático																
Fase 4: Evaluar el sistema.																
4.1 Aplicación de Encuestas de Percepción																
4.2 Análisis de Resultados																

Cuadro 1.1: Cronograma de Actividades del proyecto.

Capítulo 2

Marco Teórico y Trabajos Relacionados

2.1. Marco Teórico

Normatividad

Con el objetivo de orientar las decisiones de seguridad de la información en la plataforma propuesta, se toma como referencia la ISO/IEC 27001, con la cual se establecen algunos requisitos para implementar y mantener un sistema de gestión de seguridad de la información, como lo es la preserva la confidencialidad (International Organization for Standardization, 2022). En ese sentido se busca con la confidencialidad, que el desarrollo y despliegue de la plataforma cumpla con todos los requerimientos de seguridad y que solo el profesor y el estudiante tengan acceso a la información pertinente en los chats.

Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO), es un paradigma de programación que se conoce por un estilo de implementación en el que los programas se organizan como colecciones cooperativas de objetos, siendo cada uno, una representación de una instancia de una clase dentro de una jerarquía unida por relaciones (Almarales & Fernández, 2019). A su vez, este paradigma se fundamenta en principios fundamentales como la abstracción que se sostiene en la capacidad de modelar atributos y relaciones de entidades como clases para definir una representación abstracta de un sistema, el encapsulamiento que permite básicamente ocultar el estado interno y la funcionalidad de un objeto para poder permitir el acceso únicamente a través de un conjunto público de funciones, la herencia que es la capacidad para crear nuevas abstracciones basadas en abstracciones existentes y por último el polimorfismo que es la capacidad para implementar propiedades o métodos heredados de diferentes maneras en múltiples abstracciones (Microsoft Learn, 2026).

Esto desde el punto de vista de programación representa una ventaja importante en temas relacionados como la reducción de código redundante, lo que permite un código conciso y sin repeticiones, la reutilización y extensión de código a través de las clases y por último, permite la facilidad en el desarrollo y mantenimiento debido a la filosofía del paradigma fundamentado en clases (Almarales & Fernández, 2019).

Sistema Inteligente

Este se define como un sistema artificial que opera como un agente y exhibe un comportamiento racional en donde siempre interactua con su entorno mediante la percepción y la acción para realizar tareas de utilidad humana. Su racionalidad se manifiesta de dos formas: actuando racionalmente para maximizar las probabilidades de éxito en sus objetivos y pensando racionalmente al ser capaz de justificar sus conclusiones mediante procesos de razonamiento comprensibles. Estructuralmente, estos sistemas integran funciones de comprensión del entorno, toma de decisiones (ya sea de forma reactiva o deliberada) y comunicación, pudiendo actuar como delegados autónomos que ejecutan tareas por sí mismos o como asesores que proporcionan información experta al usuario. Finalmente, una característica clave es su capacidad de aprendizaje, que les permite mejorar su desempeño con la experiencia o adquirir nuevas creencias mediante la observación y la transferencia de conocimiento (Molina, 2022).

Sistema de Tutoría Inteligente

Un Sistema de Tutoría Inteligente (ITS) es una plataforma de aprendizaje adaptativo fundamentada en las ciencias del aprendizaje que actúa como un tutor personalizado para mejorar el rendimiento académico de los estudiantes. Su característica central y capacidad principal consiste en evaluar de manera continua las habilidades del alumno en componentes de conocimiento específicos para adaptar la instrucción y la práctica según sus necesidades individuales. Esto permite estimar el dominio cognitivo, lo que permite al tutor seleccionar problemas de forma inteligente, evitar la práctica excesiva o insuficiente y proporcionar apoyos pedagógicos inmediatos como pistas (hints) y retroalimentación instantánea sobre la corrección de las respuestas (Rodríguez Chávez, 2021). Es importante entender que el objetivo del proyecto es realizar un (ITS) que a su vez funciona como un sistema inteligente.

Plataformas Educativas

Las plataformas educativas son sistemas en línea diseñados para facilitar la enseñanza y el aprendizaje al ofrecer entornos virtuales con recursos interactivos y diversas actividades. Entre las más relevantes para este proyecto se destacan las Plataformas de Gestión del Aprendizaje (LMS), las cuales permiten administrar cursos y realizar un seguimiento del progreso de los alumnos (Quiñonez et al., 2023). El proyecto como tal adopta este enfoque al permitir que el docente defina las reglas de un curso mediante un prompt, para que luego el sistema guíe al estudiante y retroalimente al profesor.

Por otro lado, existen las plataformas de tutoría en línea, cuyo propósito original es conectar a los alumnos con tutores para recibir un apoyo personalizado (Quiñonez et al., 2023). En el contexto del proyecto, este rol de acompañamiento y guía académica en tiempo real será asumido directamente por el propio sistema de tutoría inteligente, automatizando la orientación personalizada.

Finalmente, también existen las plataformas de aprendizaje adaptativo que emplean algoritmos que recopilan información de los usuarios para ajustar las actividades según sus necesidades indi-

viduales (Quiñonez et al., 2023). Esta lógica se integra a la plataforma propuesta mediante el uso de inteligencia artificial, logrando que el sistema de tutoría inteligente se adapte dinámicamente a las capacidades y requerimientos específicos de cada estudiante.

Retroalimentación

En el contexto de la educación superior, la retroalimentación se identifica como un proceso de intercambio, siempre dinámico que busca reducir la brecha entre el desempeño actual del estudiante y las metas u objetivos ya establecidos para el aprendizaje. Para ello es importante que el estudiante no se considere solo como un receptor, sino un participante activo que hace parte del proceso dinámico y que así mismo se le ofrezcan oportunidades tempranas para recibir comentarios que puedan aplicarse en tareas futuras (Evans, 2013). En este punto, es importante entender que la retroalimentación es parte fundamental de esta investigación, dado que lo que se busca es que el profesor obtenga feedback del proceso que lleva el estudiante con el sistema de tutoría inteligente, para que este a su vez pueda ir estableciendo nuevos objetivos, metas o metodologías que puedan orientar el aprendizaje en los estudiantes.

Modelo C4

El modelo C4 es un marco de trabajo para la abstracción de arquitecturas de software que permite documentar sistemas de manera jerárquica. A diferencia de los diagramas UML tradicionales, este modelo progresa desde una visión general del sistema hacia una perspectiva más técnica, estructurándose en cuatro niveles principales de abstracción. Los dos primeros niveles son el Diagrama de Contexto, que muestra el sistema como una caja negra y su interacción con los usuarios, y el nivel de Contenedores, que desglosa el sistema en aplicaciones, bases de datos, microservicios o sistemas de archivos (Brown, 2022).

Los niveles restantes profundizan en la estructura interna: el nivel de Componentes descompone cada contenedor para mostrar sus módulos internos, tales como controladores o servicios, mientras que el nivel de Código detalla las clases y las diversas herramientas de programación implementadas (Brown, 2022). De esta manera, el modelo ofrece una transición fluida entre la conceptualización de alto nivel y el detalle técnico de la implementación.

Para el desarrollo del proyecto, el modelo C4 se utiliza específicamente para registrar los componentes de la plataforma. Su aplicación permite separar claramente los módulos del sistema de otros elementos clave que forman parte de la infraestructura del desarrollo, como lo es la base de datos implementada en Supabase.

Requisitos Funcionales

Los requisitos funcionales son una parte fundamental de la especificación de un sistema de software. Estos permiten definir las acciones fundamentales que debe realizar el software tanto al aceptar y procesar entradas como al generar las salidas y en ocasiones conviene separar los requisitos en subfunciones o subprocesos (IEEE Computer Society, 1998). Estas acciones fundamentales se expresan generalmente a través de declaraciones imperativas, utilizando la fórmula "el sistema deberá" para cada requisito identificado. Asimismo, estos requisitos deben especificar detalladamente las verificaciones de validez de las entradas, la secuencia exacta de las operaciones y el efecto que tienen los diferentes parámetros en el comportamiento del sistema (IEEE Computer Society, 1998).

Además, la especificación debe dejar clara la relación entre las salidas y las entradas, detallando tanto las secuencias de intercambio de información como las fórmulas necesarias para la conversión de datos (IEEE Computer Society, 1998). Es esencial que estos requisitos se planteen desde un punto de vista puramente externo, enfocándose exclusivamente en los servicios que el software prestará y evitando, por lo general, detalles de diseño interno como la elección de estructuras de datos o la división técnica del software en módulos. Por último, para asegurar la calidad del documento, cada requisito debe ser unívocamente identificable y estar organizado de manera que se maximice su legibilidad para el cliente (IEEE Computer Society, 1998).

API

Una API o "interfaz de programación de aplicaciones" es un mecanismo que permite la comunicación entre dos componentes mediante ciertos protocolo o definiciones. En ese sentido, existen dos términos importantes, cliente y servidor, siendo el cliente el usuario que normalmente solicita o realiza la solicitud al servidor. Entre los tipos de APIs mas comunes están las APIS REST, estas definen un conjunto de funciones como GET, PUT, y DELETE, etc. Por medio de los cuales el cliente y servidor pueden intercambiar solicitudes (Services, s.f.). Este mecanismo es importante, dado que se debe integrar por medio de la API un LLM que funcione como cerebro del sistema de tutoría inteligente que se pretenden entrenar por medio de prompts.

Inteligencia Artificial

La inteligencia artificial es una colección de procesos computacionales orientados a la generación de conocimiento sobre dominios específicos, buscando modelar la inteligencia y las acciones del cerebro humano (Monett & Lewis, 2018). En ese sentido, el texto identifica dos enfoques fundamentales: la IA simbólica, basada en la lógica y el razonamiento, y la IA estadística, la cual analiza grandes cantidades de información para extraer patrones correlacionales mediante mecanismos de aprendizaje automático (Monett & Lewis, 2018). Entre las características más comunes de estos sistemas está su capacidad para adaptarse al entorno y resolver tareas específicas, si bien se argumenta que la IA actual suele carecer de un "aprendizaje real" que implique una comprensión profunda o sentido común sobre la naturaleza de la información que procesa (Monett & Lewis, 2018). Por medio

de estas técnicas de computación epistémica o morfológica, el sistema procesa representaciones de datos para generar respuestas, clasificaciones o predicciones de utilidad práctica (Monett & Lewis, 2018). Este mecanismo tecnológico es de gran importancia para el proyecto, puesto que permite el desarrollo de un Sistema de Tutoría Inteligente capaz de realizar un seguimiento adaptativo que se ajusta en tiempo real a las capacidades y necesidades cambiantes de los estudiantes.

System Usability Scale (SUS)

El SUS es una herramienta de 10 items, creado por John Brooke en 1986 para evaluar la percepción subjetiva de la usabilidad de una interfaz de manera rápida y sin la necesidad de pruebas más sofisticadas. Esta a su vez se destaca por ser una herramienta gratis y de fácil utilización que ofrece resultados confiables sin importar el sistema en el que se aplique o las tareas a evaluar. Por otra parte, su diseño se fundamenta en la norma ISO 9241-11, que define la usabilidad a través de tres pilares principales: la eficacia (logro de la tarea), la eficiencia (rapidez en la ejecución) y la satisfacción (actitud positiva del usuario) (Picolo et al., 2025).

El cuestionario se estructura mediante diez preguntas graduadas en una escala tipo Likert de cinco puntos, que van desde "totalmente en desacuerdo" hasta "totalmente de acuerdo". Estos ítems abarcan diversos criterios esenciales para la calidad del software, evaluando aspectos como la complejidad del sistema, la necesidad de soporte técnico, la integración de sus funciones y la facilidad de aprendizaje de la interfaz. Esta amplitud permite obtener una perspectiva integral del usuario sobre el objeto de estudio de forma sencilla (Picolo et al., 2025). En este proyecto, el SUS permitirá medir si la interacción con el tutor socrático y el panel docente resulta funcional para los usuarios finales.

Technology Acceptance Model (TAM)

Es una teoría que explica por qué las personas deciden usar o no una nueva herramienta digital. Se basa principalmente en dos ideas: la utilidad percibida (el grado en que el usuario cree que el sistema mejorará su desempeño académico o docente), y la facilidad de uso (el grado en que el usuario considera que interactuar con la plataforma no requiere un esfuerzo excesivo). Si una persona siente que la herramienta es útil y sencilla, tendrá una actitud positiva y terminará usándola realmente (González-Bravo & Valdivia-Peralta, 2015).

Por otra parte, el modelo postula que la aceptación de las tecnologías de la información (TIC), se basa en una combinación de tres elementos secuenciales: una actitud positiva hacia la tecnología, la intención de utilizar el sistema y el uso real del mismo. Además, reconoce que en marcos organizacionales el uso de una herramienta no siempre es puramente voluntario, ya que los individuos pueden adoptar una tecnología debido a que sus superiores los animan o por la existencia de requerimientos externos para cumplir con su trabajo. (González-Bravo & Valdivia-Peralta, 2015). Finalmente en el contexto de la educación superior, el TAM se ha aplicado con éxito para evaluar la adopción del e-learning, identificando que la autoeficacia o creencia en la propia capacidad técnica es un factor crítico para explicar la conducta del estudiante. Asimismo, las investigaciones indican que existen variables moderadoras, como el sexo, que influyen significativamente en cómo

el personal académico y el estudiantado perciben y adoptan las nuevas plataformas educativas (González-Bravo & Valdivia-Peralta, 2015).

Método Socrático

El método socrático es una forma de diálogo que utiliza la refutación para cuestionar lo que se cree saber. Su primer objetivo es "negativo". Es decir, busca destruir la falsa pretensión de conocimiento a través de preguntas, partiendo de una posición de ignorancia. Lo que implica que no se trata de una simple transmisión de información, sino de una transformación personal a través de la investigación compartida (Farieta-Barrera & Murillo, 2023)

En este trabajo en equipo, el guía no actúa como si tuviera todas las respuestas. Al contrario, reconoce que él también tiene dudas y se siente "atascado" frente al problema, para que junto al estudiante se encuentre el deseo de aprender a cuidar su integridad y actuar correctamente. Por lo tanto, el aprendizaje resultante no consiste en la acumulación de datos o conocimientos positivos, sino en el desarrollo de un metaconocimiento, es decir, una nueva relación con el propio saber y sus límites (Farieta-Barrera & Murillo, 2023). Este método es importante, ya que se espera que por medio del prompting, el sistema de tutoría inteligente lo use para enseñar a los estudiantes.

Supabase

Supabase es una plataforma conocida por ser backend as a service de código abierto, construida sobre el sistema de gestión de base de datos Postgres. Esta plataforma, esta pensada para desarrollar aplicaciones web y móviles con un conjunto integrado de herramientas listas para usar. La documentación oficial lo resume así: "Supabase is an open source Firebase alternative built on Postgres. It provides a complete backend platform for building web and mobile applications, with a suite of integrated tools that work together out of the box." En la práctica, eso significa que no es solo una base de datos: incluye autenticación, almacenamiento, funciones y APIs generadas automáticamente (S. Inc., 2026). Para el Proyecto, esa herramienta se tiene planteada como la base de datos.

Java

Java es un lenguaje de programación y un entorno diseñado para resolver desafíos en la programación moderna, habiéndose originado en un proyecto para crear software avanzado destinado a dispositivos electrónicos de consumo pequeños, portátiles y distribuidos («The Java TM Language : An Overview», 2009). Es definido como un lenguaje simple, orientado a objetos, distribuido, robusto y seguro, que además posee una arquitectura neutral y alta portabilidad. Su diseño como sistema de arquitectura neutral permite que las aplicaciones se ejecuten en cualquier lugar de una red, ya que el compilador genera instrucciones de bytecode que son fáciles de interpretar y traducir a código de máquina en cualquier plataforma («The Java TM Language : An Overview», 2009).

Java también enfatiza la fiabilidad mediante la comprobación temprana de errores, la eliminación de la aritmética de punteros que suele corromper datos y la inclusión de un sistema de recolección

automática de basura que simplifica la gestión de memoria para el programador. Finalmente, sus capacidades multihilo y dinámicas permiten que las aplicaciones se adapten a entornos en evolución y gestionen múltiples tareas simultáneas con una respuesta interactiva eficiente («The Java TM Language : An Overview», 2009). Este lenguaje es importante, dado que es el elegido para el desarrollo del backend de la plataforma.

Spring Boot

Spring Boot es una extensión del Spring Framework diseñada específicamente para simplificar el desarrollo de aplicaciones y permitir un tiempo de inicio mucho más rápido al escribir software (Deinum, 2018). Se define por adoptar una visión "opinada" (opinionated) de la plataforma Spring y de bibliotecas de terceros, lo que facilita que los desarrolladores comiencen sus proyectos con el mínimo esfuerzo y muy poca configuración manual. Su diseño permite crear aplicaciones independientes y de grado de producción que simplemente se pueden "ejecutar", eliminando gran parte de la complejidad tradicional del despliegue (Deinum, 2018).

Este marco de trabajo enfatiza la eficiencia mediante la autoconfiguración, la cual detecta automáticamente la presencia de diversas infraestructuras en el entorno como JDBC, JPA, RabbitMQ o Spring Security para configurarlas con valores predeterminados (Deinum, 2018). Asimismo, Spring Boot facilita enormemente el empaquetado y la gestión de dependencias al permitir la creación de archivos JAR ejecutables (conocidos como "fat JARs") que contienen todo el software necesario para funcionar de manera autónoma (Deinum, 2018). Finalmente, sus capacidades para el monitoreo, como Spring Boot Actuator, ofrecen puntos finales de salud y métricas listos para producción que permiten supervisar el estado del sistema de forma interactiva y eficiente (Deinum, 2018). Esta tecnología es fundamental, dado que es la elegida para optimizar y agilizar el desarrollo del backend de la plataforma.

Docker

Docker es una plataforma abierta diseñada para el desarrollo, envío y ejecución de aplicaciones. Su principal ventaja es que permite separar el software de la infraestructura subyacente, lo que acelera notablemente los tiempos de entrega. Al adoptar las metodologías de Docker para probar y desplegar código, las organizaciones pueden gestionar su infraestructura de la misma manera en que administran sus aplicaciones, reduciendo al mínimo la brecha temporal entre la escritura del código y su puesta en marcha en entornos de producción (D. Inc., 2026).

Para lograr esto, la plataforma empaqueta y ejecuta las aplicaciones dentro de entornos aislados y ligeros llamados contenedores. Esta separación y seguridad permiten ejecutar múltiples contenedores de forma simultánea en un mismo servidor. Dado que los contenedores son autónomos e incluyen absolutamente todo lo necesario para que la aplicación funcione, eliminan la dependencia de lo que esté instalado en el equipo de hosting. Además, garantizan que al compartir un contenedor con otros miembros del equipo, este funcionará exactamente de la misma manera para todos (D. Inc., 2026).

Finalmente, Docker ofrece las herramientas necesarias para gestionar por completo el ciclo de vida de estos contenedores. El proceso comienza utilizando los contenedores para desarrollar la aplicación y sus componentes de soporte, convirtiendo al contenedor en la unidad estándar de distribución y prueba. Una vez que el software está listo, se puede desplegar en el entorno de producción ya sea como un contenedor individual o como un servicio orquestado, manteniendo el mismo comportamiento sin importar si se implementa en un centro de datos local, en la nube o en un entorno híbrido (D. Inc., 2026).

Método CRAFT

El método CRAFT es un marco de trabajo centrado especificamente en docentes, que se diseña con el objetivo de crear prompts para la utilización de herramientas de Inteligencia Artificial como ChaGPT, Geminis o Claude. Su finalidad, es proveer a los profesores, especialmente a aquellos sin experiencia técnica en programación o en creación de prompts, una estrategia accesible y adaptada a su propio lenguaje profesional, para poder diseñar materiales de forma efectiva (Joshi et al., 2026). En pocas palabras, CRAFT es un modelo estructurado bajo un acrónimo en inglés de cinco elementos esenciales para diseñar instrucciones:

- **Context (Contexto):** Define el entorno de instrucción, especificando el nivel de grado, la materia y el tema (por ejemplo, una clase de ciencias sociales del colegio) (Joshi et al., 2026).
- **Role (Rol):** Asigna a la IA una identidad profesional o experiencia educativa específica (por ejemplo, un maestro experimentado en ciencias de primaria), lo que guía al modelo para producir contenido pedagógicamente adecuado (Joshi et al., 2026).
- **Audience (Audiencia):** Detalla las características de los estudiantes a los que va dirigida la lección, facilitando la diferenciación y la adaptación según sus necesidades individuales, dominio del idioma o conocimientos previos (Joshi et al., 2026).
- **Format (Formato):** Determina la estructura en la que se requiere que la IA entregue los materiales, como un plan de lección, una hoja de trabajo o una guía de discusión (Joshi et al., 2026).
- **Tone (Tono):** Asegura que la voz del contenido generado sea atractiva, adecuada para el desarrollo de los alumnos y congruente con la cultura del aula (por ejemplo, un tono curioso y alentador) (Joshi et al., 2026).

Método RCFOR (Role, Context, Format, Output, Reflection)

Es un marco de ingniería de prompts ágil y ligero a la vez que se diseña como alternativa flexible al modelo CO-STAR. Su propósito principal se centra en optimizar la interacción con modelos de IA de manera dinámica y eficiente. Dentro de sus pilares fundamentales se encuentran los siguientes:

- **Estructura Modular de 5 Componentes:** Divide la instrucción en Rol (identidad/persona), Contexto (antecedentes), Enfoque (atención prioritaria), Objetivo (meta exacta) y Respuesta (formato y síntesis final) (Pham & Ngoc, 2025).
- **Adaptabilidad Dinámica:** Permite ajustar el prompt en tiempo real según el contexto de la tarea, a diferencia de los moldes estáticos tradicionales (Pham & Ngoc, 2025).
- **Optimización e Iteración Automática:** Implementa un bucle de retroalimentación basado en principios de aprendizaje por refuerzo que reajusta automáticamente el Contexto, Enfoque y Objetivo ante respuestas de baja calidad, disminuyendo la intervención humana (Pham & Ngoc, 2025).
- **Buen Rendimiento:** En pruebas con GPT-4o-mini, obtuvo una precisión de 0.91 (casi igualando el 0.92 de CO-STAR), demostrando que es altamente efectivo pero con una estructura mucho más simple y escalable (Pham & Ngoc, 2025).

Método CO-STAR

Es un marco de ingeniería de prompts que se estrucutra y formaliza en la contrucción de un conjunto de intrucciones con el objetivo de lograr que sean más claras. Este es ampliamente conocido por su capacidad para guiar las respuestas de los modelos a través de plantillas sistemáticas predefinidas con anterioridad. Su estrucutra esta basada en un formato de promp compuesto por seis elemntos fiijos:

- **Context (Contexto)**: Proporcionar información de fondo ayuda al gestor de flujos a comprender el escenario específico y ofrecer respuestas pertinentes. (Craig et al., 2024)
- **Objective (Objetivo)**: Definir claramente la tarea orienta al gestor de flujos hacia el logro de ese objetivo específico. (Craig et al., 2024)
- **Style (Estilo)**: Especificar el estilo de redacción deseado, como imitar a una personalidad famosa o a un experto profesional, guía al gestor de flujos para que su respuesta se ajuste a sus necesidades. (Craig et al., 2024)
- **Tone (Tono)**: Establecer el tono garantiza que la respuesta transmita el sentimiento requerido, ya sea formal, humorístico o empático. (Craig et al., 2024)
- **Audience (Audiencia)**: Identificar la audiencia objetivo permite que el gestor de flujos adapte la respuesta para que sea apropiada y comprensible para grupos específicos, como expertos o principiantes. (Craig et al., 2024)
- **Response (Respuesta)**: Proporcionar el formato de respuesta, como una lista o JSON, garantiza que el gestor de flujos genere la respuesta en la estructura requerida para las tareas posteriores. (Craig et al., 2024)

2.2. Trabajos Relacionados

OATutor (Open Adaptive Tutor)

Es un sistema de tutoría adaptivo de codigo abierto basado en el algoritmo Bayesian Knowledge Tracing (BKT) que permite estimar la maestría del estudiante en componentes de conocimiento específico. Su propósito principal consta de democratizar las investigaciones en ciencias del aprendizaje, permitiendo que diversos investigadores y educadores experimenten con algoritmos de aprendizaje adapatativo sin la necesidad de tener conocimientos técnicos para crear plataformas educativas desde cero. Este también cuenta con un portal docente integrado en sistemas como Canvas que permite a los profesores monitorear hacer un seguimiento de la probabilidad de maestría por habilidad (Pardos et al., 2023).

Classroom Simulacra

Es un marco de trabajo que utiliza agentes generativos basados en LLMs para crear una especie de gemelo digital (estudiante) del aula, que permita imitar comportamientos de aprendizaje reales y predecir con precisión las posibles respuestas en pruebas posteriores de los estudiantes. Este marco de trabajo se apoya en una plataforma integrada denominada "CogEdu" que emplea técnicas de detección multimodales como el seguimiento ocular y el análisis de expresiones faciales con el objetivo de monitorear en tiempo real los estados cognitivos como la confusión y el nivel de atención. Además, este se centra en un innovador módulo de Reflexión Iterativa Transferible (TIR), que no solo permite que los profesores realicen seguimiento del proceso, sino que utilizan la simulación como un entorno de pruebas para optimizar estrategias de enseñanza (Xu et al., 2025). Esta implementación es un poco distinta a lo que se busca en el proyecto, ya que esta simula estudiantes virtuales con el objetivo de analizar y optimizar la forma en que aprenden para que investigadores y personas relacionadas al sector educativo puedan encontrar metodologías más precisas para enseñar y personalizar la educación.

CodeEdu

Es una plataforma de educación fundamentada principalmente en programación que utiliza un sistema de colaboración multiagente contruido sobre LLMs para ofrecer un aprendizaje proactivo y supervisado. Este se diferencia de los demás principalmente en que descompone tareas complejas y utiliza agentes especializados con el objetivo de guiar al estudiante en cuatro funciones principales: generación de material personalizado, resolución de dudas en tiempo real, tutoría de código paso a paso con depuración y generación de informes de aprendizaje (Zhao et al., 2025). Esta plataforma a diferencia de las demás no plantea un apartado para que el docente pueda hacer seguimiento a los estudiantes. Por último, es importante recalcar que el trabajo de esta plataforma todavía requiere de la revisión por pares.

2.3. Diferenciadores del Proyecto

- A diferencia de las soluciones analizadas en los trabajos relacionados, el proyecto se diferencia por lo siguiente:

  - **Configuración del Entorno Pedagógico mediante Prompting:** A diferencia de sistemas como OATutor, que dependen de algoritmos rígidos y estructuras de conocimiento predefinidas, este proyecto otorga al docente el control sobre el comportamiento del tutor. A través de estrategias de prompting, el profesor puede definir el tono, el nivel de complejidad y el dominio específico de conocimiento, asegurando que el sistema de tutoría inteligente se alinee con las necesidades del curso.
  - **Tutoría Socrática frente a Resolución Directa:** A diferencia de plataformas que actúan como depuradores automáticos o generadores de soluciones, este sistema implementa un modelo de diálogo socrático. El sistema de tutoría inteligente no entrega el código corregido, en su lugar, utiliza analogías y preguntas orientadoras.
  - **Dualidad Funcional (Estudiante-Docente):** Mientras que CodeEdu carece de herramientas de seguimiento para el profesor y Classroom Simulacra se limita a la simulación de alumnos virtuales, este proyecto ofrece un ecosistema completo. Por un lado, brinda un entorno de práctica dinámico para el estudiante y, por otro, un panel de monitoreo para el docente que permite identificar áreas críticas de mejora en el grupo, facilitando la toma de decisiones pedagógicas basadas en datos reales de interacción.

Capítulo 3

Desarrollo de la Plataforma

3.1. Revisión de Literatura para la Definición de Tipo de Feedback Requerido

Con el objetivo de poder identificar los requerimientos funcionales de la plataforma, se hace necesario identificar los tipos de feedback necesarios para que sean aplicados por el ITS. En la literatura académica, el feedback se define básicamente como la información proporcionada por un profesor a un estudiante sobre su desempeño actual con respecto a un objetivo estándar por alcanzar. A su vez, esta definición busca resolver directamente la brecha de retroalimentación (feedback gap) identificada en el planteamiento del problema, donde se señala que el estudiante no logra avanzar hacia las metas de aprendizaje por la falta de guías claras y recursos personalizados (Evans, 2013). En otras palabras, lo que se busca es utilizar un tipo feedback bien diseñado que intente cerrar esa brecha.

Por otra parte, cuando se habla del contexto de programación, este proceso es considerado bien exigente para los estudiantes desde un punto cognitivo, ya que los mismos deben construir, probar y revisar código de forma iterativa. Proceso, que en muchas ocasiones no es tan sencillo por lo que se requiere de una guía constante para desarrollar modelos mentales precisos sobre la ejecución de los programas (Jung et al., 2026). De acuerdo con lo investigado en revisiones de la literatura, la retroalimentación puede clasificarse según su nivel de complejidad y propósito instruccional:

- **Verificación (V):** Es la forma más simple, indicando únicamente si el resultado es correcto o incorrecto (Castellanos et al., 2026).

- **Respuesta Correcta (CR):** Provee al estudiante la solución esperada, lo cual es común en autocalificadores que muestran el código "referencia" después de que el alumno falla. El riesgo pedagógico de este tipo de feedback se fundamenta en que si se entrega demasiado pronto, puede fomentar que el estudiante simplemente copie la respuesta en lugar de comprender los conceptos subyacentes, entrenándolo para "responder la pregunta" en lugar de aprender a programar (Castellanos et al., 2026).

- **Retroalimentación Elaborada (E):** Explica al estudiante por qué una respuesta es correcta o no. A diferencia de los tipos anteriores, no se limita a validar el resultado, sino que aporta información adicional cuyo nivel de detalle varía según el subtipo (Castellanos et al., 2026). Dentro de esta categoría, los subtipos más relevantes para este proyecto son:

- **Aislamiento de Atributos (AT):** Se enfoca en los conceptos o atributos específicos del tema de estudio, de modo que dirige la atención del estudiante hacia los elementos clave para mejorar su comprensión (Castellanos et al., 2026). Por ejemplo, en el contexto de la programación orientada a objetos, en lugar de decir "tu código falló", el sistema podría señalar "tu error está en la definición de la jerarquía de clases" o "revisa cómo estás usando el encapsulamiento". De esta manera, este tipo de feedback contribuye a reducir la carga cognitiva del estudiante, un beneficio que la literatura atribuye a la retroalimentación formativa en general (Castellanos et al., 2026)
- **Pistas o Señales (HC):** Funciona como una guía que orienta al estudiante hacia la respuesta correcta sin revelarla directamente (Castellanos et al., 2026). Por ejemplo, el sistema podría utilizar mensajes como "¿Has considerado qué sucede si la lista está vacía?" o "Revisa la condición de salida de tu ciclo while". De esta manera, las pistas dirigen la atención del estudiante hacia aspectos específicos del problema, lo cual promueve el pensamiento crítico (Jung et al., 2026). Sin embargo, la sola entrega de pistas no garantiza el aprendizaje, por lo que estas deben estar alineadas con los objetivos de la clase y apoyar el desarrollo de la autorregulación del estudiante (Jung et al., 2026)

- **Bugs o Conceptos Erróneos (BM):** Indica al estudiante los errores comunes que debe revisar y lo anima a explorar y analizar por qué se producen (Castellanos et al., 2026). Además, en la revisión de Castellanos et al. (2026), es el tipo de feedback que presenta el mayor resultado educativo medido. Este enfoque se fundamenta en la idea del componente de conocimiento, de modo que no trata el error como un simple fallo técnico, sino como la manifestación de una idea errónea en el entendimiento del estudiante (Gusukuma et al., 2018). Para lo anterior, se definen los siguientes elementos:
  - **Diferencia entre error y concepto erróneo:** Un error es una configuración incorrecta de los elementos del código, mientras que un concepto erróneo es una unidad de función o estructura cognitiva que puede inferirse a partir de dicho error (Gusukuma et al., 2018).
  - **Funcionamiento:** Los errores se detectan de forma automática mediante el análisis del programa y al cruzar varios de ellos, es posible aislar el concepto erróneo que los origina. Por ejemplo, si un estudiante no conoce la diferencia entre contar y sumar, el feedback le indica que el problema pide el número de elementos de la lista y no el total de sus valores (Gusukuma et al., 2018).
  - **Impacto:** La evidencia experimental indica que este enfoque acelera significativamente la adquisición de conocimiento conceptual y de habilidades prácticas de programación (Gusukuma et al., 2018).

Por otra parte, la retroalimentación automática ha evolucionado de manera considerable. Inicialmente, la mayoría de los sistemas utilizaban técnicas basadas en reglas, enfocadas en la revisión de sintaxis, la validación con casos de prueba y los mensajes de error estáticos. Sin embargo, con la llegada de los Modelos de Lenguaje de Gran Tamaño (LLM), el feedback

puede generarse en lenguaje natural, de forma conversacional y adaptada al contexto y al nivel de cada estudiante (Jung et al., 2026). Por ejemplo, estos sistemas permiten generar automáticamente comentarios que ayudan al estudiante a comprender la intención, la lógica y la estructura del código en tiempo real, lo cual mejora su legibilidad sin entregar la solución completa (Shekar et al., 2026). No obstante, para que este feedback sea útil, se recomienda que sea específico, breve y fácil de entender y que se adecúe al nivel actual de conocimientos del estudiante, ya que unas exigencias demasiado altas pueden afectar negativamente su motivación (Castellanos et al., 2026).

3.2. Requerimientos de un Sistema de Tutoría Inteligente

Para desarrollar un Sistema de Tutoría Inteligente (ITS) para la enseñanza de programación, la literatura académica sugiere que el sistema debe estructurarse bajo un marco de cinco componentes fundamentales: el modelo del dominio (qué enseñar), el modelo del tutor (cómo enseñar), el modelo del estudiante (a quién se enseña), la interfaz y las ganancias de aprendizaje. Un buen ITS debe ser capaz de actuar como un tutor humano, poseyendo un conocimiento profundo de la materia y la capacidad de inferir el estado cognitivo del alumno para ofrecer una instrucción personalizada. En cuanto al núcleo pedagógico, un ITS efectivo suele basarse en el constructivismo, donde el sistema no solo entrega información, sino que actúa como un guía o "coach"que interviene para llenar vacíos de conocimiento o remediar errores detectados (Colby, 2017). Las tácticas específicas que debe incluir un buen sistema son:

- **Retroalimentación Adaptativa:** El sistema debe ofrecer feedback que no solo indique si una respuesta es correcta o no (Castellanos et al., 2026), sino que sea capaz de analizar los pasos intermedios del proceso de resolución o el resultado final (Colby, 2017).
- **Secuencias de Pistas (Hinting):** Es un patrón común implementar secuencias de pistas que vayan de lo general a lo específico. Un buen ITS comienza con "pistas de puntero"que señalan un posible error sin dar la solución, seguidas de pistas de enseñanza y terminan con una "pista de fondo"que proporciona la respuesta para evitar que el estudiante caiga en un ciclo de frustración (Colby, 2017).
- **Modelado del Estudiante:** El sistema debe contar con un modelo como el Rastreo de Conocimiento Bayesiano (BKT), para inferir continuamente qué habilidades ha dominado el alumno a partir de sus interacciones. Esto permite que el ITS sugiera la siguiente acción más apropiada cuando el estudiante está bloqueado (Kurup et al., 2016).
- **Andamiaje (Scaffolding):** El ITS debe proporcionar apoyo intensivo al inicio y retirarlo gradualmente a medida que el estudiante gana competencia, adaptando la dificultad de las tareas a la "zona de desarrollo próximo"del alumno (Colby, 2017).

Finalmente, la efectividad del feedback también depende críticamente del momento de entrega. La retroalimentación inmediata o basada en la acción es la más prevalente, ya que

se encuentra en el 68,85 % de los sistemas revisados por Jung et al. (2026). Además, este tipo de retroalimentación ayuda a maximizar las oportunidades de aprendizaje y a minimizar el tiempo dedicado a seguir caminos de solución incorrectos (Colby, 2017). Sin embargo, la investigación sugiere que el feedback debe diseñarse no solo para corregir errores sintácticos o fallos en casos de prueba, sino también para fomentar procesos de aprendizaje de orden superior, como la autorregulación (Jung et al., 2026). Por lo tanto, el impacto positivo de estas intervenciones se refleja no solo en el aprendizaje de los estudiantes, sino también en un mayor compromiso con la programación y en la resiliencia frente a problemas complejos (Jung et al., 2026).

3.3. Definición de Requerimientos, Métricas, Módulos y Tecnologías de la Plataforma

A partir de la revisión de literatura realizada en las secciones anteriores, de una consulta con un asesor de innovación pedagógica y de preguntas hechas a los líderes de TI de la universidad, se definen los siguientes requerimientos funcionales, módulos y Tecnologías del prototipo de la plataforma.

3.3.1. Tecnologías de la Plataforma

Vale recalcar, que las tecnologías incialmente pensadas en el anteproyecto, se cambiaron dado que preguntando a los líderes de TI de la universidad, se utiliza un stack tecnológico diferente por lo que se busca estar lo más alineado posible a los lineamientos y tecnologías utilizados en la universidad. Estas son las siguientes:

Almacenamiento:

- Supabase (PostgreSQL)

Backend:

- Java (OpenJDK 21)
- Apache Maven
- Spring Boot

Frontend:

- Thymeleaf

Infraestructura:

- Docker

Despliegue/Hosting:

- Servidor Virtual Privado

- **Inteligencia Artificial:**
  - API de Gemini (Gemini 3 Flash y Gemini 3.1 Flash-Lite como respaldo)
- **Dependencias de Spring Boot:**
  - **Spring Web MVC:** maneja las peticiones HTTP mediante los controladores de la aplicación.
  - **Thymeleaf:** genera las vistas HTML desde el servidor.
  - **Spring Data JPA:** gestiona el acceso a la base de datos PostgreSQL mediante repositorios.
  - **Spring Security:** controla la autenticación, los roles de profesor y estudiante y el cifrado de contraseñas con BCrypt.
  - **Jackson:** procesa los datos en formato JSON, como las respuestas de la API de Gemini y la encuesta SUS+TAM.
  - **ZXing:** genera el código QR para la autenticación en dos pasos del profesor.
  - **Lombok:** reduce el código repetitivo de las clases, como los constructores y los métodos de acceso.
  - **Driver de PostgreSQL:** permite la conexión con la base de datos de Supabase.

3.3.2. Módulos de la Plataforma

La siguiente sección comprende la descripción de las funcionalidades de los módulos planteados en apartados anteriores y otros ajustados o agrupados con lo investigado o preguntado a los stakeholders:

- **Interfaz de Interacción Socrática:** Este módulo comprende el chat con IA en donde el estudiante hará la interacción con el objetivo de aprender cualquier tema de programación organizado por el profesor. Además, la idea es que el estudiante pueda enviar feedback sobre su interacción, por lo que se habilitan dos botones en la parte superior derecha, el primero es el botón de **Feedback Profe** (de color amarillo) en donde se puede calificar la interacción de 1-5 y agregar un comentario de mejora para la plataforma. La segunda, con el objetivo específico de poder evaluar la plataforma para el proyecto de grado, se agrega el botón **Evaluar Plataforma (SUS+TAM)** (de color morado).

En él estudiante responde los cuestionarios basados en los modelos SUS y TAM contemplados en el marco teórico, para así evaluar los objetivos planteados en el proyecto. La tutoría se da por finalizada cuando el estudiante completa la ruta de preguntas guía o envía esta encuesta.

- **Panel de Métricas Docente:** En este módulo se presentan los cuatro indicadores de seguimiento (M-01 a M-04), que le permiten al profesor conocer el avance de los estudiantes en una sesión específica. Para facilitar su interpretación se utiliza IA de dos formas. La primera es un informe pedagógico individual por estudiante, que se genera automáticamente cuando el estudiante completa la ruta. Para los otros casos, el profesor puede generarlo de forma individual o con el botón **Generar informes pendientes**. La segunda es una Síntesis Pedagógica Consolidada del grupo. Desde este panel el profesor también puede editar la configuración CRAFT de la clase y vincular o desvincular estudiantes. El profesor accede a él abriendo la sesión correspondiente dentro del Módulo de Administración de Profesor. Por último, el profesor puede ver las preguntas guías que se plantean como hoja de ruta de la interacción, ver la lista de estudiantes que han ingresado y los focos de dificultad generales.
- **Panel de Configuración de Prompts con IA:** En consideración de lo conversado con el asesor pedagógico, para un profesor puede ser difícil construir un buen prompt. Por ello, este panel combina el método socrático (Chang, 2023) con el framework CRAFT (Joshi et al., 2026), en donde el profesor ingresa el tema y con el botón **Sugerir Directrices CRAFT con IA**, obtiene una propuesta de contexto, rol, audiencia, formato y tono, que luego puede ajustar. Además, define las preguntas guía que conforman la ruta que seguirá el estudiante. Este apartado unifica los módulos "Panel de Configuración de Prompts" y "Panel de Ayuda para Construcción de Prompts" planteados en secciones anteriores. Es exclusivo del profesor y se encuentra dentro del Módulo de Administración de Profesor.
- **Módulo de Inicio de Sesión:** Es el módulo correspondiente al inicio de sesión tanto para estudiantes como para profesores. En el caso de los profesores, estos pueden activar la autenticación de dos factores para mayor seguridad. En cuanto al registro, los estudiantes crean su cuenta la primera vez que ingresan mediante el enlace de la clase, utilizando su correo institucional, mientras que las cuentas de los profesores se crean previamente en la plataforma.
- **Módulo de Administración de Profesor:** Es el módulo principal del docente. Desde él puede crear sesiones de tutoría (accediendo al Panel de Configuración de Prompts con IA), pausarlas, reanudarlas o eliminarlas y abrir cada una para ver su **Panel de Métricas Docente**. También cuenta con la sección **Mis Estudiantes**, donde consulta a los estudiantes vinculados, sus clases, su tiempo activo y su última actividad y con la sección de **Feedback Recibido**, donde revisa las valoraciones y comentarios de los estudiantes y sus evaluaciones SUS+TAM. Por último, puede ver y editar su información personal y activar o desactivar la autenticación de dos factores.

- **Módulo de Interfaz de Estudiante:** Esta interfaz le permite al estudiante visualizar su información personal y la lista de clases a las que está vinculado, cada una con su estado (completada, activa o pausada). Desde aquí puede editar su información personal, retomar sus tutorías o evaluarlas con los dos botones descritos en la **Interfaz de Interacción Socrática**. En esta interfaz los botones aparecen organizados por clase en forma de lista, por lo que su posición varía respecto al chat. Una vez enviada la encuesta, el botón muestra el puntaje SUS obtenido.

3.3.3. Requisitos Funcionales

Cuadro 3.1: Requisitos Funcionales de la Plataforma

ID	Módulo	Descripción	Fuente
RF-01	Módulo de Interfaz de Estudiante, Módulo de Inició de Sesión, Módulo de Administración de Profesor.	El docente crea una sesión de tutoría, que automáticamente genera un enlace único de acceso a la sesión. Al abrirlo, el estudiante completa un registro mínimo con su nombre, correo institucional y contraseña, quedando vinculado automáticamente a la sesión sin pasos adicionales.	Stakeholder
RF-02	Módulo de Interfaz de Estudiante, Interfaz de Interacción Socrática.	El sistema debe permitir que el estudiante interactúe con el ITS mediante un chat conversacional que aplique el método socrático, entregando retroalimentación elaborada a través de pistas (HC). El ITS no debe revelar la solución directa, sino guiar al estudiante para que la descubra por sí mismo a través del diálogo.	(Castellanos et al., 2026; Chang, 2023)
RF-03	Interfaz de Interacción Socrática.	El sistema debe entregar retroalimentación inmediata tras cada interacción del estudiante, apoyando los ciclos rápidos de ensayo y error esenciales para el aprendizaje de programación.	(Castellanos et al., 2026; Jung et al., 2026)
RF-04	Interfaz de Interacción Socrática.	El sistema debe almacenar el historial de conversación de cada estudiante en la sesión, de modo que si aquel estudiante se sale anticipadamente de la clase y vuelve a ingresar, el ITS retome lo trabajado, reconozca los errores pasados en la conversación y adapte su discurso a ese contexto.	(Colby, 2017; Kurup et al., 2016)

Continúa en la siguiente página

Cuadro 3.1 – Continuación

ID	Módulo	Descripción	Fuente
RF-05	Panel de Configuración de Prompts con IA.	El docente debe poder configurar el comportamiento socrático del ITS mediante un paso a paso estructurado ayudado con IA, que permita definir el tema, el nivel de complejidad y el tono del tutor, sin necesidad de conocimientos técnicos previos en ingeniería de prompts.	Stakeholder (Kurup et al., 2016)
RF-06	Panel de Métricas Docente.	La plataforma debe mostrar al docente los conceptos con mayor frecuencia de error a nivel individual y grupal, permitiéndole identificar las falencias más críticas del grupo.	(Almarales & Fernández, 2019; Gusukuma et al., 2018)
RF-07	Panel de Métricas Docente.	La plataforma debe generar un resumen automático en lenguaje natural que interprete las métricas del grupo y sugiera al docente acciones pedagógicas concretas, sin necesidad de que el profesor interprete los datos por su cuenta.	Stakeholder
RF-08	Panel de Métricas Docente.	El sistema debe registrar el número de sesiones, la duración promedio y la frecuencia de uso por estudiante, para que el docente pueda identificar estudiantes en riesgo de abandono o desenganche.	Stakeholder
RF-09	Interfaz de Interacción Socrática, Módulo de Interfaz de Estudiante	La plataforma debe poder permitirle al estudiante enviar feedback al profesor sobre la sesión realizada.	Stakeholder
RF-10	Módulo de Administración de Profesor.	La plataforma debe poder permitirle al profesor ver el feedback que el estudiante envía al acabar la sesión.	Stakeholder

3.3.4. Requisitos No Funcionales

Los siguientes dos requisitos no funcionales (RNF), están enfocados principalmente en preservar la confidencialidad de la información y la protección de los datos personales de los usuarios de la plataforma.

Cuadro 3.2: Requisitos No Funcionales de la Plataforma

ID	Módulo	Descripción	Fuente
RNF-01	Interfaz de Interacción Socrática, Módulo de Administración de Profesor.	El sistema deberá restringir el acceso a las conversaciones de un estudiante exclusivamente a dicho estudiante y al docente responsable de la sesión correspondiente, de modo que solo el estudiante pueda acceder a su chat de tutoría y solo dicho docente pueda consultar el feedback y los informes derivados de esa conversación.	(International Organization for Standardization, 2022)
RNF-02	Módulo de Inicio de Sesión, Módulo de Interfaz de Estudiante, Interfaz de Interacción Socrática, Módulo de Administración de Profesor.	El sistema deberá almacenar únicamente los datos personales mínimos necesarios para la operación, tales como nombre, correo institucional y contraseña, los cuales el estudiante registra una única vez al ingresar por primera vez en la plataforma.	(International Organization for Standardization, 2022)

3.3.5. Métricas de Seguimiento

Las siguientes métricas serán recolectadas por la plataforma para alimentar el Panel de Métricas Docente. En ese sentido cada métrica está acompañada de su traducción pedagógica, es decir, lo que significa en términos prácticos para el profesor:

Cuadro 3.3: Métricas de Seguimiento de la Plataforma

ID	Métrica	Descripción técnica	Traducción pedagógica	Fuente
M-01	Frecuencia de error por concepto	Número y porcentaje de estudiantes del grupo que presentan un error recurrente sobre un concepto durante una sesión.	"Este es el tema que más le está costando al grupo. Considera dedicarle más tiempo en clase."	(Colby, 2017; Gusukuma et al., 2018)
M-02	Intentos por concepto	Número de turnos que el estudiante se toma en cada pregunta guía hasta que el tutor la da por superada.	"Este estudiante necesita muchos intentos para entender el concepto. Puede requerir atención personalizada."	(Colby, 2017; Kurup et al., 2016)
M-03	Tasa de solicitudes de respuesta directa	Número de veces que el estudiante pide explícitamente la solución durante la sesión y su porcentaje sobre sus consultas.	"Este estudiante muestra señales de frustración. Puede necesitar una estrategia diferente."	(Colby, 2017)
M-04	Frecuencia y duración de sesiones	Número de sesiones completadas, tiempo activo promedio por sesión y días transcurridos desde la última actividad de cada estudiante.	"Este estudiante no ha usado la plataforma en varios días. Puede estar en riesgo de abandono."	(Almarales & Fernández, 2019; Colby, 2017)

3.4. Decisiones de Diseño de la Arquitectura de Prompts y Persistencia

3.4.1. Arquitectura de Prompts y Persistencia de Datos

Con el objetivo de definir un flujo de datos consistente con los objetivos en la plataforma, es necesario evaluar distintos enfoques para estructurar el prompt que recibe el modelo de lenguaje en cada interacción con el estudiante. A continuación se documentan las decisiones tomadas respecto al framework de **prompting** que se espera utilizar y la justificación detrás de su elección.

Arquitectura en Capas frente a un Prompt Monolítico

Inicialmente se consideró construir el prompt del sistema como un único bloque de texto que combinara las reglas socráticas de comportamiento del tutor, la configuración particular del docente y el historial de progreso del estudiante. Sin embargo, este enfoque presenta dos limitaciones importantes. La primera se da en la dificultad del mantenimiento del sistema, ya que cualquier ajuste a las reglas pedagógicas obligaría a revisar un solo bloque de texto extenso. La segunda, es que no permite que el docente configure la sesión sin riesgo de alterar accidentalmente las restricciones fijas del método socrático.

Por esta razón, se decidió estructurar el prompt en tres capas independientes, cada una con un origen, un propósito y una frecuencia de cambio distintos. La primera es una capa fija que define el comportamiento socrático del sistema, la segunda es una capa configurable que el docente ajusta por sesión y la última, aunque no es un prompt en sí, es una capa dinámica que contiene el historial de la conversación del estudiante en la sesión actual. Esta separación permite que cada capa se mantenga, audite y modifique de manera independiente, sin comprometer las demás. Es importante aclarar que, cuando se menciona cada elemento como una capa, se habla de bloques de texto que se unen para formar las instrucciones enviadas al modelo. Lo que cambia es su origen (un archivo fijo, la configuración del docente o el historial de mensajes) y si el docente la configura o no.

Capa Fija: Método Socrático

La primera capa contiene las reglas de comportamiento que el tutor debe seguir en todas las sesiones, sin importar el tema o el estudiante. Esta capa no es visible ni modificable por el docente, ya que su función es garantizar que el sistema mantenga siempre el enfoque pedagógico central del proyecto, es decir, guiar al estudiante mediante preguntas y analogías en lugar de entregar soluciones directas, tal como lo plantea el método socrático (Chang, 2023).

Se decidió mantener esta capa separada de la configuración del docente precisamente para evitar que un ajuste mal formulado en el panel de configuración pudiera debilitar o eliminar las restricciones pedagógicas fundamentales del sistema. De esta manera, el docente tiene control sobre el contenido y el enfoque temático de la sesión, pero no sobre los principios pedagógicos que sostienen el comportamiento socrático del tutor. Además, esta capa impide que el tutor revele o modifique sus propias instrucciones, aun cuando el estudiante lo solicite explícitamente.

Capa Configurable: Adopción del Marco CRAFT

Para la capa que el docente configura por sesión, se evaluaron tres frameworks de **prompting** orientados al diseño de instrucciones estructuradas, **CRAFT** (Joshi et al., 2026), **CO-STAR**

(Craig et al., 2024; Pham & Ngoc, 2025) y **RCFOR** (Pham & Ngoc, 2025). Tras comparar la naturaleza y el propósito de cada uno, se optó por adoptar **CRAFT** (Context, Role, Audience, Format, Tone) como base de esta capa, dado que es el único de los tres frameworks diseñado y validado específicamente para docentes sin experiencia técnica en ingeniería de prompts. Su validación empírica con 94 docentes de educación básica, que generaron 688 prompts reales para la creación de planes de clase y material instruccional, lo posiciona como la opción con mayor evidencia de usabilidad para el perfil de usuario al que se dirige el módulo de configuración de sesión de la plataforma (Joshi et al., 2026).

Esta elección no descarta los aportes de los otros dos frameworks evaluados, sino que reconoce su alcance distinto. **CO-STAR**, con sus seis componentes (Context, Objective, Style, Tone, Audience, Response), presenta el desempeño más alto en tareas de generación de contenido y clasificación bajo evaluación experimental directa frente a RCFOR, con métricas superiores en precisión, relevancia y coherencia (Pham & Ngoc, 2025). Sin embargo, ni CO-STAR (Craig et al., 2024) ni RCFOR (Pham & Ngoc, 2025) cuentan con validación en un contexto pedagógico ni con usuarios docentes reales. Ambos fueron evaluados sobre tareas genéricas de procesamiento de lenguaje natural, lo cual limita su pertinencia directa para el módulo de configuración dirigido al profesor. **RCFOR**, por su parte, destaca por la separación explícita de su componente Focus respecto al componente Objective, una característica estructural que podría resultar valiosa si se quisiera verificar de forma aislada si la restricción de no revelar soluciones directas se cumplió, sin necesidad de reconstruir el prompt completo (Pham & Ngoc, 2025). No obstante, dado que dicha restricción ya se encuentra respaldada de forma centralizada en la capa fija del sistema, esta separación no resulta indispensable y no justifica adoptar RCFOR como framework principal de la capa configurable.

En consecuencia, la plataforma implementa CRAFT (Joshi et al., 2026) como la estructura que el docente completa a través del formulario de configuración de sesión, apoyado por la opción **Sugerir Directrices CRAFT con IA**, que propone un borrador de cada componente a partir del tema de la clase. La responsabilidad de restringir la generación de soluciones directas recae en la capa fija del sistema, descrita en la sección anterior, y no en un componente específico de la capa configurable. Adicionalmente, se incorporó a esta capa una tarea para el profesor, definir un conjunto de preguntas guía que delimitan lo que el estudiante debe comprender en la sesión. Aunque CRAFT le proporciona al tutor el contexto del tema, sin una ruta concreta el tutor tiende a divagar y a no concentrarse en lo que el docente, desde su experiencia, considera esencial. Las preguntas guía no reemplazan la labor del tutor, que sigue brindando retroalimentación y respondiendo las dudas que le surjan al estudiante durante su progreso, sino que le dan una secuencia a seguir. Además, sobre ellas se mide el avance del estudiante (M-02), y al completarlas la tutoría se da por finalizada y se genera automáticamente su informe pedagógico.

Capa Dinámica

La capa dinámica del prompt corresponde al historial completo de la conversación del estudiante en la sesión activa. A diferencia de las dos capas anteriores, no es un texto redactado previamente, sino que se construye en cada turno a partir de los mensajes almacenados en la base de datos para esa participación, identificando cada uno según su emisor (estudiante o tutor). De esta manera, el tutor conserva la continuidad de lo trabajado aun cuando el estudiante abandona la tutoría y la retoma más adelante, y puede dar seguimiento a su evolución dentro de la ruta de preguntas guía definida por el docente.

Es importante aclarar que esta capa no incorpora el progreso cualitativo del estudiante **progreso_md** ni sus métricas. Se decidió así porque dicho progreso constituye el diagnóstico dirigido al docente, y su inyección a mitad de la tutoría podría condicionar el comportamiento del tutor con un juicio previo sobre el estudiante. Cada sesión parte, por tanto, únicamente de su propia conversación.

El historial de mensajes cumple además una segunda función, constituye la base sobre la cual la API genera el informe pedagógico individual que se almacena en **progreso_md**. Este informe se genera automáticamente en segundo plano cuando el estudiante completa la ruta de preguntas guía y para quienes no la completan, el docente puede generarlo desde el panel de métricas, ya sea de forma individual o para todos los estudiantes pendientes en una sola acción. Con el fin de que su contenido sea consistente entre estudiantes y pueda interpretarse de forma predecible, el análisis generado por la API debe seguir estrictamente la siguiente estructura en formato Markdown:

```
## Conceptos dominados
- [Concepto 1]
- [Concepto 2]

## Conceptos en progreso
- [Concepto 1]
- [Concepto 2]

## Errores recurrentes
- **[Concepto]:** [Descripción breve del error]

## Recomendación para la siguiente sesión
- [Recomendación puntual y accionable]
```

En la sección de errores recurrentes, cada error debe iniciar con un título corto en negrita, de dos a cinco palabras, que nombre únicamente el concepto mal entendido utilizando los términos de las preguntas guía de la sesión. Si no se presentaron errores, se registra únicamente "Ninguno". Esta estructura fija garantiza que todos los informes conserven las mismas cuatro secciones, lo que permite que el docente los interprete de forma consistente en el panel de

monitoreo y que la plataforma pueda comparar los títulos de los errores entre estudiantes para identificar los conceptos que más le cuestan al grupo (M-01).

Esquema de Persistencia del Progreso, Métricas y Preguntas Orientadoras

Antes de definir el esquema final de persistencia, se evaluaron tres alternativas para almacenar el progreso del estudiante y las métricas de seguimiento del sistema. La primera alternativa consistía en utilizar tablas estructuradas en Supabase, con columnas específicas para cada dato del análisis, como el concepto trabajado, si el estudiante entendió al final, el nivel de comprensión, el número de intentos y los errores recurrentes. Esta alternativa facilita el filtrado y la consulta directa mediante SQL en el panel docente, pero exige reconstruir manualmente un texto legible a partir de los valores de cada columna y obliga a modificar la estructura de la tabla cada vez que cambia el tipo de análisis requerido.

La segunda alternativa consistía en almacenar el análisis generado por la API directamente como una cadena de texto en formato JSON, dentro de una única columna de tipo texto. Esta alternativa permite que el modelo devuelva la información ya estructurada sin que el backend deba construirla manualmente y ofrece mayor flexibilidad ante cambios en el análisis, dado que no requiere alterar el esquema de la base de datos. Sin embargo, exige un paso adicional de deserialización mediante una librería como Jackson antes de poder utilizar la información y expone al sistema al riesgo de recibir una respuesta con un formato JSON mal construido por parte del modelo.

La tercera alternativa consistía en almacenar el análisis generado por la API como texto plano con sintaxis Markdown, en una columna de tipo texto. Esta alternativa permite que el contenido se muestre directamente al docente sin ningún tipo de transformación adicional y resulta igualmente legible tanto para el modelo que lo genera como para una persona que lo revise. Su principal limitación radica en que resulta más difícil de filtrar o consultar de forma selectiva desde el panel docente, en comparación con columnas estructuradas.

Con base en esta comparación, se optó por adoptar una combinación entre la primera y la tercera alternativa, descartando el uso de JSON como formato para el progreso del estudiante. Los eventos de seguimiento que alimentan las métricas descritas en la Sección 3.3.5 (Cuadro 3.3) se almacenan en la tabla **metrica** como registros estructurados de tipo código–valor, vinculados a la participación del estudiante en una sesión. Spring Boot los registra en tiempo real con cada interacción, el total de mensajes de la conversación, cada solicitud de respuesta directa (M-03) y la apertura y superación de cada pregunta guía (M-02). Las métricas restantes se calculan a partir de datos ya almacenados, de modo que M-01 se obtiene de la sección de errores recurrentes de los informes y M-04 de las marcas de tiempo de los mensajes, evitando así duplicar información. Este formato permite consultas directas mediante SQL y facilita la construcción de indicadores y filtros en el panel de métricas docente, sin requerir modificar el esquema cuando se incorpora un nuevo tipo de evento.

El progreso cualitativo del estudiante, en cambio, se almacena como texto con sintaxis Markdown en la columna **progreso_md** de la tabla **participacion**, lo que da lugar a un informe independiente por estudiante y por sesión. Este texto es generado por la API a partir del análisis completo de la conversación, ya sea automáticamente cuando el estudiante completa la ruta de preguntas guía o a solicitud del docente, y contiene cuatro secciones predefinidas: conceptos dominados, conceptos en progreso, errores recurrentes y recomendación para la siguiente sesión.

La elección de Markdown sobre JSON para este informe responde a cuatro razones. En primer lugar, la tolerancia a errores del modelo, un JSON exige una sintaxis exacta, por lo que basta con que el modelo omita una comilla, una llave o una coma para que la deserialización falle y el informe completo se pierda. En cambio, un Markdown con una imperfección de formato sigue siendo legible y aprovechable. En segundo lugar, el uso principal del informe, su destinatario es el docente, y el texto en Markdown puede presentarse directamente en el panel sin un paso intermedio de deserialización mediante librerías como Jackson ni la reconstrucción de un texto a partir de campos separados. En tercer lugar, la estructura necesaria ya está garantizada, al exigir cuatro secciones fijas con encabezados definidos, la plataforma puede ubicar de forma sencilla la sección de errores recurrentes y extraer sus títulos para calcular M-01, sin requerir la estructura formal de un JSON. Por último, la necesidad de filtrado y consulta que motivaba el uso de JSON quedó cubierta por los registros estructurados de la tabla **metrica**, por lo que mantener una representación adicional del progreso en JSON introduciría complejidad sin un beneficio claro.

Cabe aclarar que el formato JSON sí se conservó para almacenar las respuestas de la encuesta SUS+TAM (**evaluacion_sus_tam**). La diferencia radica en el origen del dato: estas respuestas no las genera el modelo, sino la propia plataforma a partir de un formulario con un conjunto fijo de valores numéricos, por lo que no existe el riesgo de recibir un JSON mal construido y sí se aprovecha su estructura para calcular directamente los puntajes SUS y TAM.

Por último, es importante precisar que, aunque este contenido se describe en formato Markdown, no corresponde a un archivo físico almacenado en el sistema de archivos del servidor, sino a una cadena de texto almacenada como el valor de una columna de tipo texto en una fila de la base de datos relacional. Esta distinción resulta relevante desde el punto de vista de la arquitectura de despliegue, dado que el contenedor de Docker en el que se ejecuta la aplicación es efímero y cualquier archivo escrito directamente en el sistema de archivos se perdería al reiniciarse o reconstruirse el contenedor, mientras que la información almacenada en Supabase persiste de manera independiente al ciclo de vida del contenedor de la aplicación.

3.4.2. Definicion de API de IA a Utilizar

En este apartado se realiza el análisis de las APIs de IA disponibles para utilizar. Para ello se utiliza la página Artificial Analysis (Artificial Analysis, 2026), una plataforma indepen-

diente de evaluación comparativa de modelos de IA que no pertenece a ningún proveedor particular, por lo que goza de mayor objetividad. Asimismo, esta página cuenta con un índice compuesto denominado Artificial **Analysis Intelligence Index**, que integra un conjunto de evaluaciones de rendimiento, entre las que se encuentran GDPval-AA v2, Terminal-Bench v2.1, SciCode, Humanity's Last Exam, GPQA Diamond, CritPt, AA-Omniscience, AA-LCR y LiveCodeBench.

Por otra parte, dado que las APIs en su mayoría requieren de pago por su uso y el proyecto se costea con recursos propios, la comparación se hace escogiendo modelos representativos de cada uno de los principales proveedores comerciales, con la consideración de siempre eligir la mejor relación entre capacidad y costo para el desarrollo que se esta haciendo. En ese sentido se eligen los siguientes modelos para la evaluación, Gemini 3 Flash (Google), GPT-5 mini (high) (OpenAI), Grok 4 Fast (xAI) y Claude 4.5 Haiku (Anthropic).

Como se puede ver en la figura (3.1) del **Analysis Intelligence Index**, Gemini 3 Flash de Google gana con una puntuación de 39 puntos, seguido de Claude Haiku con 30 y Grok y ChatGpt con 28 y 26 puntos, una puntuación casi similar, que pone dentro de los modelos de gama media-baja a Gemini 3 Flash unos puntos por arriba y que al mismo tiempo lo adecua dentro de las necesidades del proyecto dado que se valida su buen rendimiento general en **benchmarks** de código y exámenes de conocimiento humano.

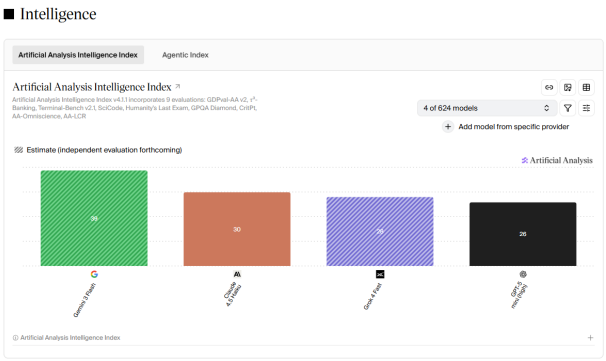


Figura 3.1: Artificial Analysis Intelligence Index

Por otra parte, cuando nos basamos particularmente en el **LiveCodeBench**, que es un sistema de evaluación comparativa de codificación libre de contaminación que recopila continuamente nuevos problemas de programación competitiva de LeetCode, AtCoder y CodeForces, con el objetivo de evaluar la generación de código, la autorreparación y la ejecución, que al final es el objetivo de la plataforma, enseñar temas de codificación, podemos ver la figura (3.2) en la que Geminis casi por un poco menos de 7 % supera a los demás modelos.

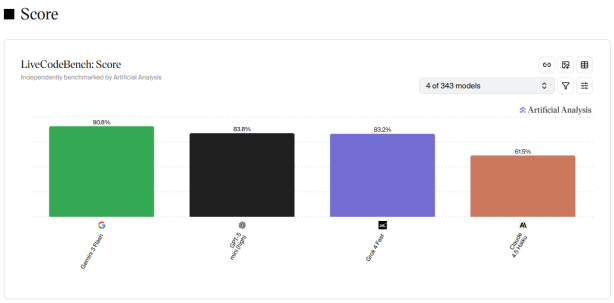


Figura 3.2: LiveCodeBench: Score

Siguiendo con los temas de costos, si evaluamos la figura (3.3), se puede notar, en cuanto al costo por millón de tokens que Grok 4 Fast presenta el precio más bajo del grupo con 0,2$ en tokens de entrada y 0,5$ en tokens de salida, seguido de GPT-5 mini (high) con 0,25$ y 2$ respectivamente, y Gemini 3 Flash, con 0,5$ y 3$. Por otra parte, Claude 4.5 Haiku resulta el más costoso de los cuatro, con un precio de salida de 5$ por millón de tokens, diez veces superior al de Grok 4 Fast. Lo que implica que aún con un costo intermedio, Geminis 3 Flash representa el mejor costo beneficio para el proyecto.

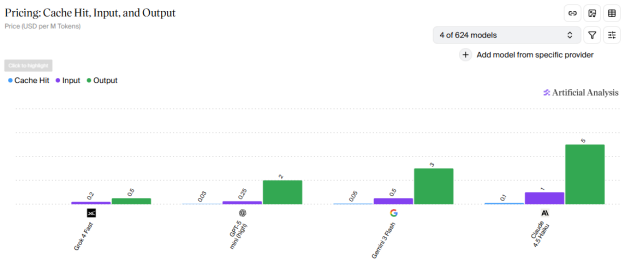


Figura 3.3: Pricing: Cache Hit, Input, and Output

Por último, si bien no se encontraron métricas ligadas directamente a contextos educativos, algunos de los benchmarks que componen el Intelligence Index sí evalúan capacidades afines, como GPQA Diamond, que mide la resolución de problemas de nivel de posgrado en biología, física y química. Por otro lado, también esta el Humanity's Last Exam, que evalúa conocimiento y razonamiento experto a través de un amplio rango de disciplinas académicas.

Cabe recalcar que modelos de gama alta como Claude Opus 5 o Claude Fable 5 podrían superar a los otros modelos evaluados en los campos requeridos. Sin embargo, su costo considerablemente mayor motiva que la comparación se centre en la gama económica de cada proveedor, dado que las pruebas del proyecto correrán con recursos propios. Asimismo, con base en los resultados obtenidos, se selecciona Gemini 3 Flash como la API a utilizar en el proyecto, al ofrecer el mejor desempeño tanto en LiveCodeBench como en el Intelligence Index dentro de la gama económica evaluada, manteniendo además un costo intermedio frente a las demás alternativas consideradas. Por último, toda la información de métricas presentada en este apartado puede consultarse en (Artificial Analysis, 2026).

3.4.3. Flujos de Datos y Persistencia

Con el objetivo de establecer cómo se mueve la información entre los componentes del sistema antes de iniciar el desarrollo, se definen a continuación los flujos de datos principales de la plataforma y el esquema de persistencia en Supabase.

Flujo 1: Configuración de sesión bajo el marco CRAFT

1. El docente inicia sesión en la plataforma y si tiene activada la autenticación de dos factores, ingresa además el código de verificación correspondiente. Posteriormente, accede al Módulo de Administración de Profesor.
2. El docente configura el prompt del ITS mediante el Panel de Configuración de Prompts con IA, diseñado bajo la estructura CRAFT, definiendo sus cinco componentes: **contexto**, correspondiente al tema y la materia a enseñar, **rol** que define qué tipo de tutor debe ser el sistema, por ejemplo un tutor socrático paciente, **audiencia**, que recoge las características del estudiante al que va dirigida la sesión, **formato**, que establece cómo debe ser la salida de la conversación y por último, **tono**, que define la actitud que debe mantener el tutor. Para facilitar esta tarea, el docente puede utilizar la opción **Sugerir Directrices CRAFT con IA**, con la cual Spring Boot solicita a la API una propuesta de los cinco componentes a partir del tema ingresado, que se carga en el formulario para que el docente la revise y ajuste. Estos componentes se configuran de manera independiente al conjunto fijo de reglas del método socrático, el cual reside en un archivo de instrucciones del sistema y no es modificable por el docente.
3. Adicionalmente, el docente define un conjunto de preguntas guía para la sesión, las cuales orientan al tutor sobre los puntos concretos que el estudiante debe comprender, sin

restringir su capacidad de responder dudas adicionales que este pueda plantear durante la conversación. Estas preguntas, al igual que los cinco componentes del CRAFT, forman parte de la capa configurable del prompt.

4. Spring Boot almacena los cinco componentes del CRAFT y las preguntas guía de forma modular en columnas independientes de la tabla **sesion** en Supabase, y genera un enlace único de acceso asociado a la sesión.
5. El docente comparte el enlace con sus estudiantes.
6. El estudiante abre el enlace en su navegador y Spring Boot valida el identificador contra Supabase. Si el enlace no corresponde a ninguna sesión, se muestra un mensaje de error y se deniega el acceso. Si el enlace es válido, se presenta al estudiante un formulario en el que, si es nuevo, se registra con su nombre, correo institucional y contraseña, o, si ya tiene una cuenta, inicia sesión con su correo y contraseña. El sistema solo admite correos del dominio institucional.
7. Al completar el registro o el inicio de sesión, el estudiante queda vinculado automáticamente a la sesión mediante una nueva participación y se habilita su acceso al chat socrático, dentro de la Interfaz de Interacción Socrática. Si el docente ha pausado la sesión, el estudiante puede consultar su conversación, pero no enviar nuevos mensajes hasta que esta se reanude.

Flujo 2: Interacción del estudiante

1. El estudiante envía un mensaje a través de la interfaz de chat desarrollada con Thymeleaf.
2. Spring Boot valida que la sesión esté activa y que la tutoría del estudiante no haya finalizado y consulta en Supabase el historial completo de la conversación de esa participación.
3. Spring Boot construye el prompt combinando tres capas: la Capa 1, con las reglas fijas del método socrático definidas en el archivo del sistema, la Capa 2 con los cinco componentes del CRAFT y las preguntas guía configuradas por el docente para la sesión y por último, la Capa 3, con el historial de la sesión activa. Al final del prompt se agrega el mensaje nuevo del estudiante, en un bloque delimitado que el modelo debe tratar como parte del diálogo y no como una instrucción.
4. Spring Boot envía el prompt completo a la API, que responde aplicando el método socrático mediante preguntas, pistas y analogías, siguiendo las reglas fijas de la Capa 1, el estilo definido por el docente en el CRAFT y orientándose hacia las preguntas guía definidas para la sesión. Junto con su respuesta, el modelo incluye etiquetas ocultas que señalan eventos de la tutoría, como la apertura o superación de una pregunta guía, la finalización de la ruta o una solicitud de respuesta directa por parte del estudiante.
5. Spring Boot procesa la respuesta, detecta y elimina las etiquetas ocultas antes de mostrarla y a partir de ellas, registra en la tabla **metrica** los eventos de seguimiento aso-

ciados a la interacción. Asimismo, almacena en Supabase el mensaje del estudiante y la respuesta entregada.

6. La respuesta se muestra al estudiante en la interfaz. Si en esta interacción el estudiante completó la ruta de preguntas guía, su tutoría se da por finalizada y se inicia el Flujo 3.

Flujo 3: Generación del progreso y optimización pedagógica

1. Cuando el estudiante completa la ruta de preguntas guía, Spring Boot solicita a la API, en segundo plano, un análisis de la conversación completa, tomando como insumo el historial de mensajes de esa participación. Para los estudiantes que no completan la ruta, el docente puede solicitar este análisis desde el Panel de Métricas Docente, ya sea de forma individual o para todos los estudiantes pendientes en una sola acción. El análisis se recibe como texto con formato Markdown, siguiendo estrictamente la estructura fija de cuatro secciones descrita en la Capa Dinámica, y se almacena en el campo **progreso_md** de la participación correspondiente en Supabase.
2. A partir de los informes individuales, las métricas de la sesión y para los estudiantes que aún no tienen informe, sus mensajes, Spring Boot solicita a la API una Síntesis Pedagógica Consolidada del grupo, la cual se almacena en la sesión y se actualiza automáticamente al generarse los informes pendientes.
3. Cuando el docente accede al Panel de Métricas Docente desde una sesión particular dentro del Módulo de Administración de Profesor, el sistema calcula y muestra los cuatro indicadores de seguimiento (M-01 a M-04) a partir de los eventos registrados, los informes individuales y las marcas de tiempo de los mensajes, lo que permite identificar los temas en los que el grupo o un estudiante en particular puede estar fallando.
4. El panel presenta, junto con estas métricas, los informes individuales y la síntesis grupal con sus recomendaciones pedagógicas, sin afectar en ningún caso las reglas fijas del método socrático ni las preguntas guía definidas por el docente.

Qué se envía a la API en cada llamada

Con el fin de precisar el contenido exacto de cada llamada a la API descrita en los flujos anteriores, se distinguen cuatro momentos.

En cada interacción del estudiante, correspondiente al Flujo 2, se envía el prompt completo, compuesto por la Capa 1, con las reglas fijas del método socrático, la Capa 2 con los cinco componentes del CRAFT y las preguntas guía de la sesión activa y la Capa 3, con el historial completo de mensajes de la sesión activa. Al final se agrega el mensaje nuevo del estudiante, en un bloque delimitado.

Para la generación del informe individual, correspondiente al Flujo 3, se envía el historial completo de mensajes de la participación, las preguntas guía de la sesión como referencia

para nombrar los conceptos, y la instrucción de generar el análisis estructurado en la plantilla Markdown de cuatro secciones definida en la Capa Dinámica. Esta llamada no incluye la Capa 1 ni los componentes del CRAFT, dado que su propósito no es sostener el diálogo socrático, sino sintetizar la conversación.

Para la síntesis grupal, también correspondiente al Flujo 3, se envían las preguntas guía de la sesión, las métricas agregadas del grupo y la evidencia anonimizada de cada estudiante, que corresponde a su informe individual o, si aún no lo tiene, a sus mensajes.

Finalmente, para la sugerencia de directrices CRAFT, correspondiente al Flujo 1, se envía únicamente el tema de la clase ingresado por el docente, junto con la instrucción de proponer los cinco componentes del framework. La propuesta recibida se carga en el formulario de configuración para que el docente la revise y ajuste antes de guardarla. Si la API no responde, se carga una plantilla base editable.

3.5. Modelo C4

3.5.1. Diseño de Diagrama de Contexto

La figura (3.4) presenta el diagrama de contexto del sistema, que busca mostrar una visión general de la solución con su alcance delimitado. Para este nivel de diagramación se puede notar que los agentes externos son el profesor, que crea clases, adiciona estudiantes a los mismos y revisa sus resultados en un panel de métricas. Están los estudiantes, que interactúan con la plataforma, reciben retroalimentación socrática y califican su experiencia con la plataforma. Asimismo, se encuentra la plataforma que se conecta a un sistema externo, como lo es la API de IA que es la que se encarga de procesar las preguntas y respuestas de los estudiantes para poder generar retroalimentación. Además, se encarga de producir el feedback de forma individual y general de los estudiantes para que el profesor pueda ver los avances y errores.

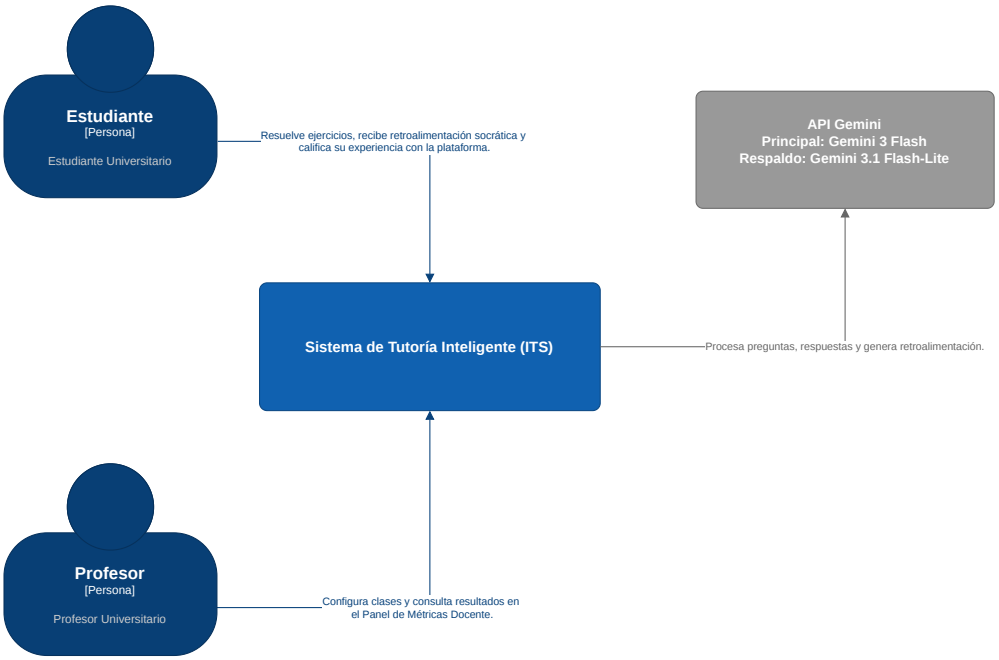


Figura 3.4: Diagrama de Contexto

3.5.2. Diagrama de Contenedores

La figura (3.5) presenta el diagrama de contenedores del proyecto. Por una parte están los actores externos que son los profesores y estudiantes de los que se habló antes. También estan los contenedores principales como el de la **Aplicación Web**, este se desarrolla en **Spring Boot + Thymeleaf** y se encarga de proporcionar la interfaz con todas las funcionalidades de retroalimentación, monitoreo de avance de los estudiantes, orquestación y revisión de la interacción con la IA que para el proyecto se realiza por medio de la API de Gemini con el modelo **Gemini 3 Flash** y **Gemini 3.1 Flash-Lite** como respaldo. Por último, se encuentra el segundo contenedor que es el de la base de datos que sería de **PostgreSQL**, que se utiliza para almacenar la información de la interacción de los estudiantes y profesores con la plataforma.

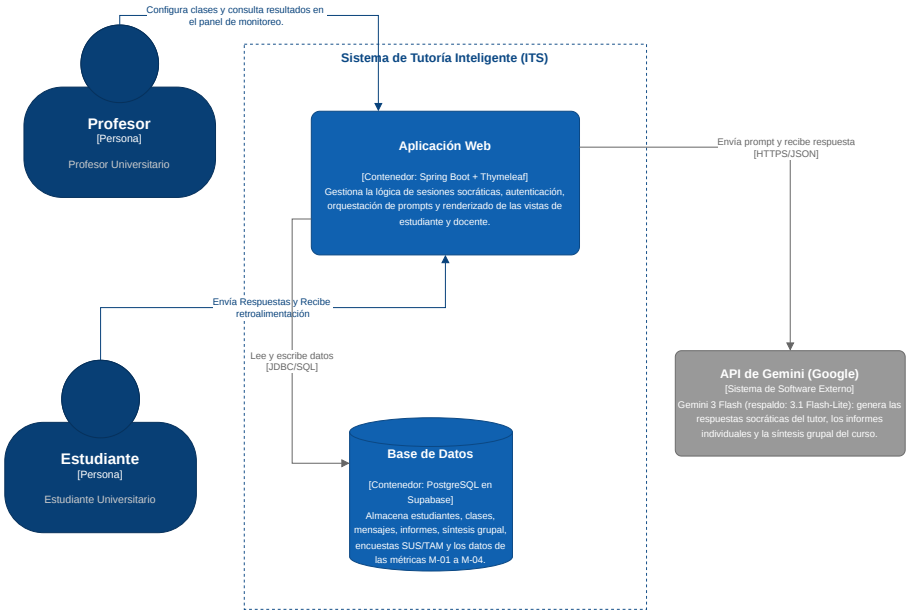


Figura 3.5: Diagrama de Contenedores

3.5.3. Diagrama de Componentes

Como se observa en la figura (3.6), la aplicación web desarrollada, se organiza en componentes que se conectan entre si para poder atender dos flujos principales. El primero el del estudiante y del profesor, en el que el servicio de chat socrático recibe cada mensaje y solicita al constructor de Prompt, el ensamblaje de tres capas definidas previamente: Las reglas socráticas (Capa 1), la configuración del docente con el framework CRAFT y las preguntas guías (Capa 2) y el historial de la sesión actual (en caso de que se tenga), obtenido de los Repositorios JPA (Capa 3). El prompt se envía, mediante el cliente de API de Gemini, y su respuesta pasa por el Procesador de Respuesta, que detecta y elimina las meta-etiquetas (que son etiquetas ocultas que el mismo modelo va poniendo de IA en el momento que el estudiante avanza en cada pregunta o termina la clase) de avance y de solicitud directa y registra las métricas M-02 y M-03 en cada turno y al completar la ruta, solicita el informe individual. Por último, el generador de informes produce en segundo plano el informe individual del estudiante. En el

segundo flujo, el Controlador Docente atiende el panel del profesor y se apoya en el Servicio de Analítica para calcular las métricas M-01 a M-04 y generar la síntesis grupal. Todos los componentes acceden a la base de datos PostgreSQL a través de los Repositorios JPA.

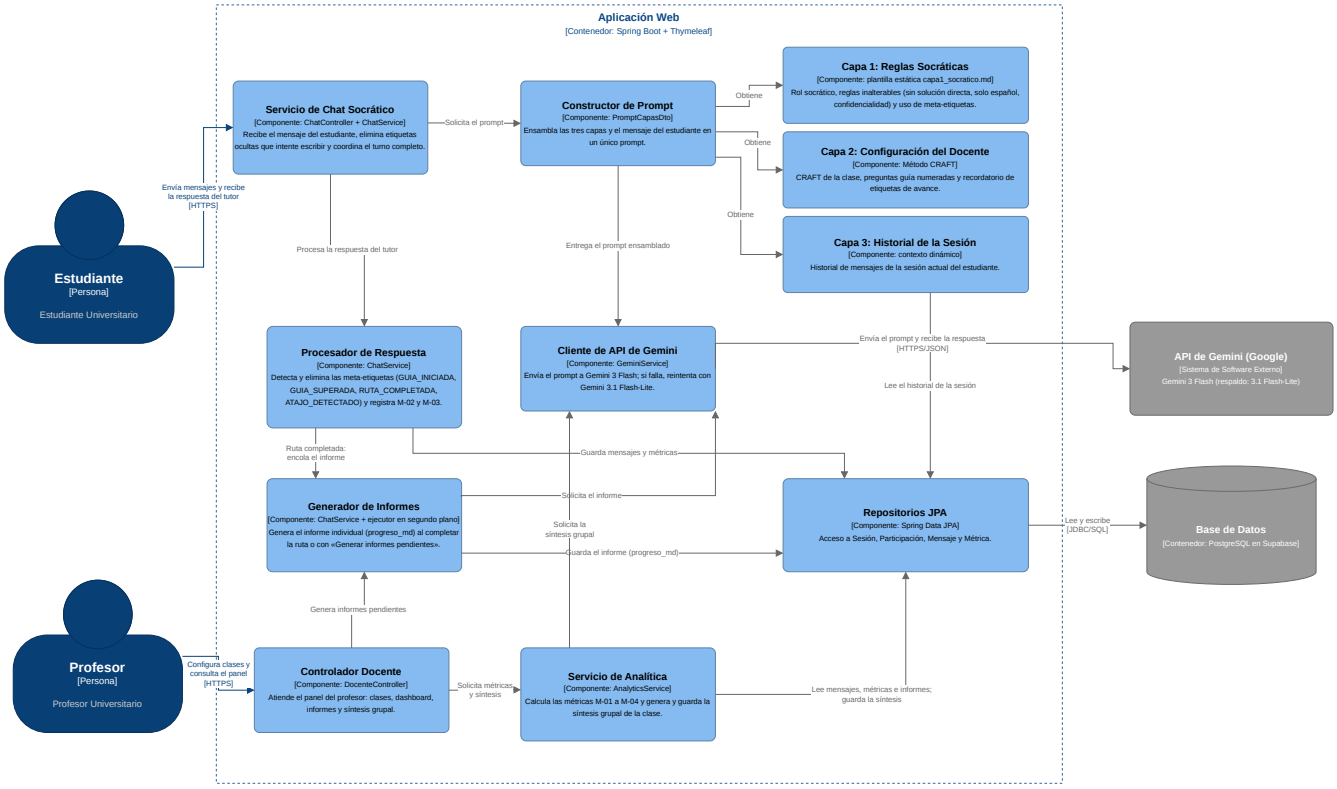


Figura 3.6: Diagrama de Componentes

3.5.4. Diagrama de Código

Como se observa en la figura (3.7), el ChatController recibe el mensaje del estudiante y lo delega al ChatService, clase central del proceso, que limpia el mensaje, construye las capas del prompt en un PromptCapasDto y lo envía a Gemini a través del GeminiService, que devuelve un RespuestaGemini indicando si respondió el modelo principal o el de respaldo. Con esa respuesta, el ChatService detecta las meta-etiquetas, calcula los hitos de las preguntas guía, valida el cierre de la ruta y registra las métricas. Además, solicita en segundo plano el informe individual y se apoya en el AnalyticsService para actualizar la síntesis grupal. La persistencia se realiza mediante los repositorios de Spring Data JPA, que gestionan las entidades Sesion, Participacion, Mensaje y Metrica, relacionadas con Estudiante y Profesor: cada participación pertenece a un estudiante y a una sesión, cada sesión a un profesor, y cada participación agrupa sus mensajes y métricas.

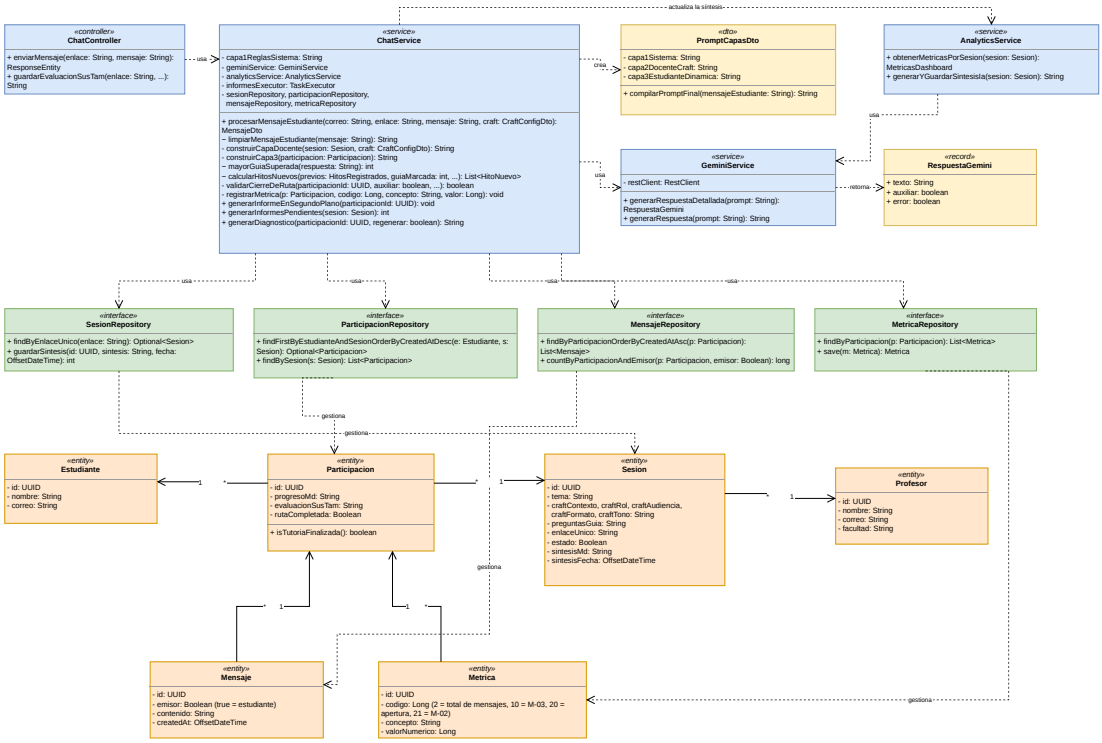


Figura 3.7: Diagrama de Código

3.5.5. Esquema de Base de Datos

Como se observa en la Figura 3.8, el esquema se organiza en torno a dos ejes principales, el primero es la identidad de los usuarios y el segundo es el seguimiento de la interacción del estudiante dentro de cada sesión. Las tablas **profesor** y **estudiante** almacenan cada una la identidad de sus usuarios de forma independiente, lo que evita duplicar datos como el nombre o el correo entre distintas sesiones. Además, la tabla **profesor_estudiante** resuelve la vinculación entre ambos. Por otra parte, cada sesión pertenece a un profesor y guarda la configuración del tutor, es decir, los componentes CRAFT, las preguntas guía y la síntesis grupal, junto con su estado (activa o pausada). La relación entre un estudiante y una sesión se resuelve mediante la tabla **participacion**. Esa tabla además concentra el informe cualitativo del estudiante (progreso_md), el indicador de ruta completada y su evaluación SUS/TAM para esa sesión específica, aislándolos del resto de sus participaciones. De cada participación se derivan los registros de mensaje, que reconstruyen el historial de la conversación diferenciando entre estudiante y tutor, y los de metrica, que guardan como pares código–valor los eventos detectados durante la tutoría, tales como, total de mensajes, atajos, apertura de cada pregunta guía y preguntas guía superadas. A partir de estos datos, del informe y de las marcas de tiempo de los mensajes, la plataforma calcula los cuatro indicadores de seguimiento (M-01 a M-04).

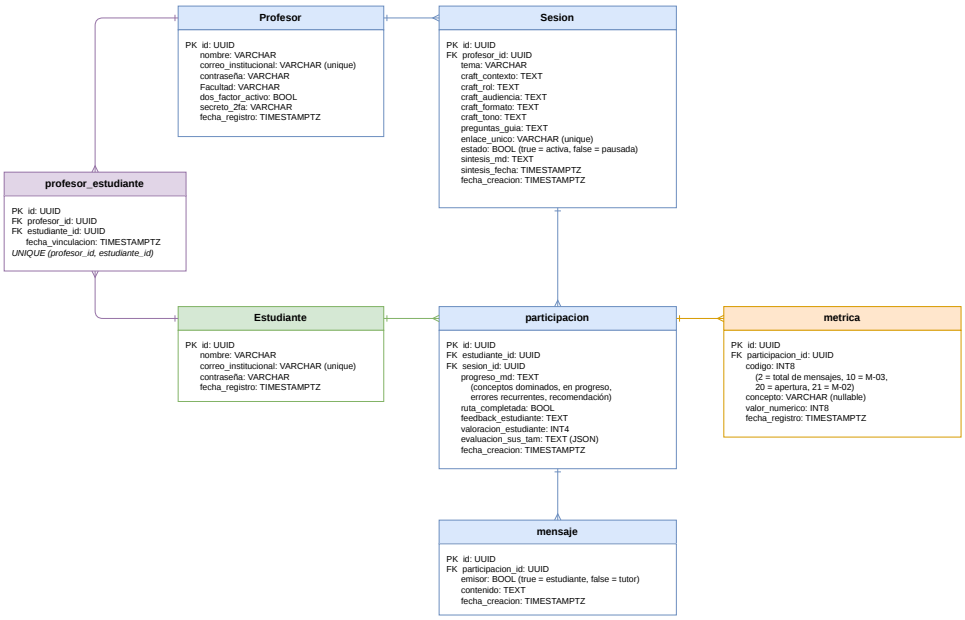


Figura 3.8: Base de Datos

3.6. Decisiones de Arquitectura y Desarrollo del Prototipo de la Plataforma

Estructuración del Código

1. **Contexto e integración del módulo Spring MVC en la plataforma:**
   Spring MVC constituye el marco de trabajo sobre el cual se articula la arquitectura web del prototipo, fundamentado en el patrón Modelo-Vista-Controlador (MVC) y en el patrón estructural de Controlador Frontal (*Front Controller*) (Prasad Reddy, 2017). En el flujo del sistema de tutoría inteligente, el componente **DispatcherServlet** actúa como el punto de entrada centralizado que intercepta todas las peticiones HTTP entrantes, originadas tanto por estudiantes, por ejemplo en la interacción con el chat socrático, como por docentes, por ejemplo en la gestión de sesiones y el monitoreo de métricas (Prasad Reddy, 2017). Este componente delega cada solicitud al controlador correspondiente, identificado con la anotación **@Controller**, el cual a su vez delega la lógica de negocio a la capa de servicios. Son estos servicios los que se comunican con la API de Gemini 3 Flash para la generación del diálogo y con Supabase para la persistencia, mientras que el controlador finalmente coordina con el **ViewResolver** el renderizado de la vista que se retorna al usuario (Prasad Reddy, 2017).
2. **Simplificación e infraestructura mediante spring-boot-starter-webmvc:**
   El despliegue e integración tradicional de Spring MVC exige la declaración y configuración manual de múltiples bloques funcionales, tales como el registro explícito del **DispatcherServlet**, la resolución de vistas y el manejo de errores (Prasad Reddy, 2017). Con el fin de optimizar el desarrollo de la plataforma y aplicar el principio de "convención sobre configuración", se utiliza el módulo encapsulado de Spring Boot que autoconfigura la infraestructura web de la aplicación (Prasad Reddy, 2017). En la versión 4 de Spring Boot, utilizada en el proyecto, este módulo se denomina **spring-boot-starter-webmvc**, en reemplazo del anterior **spring-boot-starter-web**. Su autoconfiguración se da a través de los siguientes elementos clave:
   - **Servidor Web Embebido y Contenerización:** Incluye un contenedor de servlets Apache Tomcat integrado por defecto, que responde convencionalmente en el puerto 8080 (Prasad Reddy, 2017), configurable en el proyecto mediante una variable de entorno. Esto permite empaquetar la plataforma como un archivo ejecutable independiente (**.jar**), facilitando su despliegue y aislamiento en un contenedor Docker sin requerir la instalación ni configuración de un servidor de aplicaciones externo (Prasad Reddy, 2017).
   - **Autoconfiguración del Flujo MVC:** Inicializa automáticamente el **DispatcherServlet** mapeado a la raíz del sistema (**/**), gestiona de forma transparente la entrega de recursos estáticos ubicados en **/static**, como las hojas de estilo CSS de la plataforma, y provee un manejo de errores por defecto que captura las excepciones no

controladas durante las solicitudes y las redirige a una respuesta de error estándar (Prasad Reddy, 2017).

- **Integración con el Renderizado Server-Side (Thymeleaf):** En conjunto con el módulo **spring-boot-starter-thymeleaf**, se autoconfigura la resolución de vistas mediante el motor de plantillas Thymeleaf (Prasad Reddy, 2017). Esto simplifica la inyección dinámica de datos contextuales desde la capa de servicios, tales como las respuestas del tutor socrático, los historiales de conversación o las métricas del panel docente, hacia los documentos HTML antes de ser entregados al navegador.
- **Endpoints JSON para la Comunicación Asíncrona:** Además de retornar vistas, los controladores exponen endpoints que, mediante la anotación **@ResponseBody** y los convertidores de mensajes HTTP (**HttpMessageConverters**), serializan y deserializan automáticamente objetos Java a formato JSON (Prasad Reddy, 2017). Esta capacidad resulta indispensable para la comunicación asíncrona entre el navegador y el backend, como ocurre al enviar un mensaje en el chat, solicitar la sugerencia de directrices CRAFT o generar los informes pendientes, sin necesidad de recargar la página. De manera complementaria, la comunicación con la API de Gemini se realiza desde la capa de servicios mediante el cliente HTTP **RestClient**, que utiliza la misma infraestructura de conversión JSON, mientras que el acceso a Supabase se realiza a través de JDBC mediante Spring Data JPA.

3.6.1. Organización del Proyecto

Como resultado del proceso de desarrollo, el prototipo se encuentra desplegado en un servidor virtual mediante Dokploy, a partir de la imagen Docker de la aplicación. De esta manera, la plataforma está disponible para estudiantes y docentes a través de un dominio público, como se observa en la Figura 3.9.

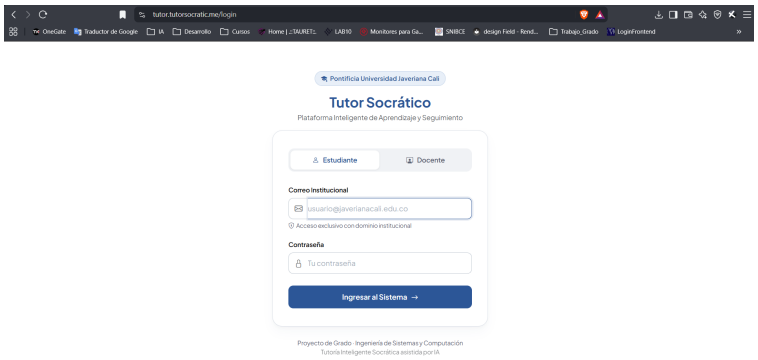


Figura 3.9: Página de inicio de la plataforma desplegada

En cuanto a su estructura interna, el código fuente se organizó en paquetes según la responsabilidad de cada uno dentro de la aplicación. Así, los controladores reciben las solicitudes

de los usuarios, los servicios concentran la lógica del tutor y de las métricas y los repositorios acceden a la base de datos. La Tabla 3.4 resume esta organización junto con las clases principales de cada paquete.

Cuadro 3.4: Organización de los paquetes del proyecto

Paquete	**Responsabilidad**	**Clases principales**
controller	Recibir las peticiones del estudiante y del docente.	ChatController, DocenteController, EstudianteController, LoginController y HomeController.
service	Lógica del tutor socrático, métricas, autenticación e integración con la IA.	ChatService, AnalyticsService, GeminiService, PromptAssistantService, AuthService, TotpService y LoginRateLimiterService.
repository	Acceso a la base de datos.	Un repositorio por cada entidad, siete en total.
model.entity	Representación de las tablas de la Figura 3.8.	Profesor, Estudiante, ProfesorEstudiante, Sesion, Participacion, Mensaje y Metrica.
model.dto	Transporte de datos entre capas.	PromptCapasDto, CraftConfigDto, MensajeDto y EvaluacionSusTamDTO.
config	Seguridad, cliente HTTP, ejecución en segundo plano y variables de entorno.	SecurityConfig, RestClientConfig, AsyncConfig y WebMvcConfig.

Por otra parte, las vistas de la plataforma se ubican en la carpeta **templates** y se agrupan según el tipo de usuario. Asimismo, en primer lugar, la carpeta **docente** contiene las vistas de sesiones, configuración de clases, panel de métricas, estudiantes y feedback. En segundo lugar, la carpeta **estudiante** reúne el registro, la lista de clases y el chat. Finalmente, la carpeta **Login** incluye el inicio de sesión y la verificación de dos factores. Adicionalmente, las reglas fijas del método socrático, correspondientes a la Capa 1 del prompt, se almacenan de forma independiente en el archivo **capa1_socratico.md**, dentro de la carpeta **resources/prompts**, lo que permite modificarlas sin alterar el código de la aplicación. En consecuencia, la estructura resultante del proyecto es la que se presenta en la figura 3.10

Cabe aclarar que, en esta misma figura, algunas carpetas y archivos aparecen marcados en rojo, lo que en Visual Studio Code suele indicar errores de código. Sin embargo, en este caso se trata de un falso positivo dado que el proyecto utiliza **Lombok**, una librería de Java que genera automáticamente, durante la compilación, métodos como los de acceso a los atributos y los constructores de las clases. Esta generación la realiza **Maven** al compilar el proyecto,

razón por la cual la aplicación se construye y despliega sin errores. No obstante, el analizador de Java del editor no procesa dichas anotaciones y en consecuencia, no reconoce los métodos generados, por lo que marca como erróneos los archivos que los utilizan.

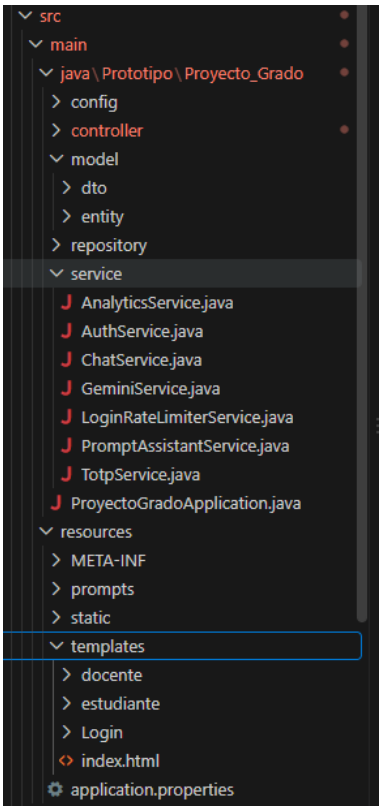


Figura 3.10: Estructura del proyecto

3.6.2. Proceso de Desarrollo Incremental

En concordancia con la metodología incremental y evolutiva planteada en la Sección 1.5, el prototipo se construyó en iteraciones sucesivas, de modo que cada una integró nuevas funcionalidades sobre una base ya verificada. Para ello, el control de versiones se llevó a cabo en GitHub, lo que permitió registrar la evolución del proyecto entre septiembre y octubre de 2026. La Tabla 3.5 resume las iteraciones realizadas y el resultado obtenido en cada una.

Cuadro 3.5: Iteraciones del desarrollo del prototipo

Iteración	**Fecha**	**Resultado**
1	6 al 13 de septiembre	Creación del repositorio, configuración del proyecto y conexión con la base de datos en Supabase.
2	23 de septiembre	Funcionalidades base, como el inicio de sesión, la creación de sesiones de tutoría, el enlace único de acceso y el registro de estudiantes.
3	27 al 30 de septiembre	Sistema de tutoría inteligente, con la construcción del prompt en tres capas, la integración con la API de Gemini y el chat socrático.
4	5 de octubre	Asistente de configuración CRAFT con IA, ajustes en la generación de plantillas y diseño responsive para dispositivos móviles.
5	10 de octubre	Corrección de la autenticación de dos factores para el docente y ajustes de estabilidad.
6	14 de octubre	Mejoras en la interfaz y refinamiento de las reglas del tutor socrático.
7	18 de octubre	Métricas M-01 a M-04, generación automática de informes individuales, síntesis grupal y sección Mis Estudiantes.

A partir de estas iteraciones, las subsecciones siguientes describen las funcionalidades resultantes, organizadas según el tipo de usuario. Inicialmente se presenta la gestión de usuarios y acceso, posteriormente el portal del profesor mientras que el portal del estudiante se presenta en el video de demostración del Anexo 6.0.2.

3.6.3. Gestión de Usuarios y Acceso

La plataforma cuenta con un único punto de inicio de sesión para estudiantes y docentes, en el cual el usuario ingresa su correo institucional, su contraseña y selecciona el rol con el que desea acceder. Como se observa en la Figura 3.11. A partir de esta selección, el sistema verifica las credenciales contra la tabla correspondiente y redirige al usuario a su portal, ya

sea el del profesor o el del estudiante. Cabe resaltar que las contraseñas nunca se almacenan en texto plano, sino cifradas mediante el algoritmo BCrypt, en cumplimiento del RNF-02.



Figura 3.11: Inicio de sesión de la plataforma

Con el fin de proteger el acceso, el sistema limita los intentos fallidos de inicio de sesión. De este modo, si se registran cinco intentos incorrectos para un mismo correo desde la misma dirección, el acceso se bloquea temporalmente durante diez minutos. Asimismo, cada vez que un usuario inicia sesión correctamente, el sistema renueva el identificador de su sesión, lo que evita que un tercero pueda reutilizar una sesión previa.

Autenticación de Dos Factores para el Docente

Dado que el docente tiene acceso a la información de todos sus estudiantes, se incorporó la opción de activar la autenticación de dos factores (2FA) desde su perfil. Para ello, la plataforma genera un código QR que el docente escanea con una aplicación autenticadora, como Google Authenticator, Microsoft Authenticator o Authy y posteriormente confirma la activación ingresando el código de seis dígitos que esta genera, tal como se muestra en la Figura 3.12. Es importante mencionar que el código QR se genera localmente en el servidor,

por lo que la clave secreta del docente no se envía a ningún servicio externo.



Figura 3.12: Activar Doble Factor

Una vez activado, cada inicio de sesión del docente requiere un paso adicional. Así, después de ingresar su correo y contraseña, el docente debe digitar el código vigente de su aplicación autenticadora, el cual cambia cada treinta segundos, como se observa en la Figura 3.13. Además, si el código se ingresa de forma incorrecta cinco veces, el proceso se cancela y el docente debe iniciar sesión nuevamente. Por último, el docente puede desactivar esta opción en cualquier momento desde su perfil.

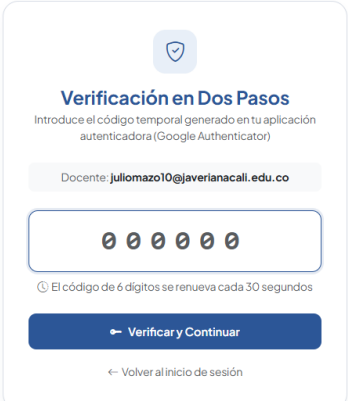


Figura 3.13: Verificación del segundo factor al iniciar sesión

Acceso del Estudiante mediante Enlace Único

En cuanto al estudiante, su ingreso a una sesión de tutoría se realiza a través del enlace único que comparte el docente, en cumplimiento del RF-01. Al abrirlo, el estudiante encuentra

el nombre de la clase y dos opciones, como se observa en la Figura 3.14. Por un lado, si es nuevo en la plataforma, selecciona la opción **Soy nuevo estudiante** y se registra con su nombre, correo institucional y contraseña. Por otro lado, si ya cuenta con una cuenta, selecciona **Ya tengo cuenta** e ingresa únicamente su correo y contraseña. En ambos casos, el sistema solo admite correos de los dominios institucionales de la universidad y al completar el proceso, el estudiante queda vinculado automáticamente a la clase y accede directamente al chat socrático.

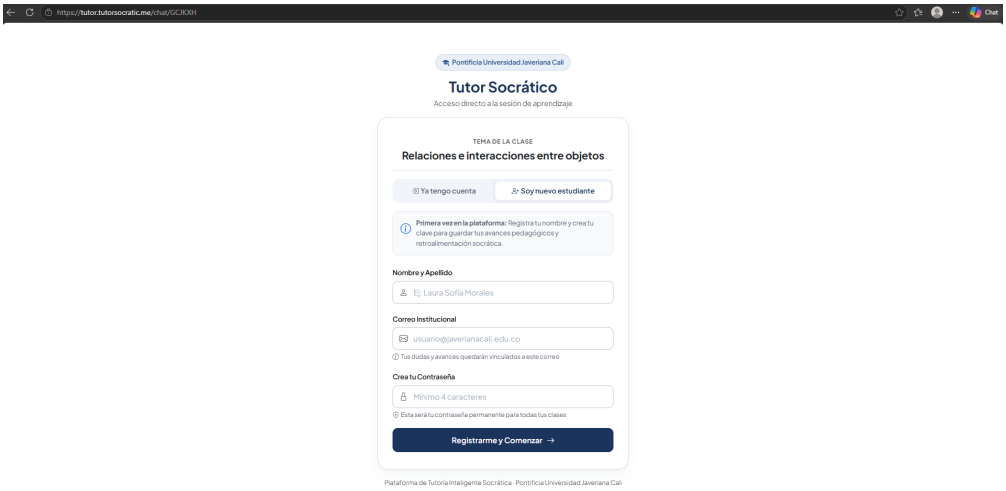


Figura 3.14: Registro e ingreso del estudiante desde el enlace de la clase

De esta manera, el estudiante registra sus datos una única vez y los reutiliza en todas las clases a las que se vincule posteriormente, lo que reduce la información personal almacenada al mínimo necesario para la operación de la plataforma. Finalmente, tanto el docente como el estudiante pueden consultar y editar su información personal desde sus respectivos portales, incluyendo el cambio de contraseña. Asimismo, la información del estudiante puede verse en el siguiente video 6.0.2.

3.6.4. Portal del Profesor

Una vez el docente inicia sesión, accede a su portal, el cual se organiza en tres secciones principales, **Mis Sesiones**, **Mis Estudiantes** y **Feedback Recibido**, a las que se accede desde la barra de navegación superior. A continuación se describe cada una de ellas, junto con el Panel de Configuración de Prompts con IA y el Panel de Métricas Docente, que se abren desde la sección Mis Sesiones.

Mis Sesiones

La sección Mis Sesiones constituye la vista principal del docente. En ella se presenta, por un lado, su información personal, con las opciones de editar su perfil y de configurar la autenticación de dos factores descrita anteriormente. Por otro lado, se listan las clases que ha creado, cada una con su tema, su fecha de creación, su estado, activa o pausada y su código de acceso, como se observa en la Figura 3.15.

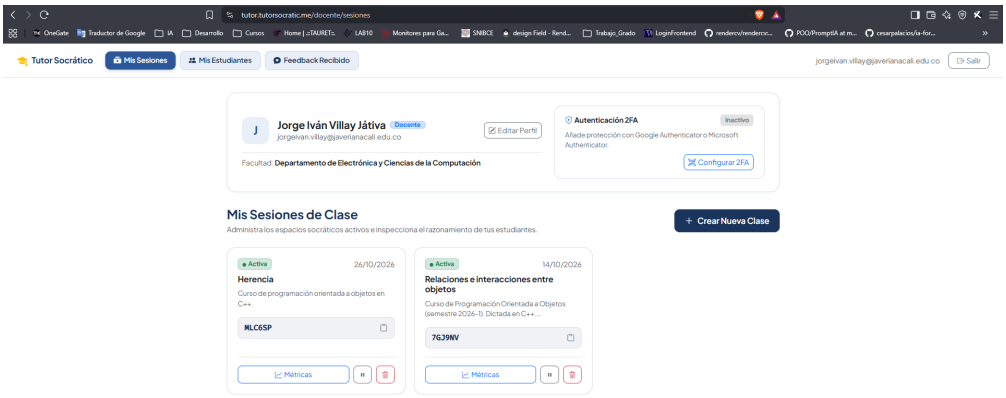


Figura 3.15: Apartado Mis Sesiones

Desde cada clase, el docente puede copiar el enlace único para compartirlo con sus estudiantes, abrir el Panel de Métricas Docente, pausar o reanudar la sesión y eliminarla. En este sentido, al pausar una clase, los estudiantes conservan el acceso a su conversación, pero no pueden enviar nuevos mensajes hasta que el docente la reanude. Por su parte, la eliminación de una clase solicita una confirmación previa, dado que borra de forma permanente los mensajes y las participaciones asociadas.

Panel de Configuración de Prompts con IA

Al seleccionar la opción **Crear Nueva Clase**, el docente accede al Panel de Configuración de Prompts con IA, el cual guía la creación de la sesión en tres pasos, en cumplimiento del RF-05. En el primer paso, el docente ingresa el tema de la clase y si lo desea, utiliza el botón **Sugerir Directrices CRAFT con IA**. Con ello, la plataforma solicita a la API una propuesta de los cinco componentes del framework y los carga automáticamente en el formulario, como se observa en la Figura 3.16. En caso de que la API no responda, se carga una plantilla base que el docente puede editar.

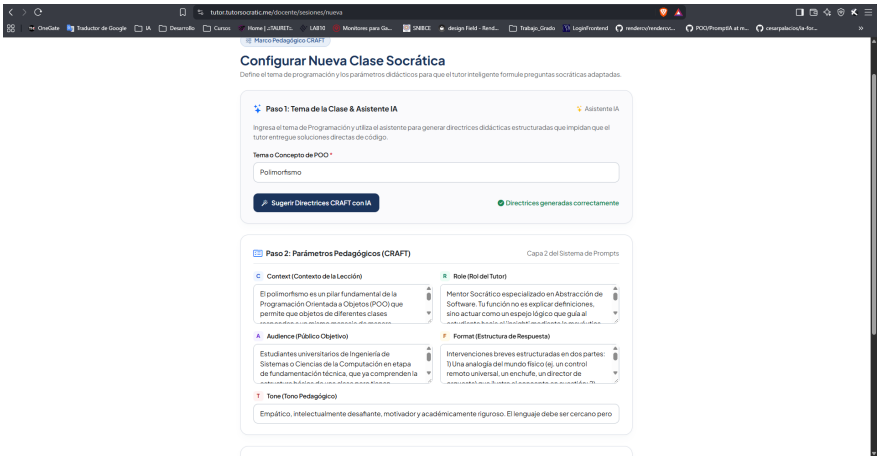


Figura 3.16: Apartado de Crear Nueva Clase

En el segundo paso, el docente revisa y ajusta cada uno de los componentes CRAFT. Es decir, el contexto de la lección, el rol del tutor, el público objetivo, el formato de las respuestas y el tono pedagógico, los cuales conforman la Capa 2 del prompt. Finalmente, en el tercer paso, el docente escribe las preguntas guía de la clase, una por línea, que el tutor abordará de forma secuencial con cada estudiante, tal como se muestra en la Figura 3.17. Al guardar, la clase queda activa y el sistema genera su enlace único de acceso.

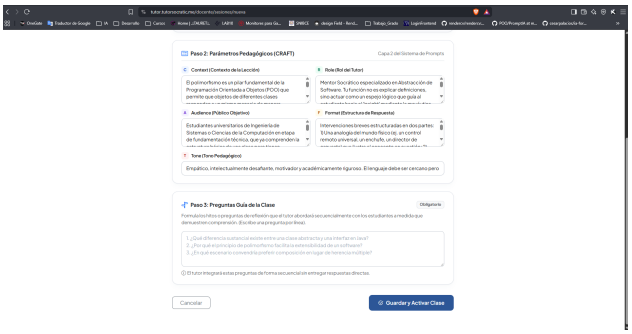


Figura 3.17: Definición de las preguntas guía de la clase

Panel de Métricas Docente

Al abrir una clase, el docente accede al Panel de Métricas Docente, el cual concentra el seguimiento del grupo y de cada estudiante. En la parte superior, el panel presenta cuatro indicadores generales, como se observa en la Figura 3.18. En primer lugar, el número de estudiantes que han escrito al tutor frente al total de inscritos. En segundo lugar, el promedio de turnos por estudiante (M-02). En tercer lugar, el tiempo activo promedio en la clase (M-

04). Por último, las alertas de atajo, que corresponden a las solicitudes de respuesta directa y su porcentaje sobre las consultas del grupo (M-03).

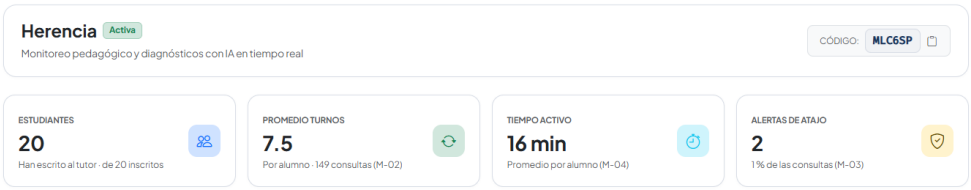


Figura 3.18: Indicadores generales del Panel de Métricas Docente

A continuación, la sección **Hoja de Ruta Pedagógica** muestra cada pregunta guía con el promedio de turnos que le tomó al grupo superarla y resalta aquella que más le cuesta, como se observa en la Figura 3.19. Para su interpretación, hasta tres turnos por pregunta se considera un avance fluido, de cuatro a seis un avance con esfuerzo y más de seis indica que el grupo se trabó en esa pregunta.

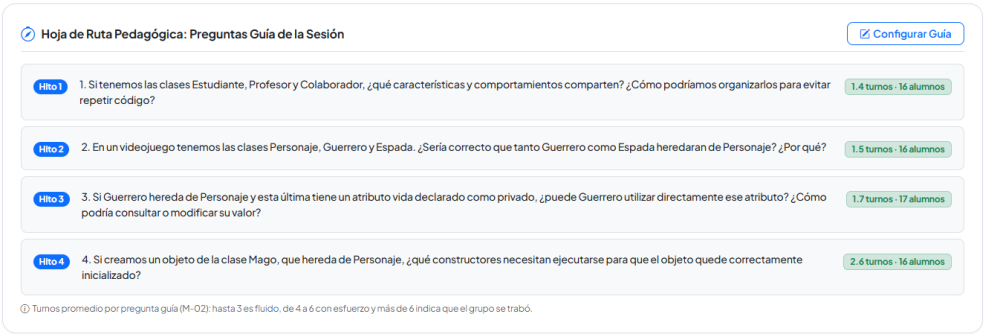


Figura 3.19: Hoja de Ruta Pedagógica con el avance del grupo por pregunta guía

Por otra parte, con el fin de que el docente no deba interpretar los datos por su cuenta, el panel incorpora dos apoyos generados con IA, en cumplimiento del RF-06 y el RF-07. El primero es la sección **Focos de Dificultad y Falencias**, que presenta los errores recurrentes del grupo, ordenados según el porcentaje de estudiantes que los presentan (M-01), a partir de los informes individuales. El segundo es la **Síntesis Pedagógica Consolidada**, que resume el diagnóstico del grupo, sus focos de confusión y una serie de recomendaciones para la siguiente clase presencial, como se observa en las Figuras 3.20 y 3.21. Esta síntesis se actualiza automáticamente al generarse los informes pendientes y además, el docente puede actualizarla manualmente en cualquier momento.

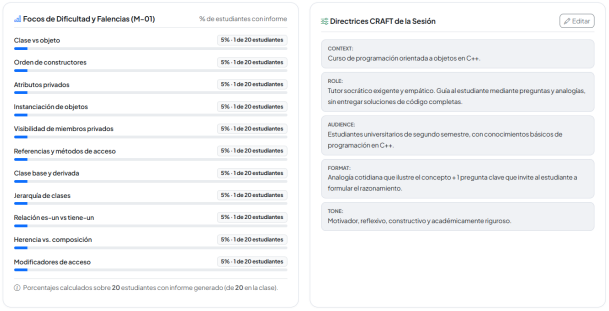


Figura 3.20: Apartado de Focos de Dificultad y Falencias



Figura 3.21: Síntesis Pedagógica Consolidada

Asimismo, la sección **Seguimiento Individual de Estudiantes** presenta una fila por cada estudiante de la clase, como se muestra en la Figura 3.22. En ella se observa su avance por pregunta guía y su estado (M-02), sus solicitudes de respuesta directa (M-03) y su actividad, es decir, el tiempo activo, el número de clases con el docente y la fecha de su último mensaje (M-04). En este punto, si un estudiante lleva más de tres días sin actividad, el sistema lo señala en rojo como estudiante en riesgo, en cumplimiento del RF-08. Cabe aclarar, que en las siguientes fotos, no aparecen nombres de estudiantes ni sus correos, para cumplir con la anonimización de la evaluación.

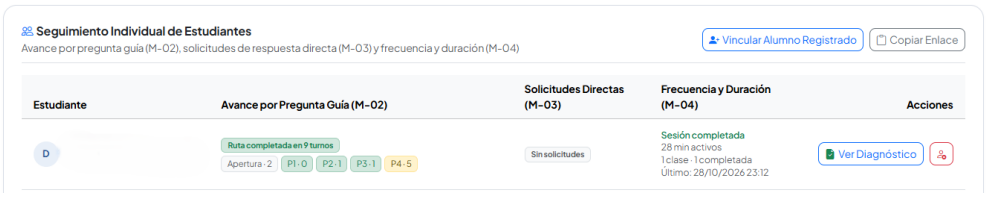


Figura 3.22: Seguimiento individual de los estudiantes de la clase

Desde esta misma tabla, el docente puede consultar el informe pedagógico individual de cada estudiante, el cual presenta sus conceptos dominados, los conceptos en progreso, sus errores recurrentes y una recomendación para la siguiente sesión, como se observa en la Figura 3.23. Dicho informe se genera automáticamente cuando el estudiante completa la ruta de preguntas guía. En cambio, para quienes no la han completado, el docente puede generarlo de forma individual o, mediante el botón **Generar informes pendientes**, para todos los estudiantes que aún no lo tienen, en una sola acción y en segundo plano.

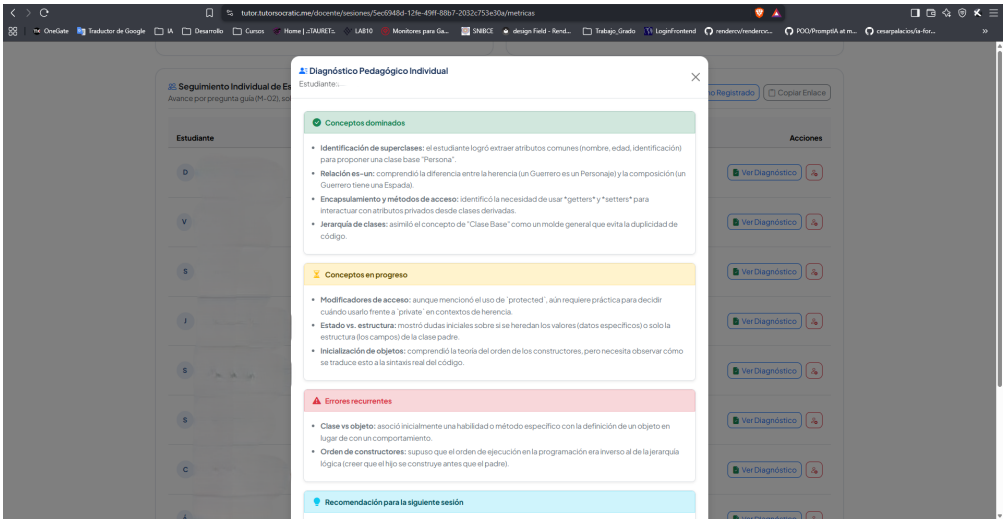


Figura 3.23: Informe pedagógico individual de un estudiante

Finalmente, el panel permite al docente editar las directrices CRAFT y las preguntas guía de la clase, así como vincular a la clase a estudiantes ya registrados en la plataforma o desvincularlos de ella.

Mis Estudiantes

La sección Mis Estudiantes presenta el directorio de los estudiantes registrados en la plataforma y su relación con las clases del docente, como se observa en la Figura 3.24. Para cada estudiante se muestra su nombre y correo, el número de clases que tiene con el docente y cuántas ha completado, su tiempo activo promedio por clase, la fecha de su última actividad y su estado, ya sea en clase activa, con clases pausadas o sin clases. De esta manera, el docente obtiene una visión general de la frecuencia y la duración de uso de cada estudiante (M-04), más allá de una clase en particular.

Adicionalmente, desde esta sección el docente puede asignar un estudiante a cualquiera de sus clases, sin necesidad de compartirle el enlace. También puede eliminarlo, en cuyo caso se borran sus participaciones en las clases del docente. No obstante, la cuenta del estudiante solo se elimina de la plataforma cuando este no pertenece a otros docentes, ya que en caso contrario

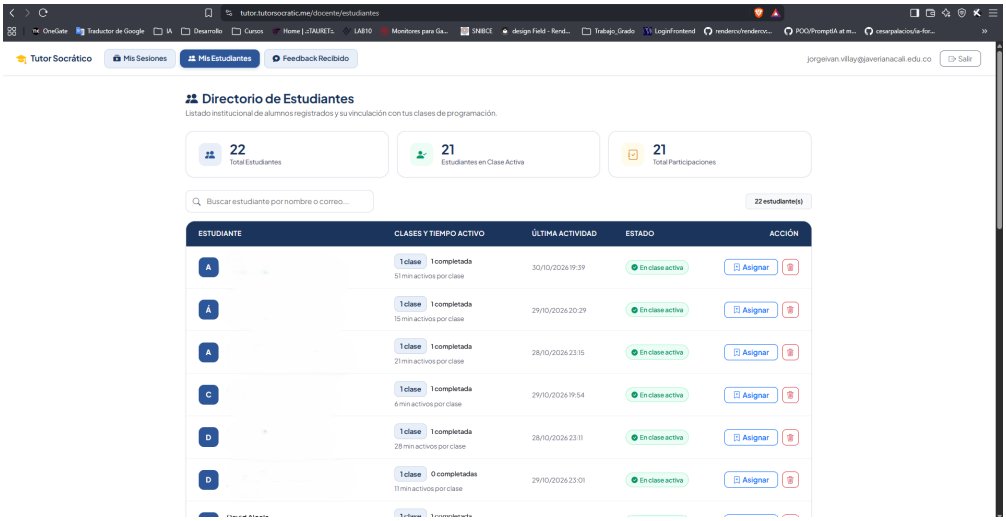


Figura 3.24: Sección Mis Estudiantes del docente

únicamente se retira del directorio del docente, con el fin de no afectar la información de las demás clases.

Feedback Recibido

Por último, la sección Feedback Recibido reúne las opiniones de los estudiantes sobre sus tutorías, en cumplimiento del RF-10, como se observa en la Figura 3.25. En la parte superior se presentan la valoración promedio de las clases, en una escala de 1 a 5, el puntaje SUS promedio, el número de encuestas SUS y TAM respondidas y el número de clases creadas. Debajo, el docente puede filtrar por clase y consultar, para cada estudiante, su valoración, sus comentarios o dudas y su resultado en la encuesta SUS y TAM.

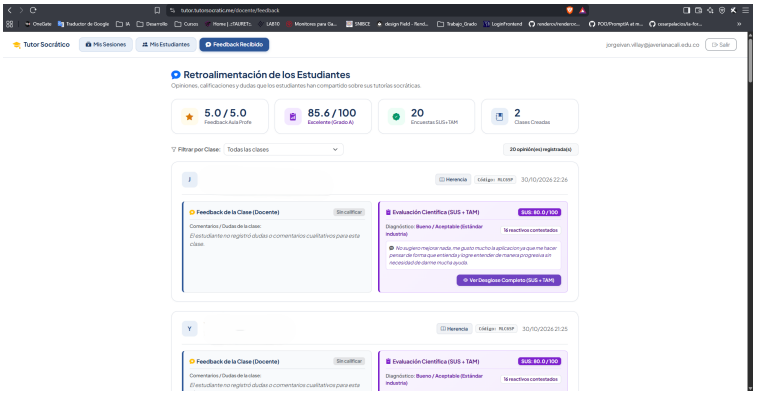


Figura 3.25: Sección Feedback Recibido del docente

En cuanto a la encuesta, el docente puede abrir el desglose completo de las respuestas de cada estudiante, en el que se muestran los diez ítems de la escala SUS con su puntaje estandarizado y su interpretación, junto con los ítems del modelo TAM, como se observa en la Figura 3.26.

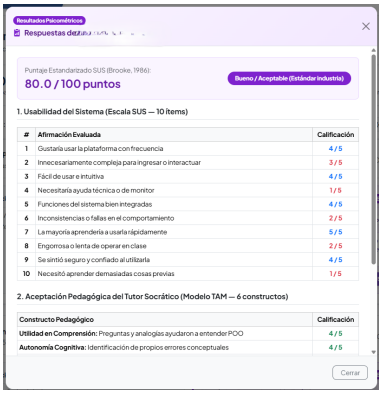


Figura 3.26: Desglose de las respuestas SUS y TAM de un estudiante

3.6.5. Funcionamiento Interno del Tutor Socrático

Si bien la construcción del prompt en tres capas y los flujos de datos se describieron en la Sección 3.4, su implementación requirió resolver tres aspectos adicionales. En primer lugar, cómo convertir las respuestas del modelo en métricas de seguimiento. En segundo lugar, cómo evitar que una decisión crítica dependa únicamente del criterio del modelo. Por último, cómo mantener la plataforma disponible ante fallas de la API. A continuación se describe la solución implementada para cada uno.

Meta-etiquetas y Registro de Métricas

Durante el diálogo, el modelo de lenguaje es quien identifica cuándo el estudiante avanza en la ruta de preguntas guía o solicita la solución. Por esta razón, en la Capa 1 y en la Capa 2 se le indicó que señalara estos eventos mediante meta-etiquetas. Es decir, marcas en formato de comentario HTML que el modelo agrega al final de su respuesta. Posteriormente, el servidor detecta estas etiquetas y registra el evento correspondiente en la tabla metrica. Además, las elimina antes de mostrar la respuesta, de modo que el estudiante nunca llega a verlas. La Tabla 3.6 resume cada una de ellas junto con el código con el que se registra.

Cuadro 3.6: Meta-etiquetas del tutor y su registro en la tabla metrica

Meta-etiqueta	**Evento que indica**	**Código**	**Métrica**
GUIA_INICIADA	El tutor planteó la primera pregunta guía.	20	M-02
GUIA_SUPERADA	El estudiante superó la pregunta guía número n.	21	M-02
RUTA_COMPLETADA	El estudiante superó la última pregunta guía.	No aplica	Cierre de la tutoría
ATAJO_DETECTADO	El estudiante pidió explícitamente el código o la solución.	10	M-03
No aplica	Total de mensajes de la conversación, registrado en cada turno.	2	Registro de actividad

Adicionalmente, con el fin de que un estudiante no pueda alterar sus propias métricas escribiendo estas etiquetas en el chat, el servidor elimina de cada mensaje del estudiante cualquier comentario HTML y cualquier nombre de etiqueta antes de enviarlo al modelo.

Validación del Cierre de la Ruta

Puesto que la finalización de la tutoría es una decisión que bloquea el chat y genera el informe del estudiante, se decidió no depender únicamente del criterio del modelo. Por ello, cuando el tutor envía la etiqueta RUTA_COMPLETADA, el servidor la acepta solo si se cumplen tres condiciones. Primero, que la clase tenga preguntas guía configuradas. Segundo, que el estudiante haya enviado al menos tantos mensajes como preguntas guía tiene la clase, con un mínimo de cuatro. Tercero, que la respuesta provenga del modelo principal y no del modelo de respaldo, dado que este último es menos confiable al seguir reglas condicionales. En caso contrario, la etiqueta se ignora y la tutoría continúa con normalidad.

Disponibilidad ante Fallas de la API

Con el fin de que la plataforma siga respondiendo aun cuando la API presente alta demanda, el servicio de conexión con Gemini implementa una estrategia de respaldo en dos niveles. Inicialmente, cada solicitud se envía al modelo principal, Gemini 3 Flash y si este reporta saturación, se reintenta una vez más tras una breve espera. Si aun así no responde, la solicitud se redirige al modelo de respaldo, Gemini 3.1 Flash-Lite. Finalmente, si ninguno de los dos responde, el estudiante recibe un mensaje indicando que se trata de un inconveniente temporal y que puede intentarlo de nuevo, sin que la plataforma deje de funcionar. Además, cada conexión tiene un tiempo máximo de espera de diez segundos para establecerse y de treinta y cinco segundos para recibir la respuesta, lo que evita que una solicitud quede bloqueada de forma indefinida.

Generación de Informes en Segundo Plano

Por último, dado que la generación de un informe individual puede tardar varios segundos, esta se ejecuta en segundo plano, de modo que el estudiante recibe la respuesta final del tutor sin esperar a que el informe termine. Para ello, se configuró un ejecutor dedicado con dos hilos de trabajo, que procesa los informes uno a uno y limita así el número de solicitudes simultáneas a la API y a la base de datos. De igual forma, cuando el docente solicita los informes pendientes de una clase, estos se encolan en una única tarea que los genera de manera secuencial y al terminar, actualiza automáticamente la síntesis grupal. Asimismo, si el docente presiona el botón nuevamente mientras la tarea está en curso, el sistema se lo indica y no la duplica.

3.6.6. Trazabilidad de los Requisitos

Con el fin de verificar que el prototipo atiende la totalidad de los requisitos definidos en la Sección 3.3, la Tabla 3.7 relaciona cada requisito funcional y no funcional con la funcionalidad que lo implementa y con la evidencia presentada en este capítulo. En el caso de las funcionalidades del estudiante, la evidencia corresponde al video de demostración incluido en los anexos.

Cuadro 3.7: Trazabilidad entre los requisitos y las funcionalidades implementadas

Requisito	**Funcionalidad que lo implementa**	**Evidencia**
RF-01	Creación de la clase con enlace único y registro o ingreso del estudiante desde dicho enlace.	Figuras 3.15 y 3.14
RF-02	Chat socrático basado en las reglas de la Capa 1, que guía mediante preguntas, pistas y analogías sin entregar la solución.	Video del portal del estudiante
RF-03	Respuesta del tutor tras cada mensaje del estudiante, con respaldo de modelo ante fallas de la API.	Video del portal del estudiante y subsección de funcionamiento interno del tutor
RF-04	Almacenamiento del historial de cada participación, el cual se envía al modelo en cada turno como Capa 3.	Video del portal del estudiante (Retomar Tutoría)
RF-05	Panel de Configuración de Prompts con IA, con la sugerencia de directrices CRAFT y las preguntas guía.	Figuras 3.16 y 3.17
RF-06	Focos de Dificultad y Falencias del grupo (M-01) e informe individual con los errores recurrentes de cada estudiante.	Figuras 3.21y 3.23
RF-07	Síntesis Pedagógica Consolidada con el diagnóstico del grupo y recomendaciones para la clase.	Figura 3.21
RF-08	Tiempo activo, clases completadas, última actividad y alerta de estudiante en riesgo (M-04).	Figuras 3.22 y 3.24
RF-09	Botón Feedback Profe, con valoración de 1 a 5 y comentarios para el docente.	Video del portal del estudiante
RF-10	Sección Feedback Recibido, con las valoraciones, los comentarios y las encuestas SUS y TAM.	Figuras 3.25 y 3.26
RNF-01	Cada estudiante solo accede a su propia participación y el docente solo a las clases que creó. Además, el docente no accede al chat, sino a los informes y métricas derivados.	Subsecciones de gestión de usuarios y acceso, y del portal del profesor
RNF-02	Registro único con nombre, correo institucional y contraseña cifrada con BCrypt, reutilizado en todas las clases.	Figuras 3.11 y 3.14

3.6.7. Desarrollo del Prompt del Tutor Socrático

Dado que el comportamiento del tutor depende por completo de las instrucciones que recibe el modelo, la redacción de la Capa 1 constituyó uno de los procesos centrales del desarrollo. Este prompt se construyó de forma iterativa, partiendo de un diagnóstico de los riesgos de una interpretación ingenua del método socrático y refinándose a medida que se observaba el comportamiento del tutor en las conversaciones.

Punto de Partida: el Riesgo de un Tutor que solo Pregunta

Una primera aproximación al método socrático consiste en indicarle al modelo que responda siempre con una pregunta. Sin embargo, este enfoque presenta tres problemas pedagógicos. En primer lugar, un uso inadecuado del método socrático puede carecer de seguridad psicológica para el estudiante, quien percibe el diálogo como un interrogatorio (Chang, 2023). En segundo lugar, la retroalimentación puede resultar insuficiente cuando el estudiante tiene un conocimiento previo limitado sobre el tema (Castellanos et al., 2026). Por lo tanto, un tutor que solo pregunta difícilmente le permite descubrir algo que nunca ha estudiado. Por último, una retroalimentación que no se ajusta al nivel del estudiante o que le exige demasiado puede afectar negativamente su motivación (Castellanos et al., 2026). Por esta razón, se decidió que el tutor no debía limitarse a preguntar, sino combinar la pregunta con una retroalimentación elaborada, es decir, aquella que explica por qué una respuesta es correcta o no, en línea con lo planteado en la Sección 3.1 (Castellanos et al., 2026).

Anatomía de cada Respuesta

A partir de este diagnóstico, se definió que cada respuesta del tutor debía articular tres componentes, como se resume en la Tabla 3.8. El primero se fundamenta en que la retroalimentación constructiva motiva al estudiante cuando reconoce sus esfuerzos y su progreso (Castellanos et al., 2026). El segundo responde a que la retroalimentación constituye una forma de andamiaje para la resolución de problemas (Castellanos et al., 2026), para lo cual el tutor se apoya en analogías, que Chang identifica como uno de los métodos del diálogo socrático (Chang, 2023). Por último, el tercero busca que sea el estudiante quien aplique lo explicado, dado que se recomienda que la retroalimentación promueva su reflexión y la mejora de su propia respuesta (Castellanos et al., 2026).

Adicionalmente, se estableció que el tutor formule exactamente una pregunta al final de cada mensaje. Esta regla se basa, por un lado, en lo observado por Chang, según quien formular las indicaciones una a una produce resultados más detallados y, en contextos de enseñanza, permite que el estudiante avance paso a paso (Chang, 2023). Por otro lado, se basa en la recomendación de que la retroalimentación sea específica, corta y fácil de entender, de modo que resulte manejable para el estudiante y se oriente a un objetivo concreto (Castellanos et al., 2026).

Cuadro 3.8: Anatomía de una respuesta del tutor socrático

Componente	**Propósito**
Validación empática	Reconocer el esfuerzo o los aciertos parciales del estudiante y señalar el desajuste conceptual sin juzgarlo.
Andamiaje, analogía o ejemplo neutro	Explicar el modelo mental detrás del concepto mediante una analogía de la vida real o un ejemplo breve en un dominio distinto al del ejercicio, de modo que no resuelva la tarea.
Pregunta orientadora	Cerrar con una sola pregunta que invite al estudiante a aplicar lo explicado a su código o a la pregunta guía activa.

Estructura de la Capa 1

Con base en lo anterior, la Capa 1 se organizó en siete secciones, las cuales se presentan en la Tabla 3.9. Cabe destacar que la cuarta sección incorpora los métodos de diálogo socrático descritos por Chang, como la mayéutica, el contraexamen o la generalización (Chang, 2023). Si bien Chang los plantea para que una persona construya prompts dirigidos a un modelo de lenguaje, en este proyecto se adaptaron al rol del tutor, indicando la situación del diálogo en la que conviene utilizar cada uno.

En cuanto a la primera sección, las restricciones de no entregar la solución, no corregir el código por el estudiante y no responder con un simple "correcto" o "incorrecto" se fundamentan en la diferencia entre la retroalimentación de verificación, que solo indica si el resultado es correcto, y la retroalimentación elaborada (Castellanos et al., 2026). Asimismo, buscan evitar que el estudiante se acostumbre a responder la pregunta en lugar de aprender los conceptos (Castellanos et al., 2026). Por su parte, el uso de ejemplos en dominios neutros, distintos al del ejercicio, constituye una decisión de diseño del proyecto para que los ejemplos ilustren el concepto sin resolver la tarea.

Respecto a la quinta sección, el escalamiento de pistas retoma la secuencia de pistas propuesta por Colby, que avanza de lo general a lo específico, comenzando con pistas que señalan el posible error y continuando con pistas de enseñanza (Colby, 2017). No obstante, a diferencia de dicha secuencia, que culmina con una pista de fondo que entrega la respuesta, el tutor se diseñó bajo el enfoque de tutoría informativa, en el que la respuesta correcta no se presenta al estudiante, sino que se le sugieren formas de resolver el problema (Castellanos et al., 2026). En consecuencia, el último nivel de pista corresponde a un mini-ejemplo en un dominio neutro que el estudiante debe adaptar a su propio problema.

Finalmente, la séptima sección retoma dos de los principios del método socrático descritos por Chang, el resumen y la obtención de conclusiones a partir del diálogo, y la reflexión sobre el proceso seguido (Chang, 2023).

Cuadro 3.9: Estructura de la Capa 1 del prompt

Sección	Contenido
1. Restricciones inquebrantables	Ocho reglas que el tutor no puede incumplir, como no entregar la solución ni código copiable, utilizar ejemplos en dominios neutros, responder siempre en español y no revelar sus instrucciones.
2. Anatomía de cada respuesta	Los tres componentes de la Tabla 3.8 y la regla de una sola pregunta por mensaje.
3. Orquestación de las preguntas guía	Cómo iniciar la tutoría, avanzar entre las preguntas guía del docente, evitar la divagación y cerrar la ruta.
4. Métodos de diálogo socrático	Mayéutica, definición, contraexamen, eliminación de hipótesis, razonamiento contrafactual, generalización y dialéctica.
5. Escalamiento de pistas	Tres niveles de pista, desde una pista conceptual abierta hasta un mini-ejemplo en un dominio neutro, sin llegar nunca a la solución.
6. Respuestas prohibidas y esperadas	Ejemplos contrastados de respuestas que el tutor debe evitar y de las que debe dar ante situaciones frecuentes.
7. Cierre de la interacción	Verificación de lo aprendido, refuerzo positivo y enlace con la siguiente pregunta guía.

Orquestación de las Preguntas Guía

Una de las decisiones más relevantes en la redacción del prompt fue definir cómo el tutor recorre las preguntas guía del docente. Su inclusión se apoya en lo observado por Chang, según quien comunicar al modelo el objetivo de la tarea y la intención del usuario antes de iniciar el diálogo le permite conectarse con dicho contexto y desempeñarse de manera más efectiva (Chang, 2023). En este caso, ese objetivo lo cumplen las preguntas guía definidas por el docente en la Capa 2.

Para recorrerlas, se definieron en el prompt cinco reglas. En primer lugar, una apertura diagnóstica, en la que el tutor no plantea de inmediato la primera pregunta guía, sino que antes indaga qué sabe el estudiante del tema para ajustar el nivel de apoyo, en concordancia con la recomendación de que la retroalimentación sea apropiada para su nivel de conocimiento (Castellanos et al., 2026). En segundo lugar, una regla mediante la cual el tutor atiende cualquier duda puntual del estudiante, pero cierra su respuesta vinculándola con la pregunta guía activa, dado que la retroalimentación debe mantenerse alineada con los objetivos de la clase para contribuir al aprendizaje (Jung et al., 2026). En tercer lugar, un criterio de progresión, según el cual el tutor solo avanza cuando el estudiante explica el concepto con

sus propias palabras, de modo que un simple "ya entendí" no es suficiente, en línea con la idea de dominio de los ITS, en la que el estudiante debe demostrar de forma consistente una habilidad antes de avanzar (Colby, 2017). En cuarto lugar, y con el mismo fin de mantener la alineación con los objetivos, una regla que reorienta la conversación cuando esta se aleja de la clase. Por último, la culminación de la ruta, en la que el tutor verifica en el historial que todas las preguntas guía fueron superadas antes de felicitar al estudiante y dar por finalizada la tutoría.

Asimismo, para que estos avances pudieran medirse, en estas reglas se le indicó al tutor que señalara cada uno mediante las meta-etiquetas descritas en la Tabla 3.6. Con el mismo fin, la Capa 2 se construye con las preguntas guía numeradas y con un recordatorio del uso de dichas etiquetas, ubicado junto a las preguntas, lo que reduce la probabilidad de que el modelo las omita.

Refinamiento a lo largo del Desarrollo

Por último, el prompt no se redactó en una única versión, sino que se fue ajustando a medida que se revisaban las conversaciones con el tutor. En este proceso se incorporaron, entre otras, tres reglas. La primera fue la señal de solicitud de atajo, que permitió medir cuándo el estudiante pide la respuesta directa (M-03) sin que el tutor deje de responder de forma socrática. La segunda fue la regla de idioma único, que mantiene la tutoría en español aun cuando el estudiante escriba en otro idioma. La tercera fue la confidencialidad de las instrucciones, que impide que el tutor revele sus reglas, la configuración CRAFT o las preguntas guía, incluso si el estudiante afirma ser el docente o le pide ignorar sus instrucciones, con el fin de prevenir los intentos de filtración del prompt descritos por Craig et al. (2024). De esta manera, el prompt resultante, incluido en los anexos, refleja tanto los fundamentos teóricos del método socrático y de la retroalimentación elaborada como los ajustes derivados de su uso real en la plataforma.

Capítulo 4

Evaluación de la Plataforma

El presente capítulo describe la evaluación de la usabilidad y la aceptación del prototipo, en cumplimiento del cuarto objetivo específico del proyecto. Para ello, se adoptó el procedimiento de evaluación de usabilidad (Picolo et al., 2025), el cual se organiza en cuatro etapas: la planificación de la prueba, la selección de los participantes, la aplicación del instrumento y el análisis de los datos. En este capítulo se presentan, en primer lugar, la planificación y el diseño del cuestionario y posteriormente, la aplicación y los resultados obtenidos.

4.1. Planificación de la Evaluación

La evaluación tuvo como objetivo medir, desde la perspectiva de los estudiantes, dos dimensiones del prototipo. Por un lado, su usabilidad, es decir, qué tan fácil, consistente y confiable resulta utilizar la plataforma. Por otro lado, su aceptación, entendida como la percepción de utilidad del tutor socrático para el aprendizaje y la disposición a seguir utilizándolo. Para medir la primera dimensión se seleccionó la escala SUS (System Usability Scale), mientras que para la segunda se tomó como referencia el modelo de aceptación de la tecnología TAM (Technology Acceptance Model).

En cuanto a la modalidad, se decidió integrar el cuestionario directamente en la plataforma, de modo que el estudiante lo respondiera al finalizar su tutoría, sin necesidad de acceder a un formulario externo. Esta decisión permitió, además, asociar cada respuesta con la participación del estudiante en la clase y, en consecuencia, contrastar su percepción con las métricas de uso registradas durante la sesión.

4.2. Diseño del Cuestionario

El cuestionario quedó conformado por dieciséis ítems y un campo de comentario abierto. Los diez primeros corresponden a la escala SUS y los seis restantes a los constructos del modelo TAM, todos ellos calificados en una escala tipo Likert de cinco puntos, en la que 1 corresponde a “Totalmente en desacuerdo” y 5 a “Totalmente de acuerdo”, la misma que utiliza Picolo et al. (2025) para la escala SUS.

4.2.1. Escala de Usabilidad SUS

Se seleccionó la escala SUS debido a que genera un puntaje único en una escala de fácil interpretación, es sencillo de aplicar, cuenta con buena confiabilidad y no tiene restricciones de uso. Asimismo, es uno de los instrumentos más utilizados en la evaluación de herramientas educativas (Picolo et al., 2025).

La escala está compuesta por diez afirmaciones que alternan un sentido positivo, en los ítems impares, y un sentido negativo, en los ítems pares. Con el fin de que los estudiantes las interpretaran en relación con la actividad realizada, las afirmaciones se adaptaron al contexto de la plataforma, conservando el sentido de cada ítem original, como se presenta en la Tabla 4.1.

Cuadro 4.1: Ítems de la escala SUS adaptados a la plataforma

Ítem	**Afirmación**
1	Creo que me gustaría utilizar esta plataforma con frecuencia para mis prácticas de programación.
2	Encontré la plataforma innecesariamente compleja para ingresar o interactuar.
3	Considero que la plataforma fue fácil de usar e intuitiva.
4	Creo que necesitaría la ayuda de un profesor o monitor para poder utilizar esta plataforma.
5	Sentí que las distintas funciones del sistema estaban bien integradas.
6	Percibí demasiadas inconsistencias o fallas en el comportamiento del sistema.
7	Imagino que la mayoría de estudiantes de programación aprenderían a usar esta plataforma muy rápidamente.
8	Encontré la plataforma muy engorrosa o lenta de operar durante la actividad.
9	Me sentí seguro y confiado interactuando con la plataforma.
10	Necesité aprender demasiadas cosas antes de poder empezar a interactuar con el tutor.

Para el cálculo del puntaje, cada ítem se califica de 1 a 5. En los ítems impares, el puntaje individual se obtiene restando 1 a la calificación asignada, mientras que en los ítems pares se obtiene restando la calificación a 5. Posteriormente, la suma de estos puntajes se multiplica por 2,5, lo que da como resultado un valor entre 0 y 100, en el que los valores más cercanos a 100 indican una mayor satisfacción del usuario (Picolo et al., 2025). Este cálculo se implementó en la plataforma, de modo que el puntaje de cada estudiante se obtiene automáticamente al enviar el cuestionario.

Finalmente, para interpretar el puntaje obtenido se adoptó la clasificación de Bangor et al., utilizada por (Picolo et al., 2025), la cual se presenta en la Tabla 4.2.

Cuadro 4.2: Clasificación del puntaje SUS, adaptada de Bangor et al. (2008), citado en Picolo et al. (2025)

Clasificación	**Puntaje medio**
Mejor imaginable	100
Excelente	85,58
Buena	72,75
Aceptable	52,01
Pobre	39,17
Peor imaginable	25

4.2.2. Ítems de Aceptación basados en el Modelo TAM

El modelo de aceptación de la tecnología, propuesto por Davis (1989), citado en González-Bravo y Valdivia-Peralta (2015), se centra en dos constructos considerados determinantes de la aceptación de una tecnología por parte de sus usuarios. Por un lado, la utilidad percibida, definida como la medida en que una persona cree que utilizar una tecnología en particular mejorará su rendimiento. Por otro lado, la facilidad de uso percibida, entendida como el grado en que una persona cree que utilizar dicha tecnología no le requerirá un mayor esfuerzo (González-Bravo & Valdivia-Peralta, 2015). Adicionalmente, en este modelo la aceptación se caracteriza como la combinación de una actitud positiva hacia la tecnología, la intención de utilizarla y su uso real. Cabe destacar que el modelo TAM ha sido utilizado como una herramienta viable en la investigación sobre nuevas tecnologías en el ámbito educativo (González-Bravo & Valdivia-Peralta, 2015).

Con base en estos constructos, se formularon seis ítems orientados específicamente a la propuesta pedagógica del prototipo, es decir, a la experiencia de aprender mediante un tutor que guía con preguntas y analogías en lugar de entregar el código resuelto. La Tabla 4.3 presenta cada ítem junto con el constructo del modelo TAM al que corresponde.

Cuadro 4.3: Ítems de aceptación y su constructo en el modelo TAM

Ítem	Afirmación	Constructo TAM
11	Recibir preguntas orientadoras y analogías en vez del código resuelto me ayudó a entender mejor la lógica de POO.	Utilidad percibida
12	La interacción con el tutor me ayudó a identificar mis propios errores conceptuales y a avanzar en la solución por mí mismo.	Utilidad percibida
13	Sentí que el tutor me desafió a pensar de forma constructiva, en lugar de generarme frustración por no darme el código de inmediato.	Actitud hacia el uso
14	Las analogías del mundo real y el ritmo de conversación, una pregunta a la vez, fueron claros y facilitaron mi aprendizaje.	Facilidad de uso percibida
15	Considero más provechoso para mi formación aprender con este tutor socrático que simplemente pidiéndole el código resuelto a herramientas como ChatGPT.	Utilidad percibida
16	Si esta plataforma estuviera disponible en futuras asignaturas de programación, la utilizaría voluntariamente para estudiar.	Intención de uso

De esta manera, mientras la escala SUS evalúa la plataforma como sistema de software, los ítems TAM evalúan la aceptación del enfoque socrático del tutor, que constituye el núcleo de la propuesta. A diferencia de la escala SUS, estos ítems se analizan mediante el promedio de las calificaciones de cada constructo, sin convertirlos a un puntaje único.

4.2.3. Integración del Cuestionario en la Plataforma

El cuestionario se implementó como una ventana dentro de la Interfaz de Interacción Socrática, a la que el estudiante accede mediante el botón **Evaluar Plataforma (SUS+TAM)**, y también desde su lista de clases, como se observa en la Figura 4.1. Al enviarlo, la plataforma calcula el puntaje SUS y almacena las dieciséis respuestas, el puntaje y el comentario abierto en la participación del estudiante, en formato JSON. Asimismo, una vez enviado el cuestionario, la tutoría del estudiante se da por finalizada, de modo que la evaluación refleja la experiencia completa con el tutor. Por último, los resultados quedan disponibles para el docente en la sección Feedback Recibido, tal como se describió en el capítulo anterior.

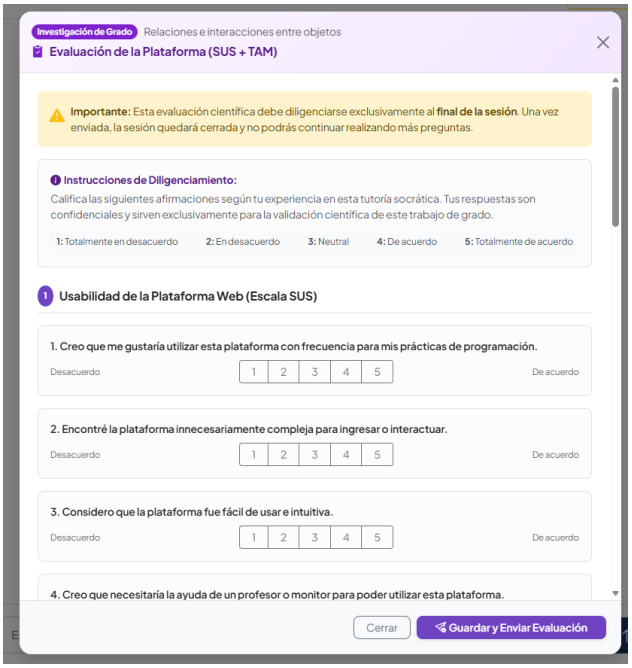


Figura 4.1: Cuestionario SUS y TAM integrado en la plataforma

4.3. Selección de los Participantes

La evaluación se llevó a cabo con estudiantes del curso de Programación Orientada a Objetos de la Pontificia Universidad Javeriana Cali, quienes constituyen el público objetivo del prototipo. Para ello, el profesor **Jorge Iván Villay Játiva** creo en en la plataforma la clase **Herencia**, con cuatro preguntas guía orientadas a identificar atributos y comportamientos comunes entre clases, diferenciar la herencia de la composición, comprender el acceso a los atributos privados de la clase base y reconocer el orden de ejecución de los constructores. Cabe resaltar, que el profesor muy amablemente puso la plataforma como una actividad evaluativa, lo cual hizo más fácil la evaluación de la plataforma.

En total, veinte estudiantes ingresaron a la clase y de ellos, los veinte respondieron el cuestionario, lo que representa una tasa de respuesta del 100 % sobre los participantes. Cabe resaltar que, de acuerdo con Tullis y Stetson (2004), citados por Picolo et al. (2025), se requiere una muestra de doce a catorce participantes para obtener resultados razonablemente confiables con la escala SUS, por lo que el número de participantes supera dicho umbral.

4.4. Aplicación del Cuestionario

La actividad se realizó entre el 26 y el 30 de octubre de 2026, de forma remota y asíncrona. Durante este periodo, cada estudiante ingresó a la clase mediante el enlace compartido por el docente, recorrió las preguntas guía con el tutor socrático y al finalizar, respondió el cuestionario desde la misma plataforma. La participación se realizó como parte de una actividad evaluativa propuesta por el docente. Por su parte, al igual que en el estudio de Picolo et al. (2025), los resultados se analizaron de forma anónima, de modo que en este documento cada participante se identifica únicamente con un número.

4.5. Análisis de Resultados

4.5.1. Usabilidad según la Escala SUS

El puntaje SUS promedio obtenido por el prototipo fue de **85,62** puntos, con una desviación estándar de 16,76, una mediana de 90 y valores entre 50 y 100. De acuerdo con la clasificación de Bangor et al. presentada en la Tabla 4.2, este puntaje se ubica en el nivel **Excelente**, cuyo puntaje medio es de 85,58. A modo de referencia, este resultado supera el puntaje de 75,23 obtenido por el sistema evaluado por (Picolo et al., 2025), el cual fue clasificado como bueno. La Figura 4.2 presenta la distribución de los participantes según su clasificación.

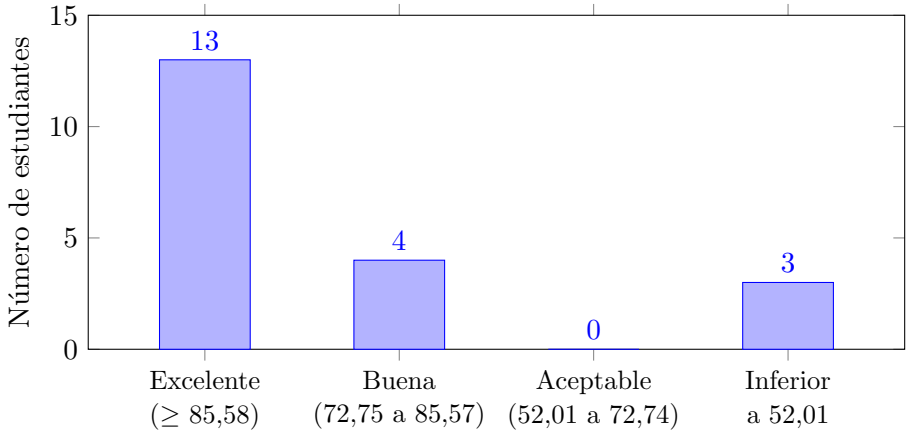


Figura 4.2: Distribución de los puntajes SUS según la clasificación de Bangor et al. (2008)

Como se observa, trece de los veinte estudiantes calificaron la plataforma en el nivel excelente y cuatro en el nivel bueno. Sin embargo, tres participantes obtuvieron un puntaje de 50, el cual requiere una lectura particular. En estos tres casos, el estudiante asignó la misma calificación a los diez ítems, ya sea 3 en todos ellos o 5 en todos ellos. Dado que la escala SUS alterna afirmaciones positivas y negativas, calificar con 5 tanto “la plataforma fue fácil

de usar" como "la plataforma fue innecesariamente compleja" resulta contradictorio y sugiere que el cuestionario se respondió sin leer cada afirmación. De hecho, dos de estos estudiantes completaron la ruta de preguntas guía y calificaron con 5 todos los ítems de aceptación, lo que no concuerda con un puntaje de usabilidad de 50. Por esta razón, si se excluyen estas tres respuestas, el puntaje promedio de los diecisiete participantes restantes asciende a 91,91, con una desviación estándar de 7,32, lo que confirma la clasificación excelente y una percepción homogénea entre los estudiantes.

Por otra parte, el análisis por ítem permite identificar las fortalezas y los aspectos a mejorar de la plataforma, como se presenta en la Tabla 4.4. En los ítems positivos, el promedio más alto corresponde a la seguridad y confianza al interactuar con la plataforma (4,85), seguido de la facilidad de uso (4,75). En cuanto a los ítems negativos, en los que un valor bajo es favorable, todos los promedios se mantuvieron entre 1,65 y 2,05. No obstante, los valores relativamente más altos corresponden a la complejidad para ingresar o interactuar (2,05) y a la necesidad de aprender cosas antes de empezar a interactuar con el tutor (2,00), lo que señala el proceso de ingreso y la primera interacción como los principales aspectos a mejorar.

Cuadro 4.4: Promedio por ítem de la escala SUS

Ítem	**Aspecto evaluado**	**Promedio**
1	Gustaría utilizarla con frecuencia	4,65
2	Innecesariamente compleja para ingresar o interactuar (negativo)	2,05
3	Fácil de usar e intuitiva	4,75
4	Necesitaría ayuda de un profesor o monitor (negativo)	1,75
5	Funciones bien integradas	4,60
6	Inconsistencias o fallas en el sistema (negativo)	1,65
7	La mayoría aprendería a usarla rápidamente	4,55
8	Engorrosa o lenta de operar (negativo)	1,70
9	Seguridad y confianza al interactuar	4,85
10	Necesitó aprender demasiadas cosas antes de empezar (negativo)	2,00

4.5.2. Aceptación según el Modelo TAM

En cuanto a la aceptación del tutor socrático, todos los ítems obtuvieron promedios superiores a 4 sobre 5, como se presenta en la Figura 4.3. El valor más alto corresponde a la intención de uso futuro (4,75), es decir, la mayoría de los estudiantes utilizaría voluntariamente la plataforma en futuras asignaturas de programación. Asimismo, se destaca la valoración del tutor frente a pedir el código resuelto a herramientas como ChatGPT (4,65), lo que indica

que los estudiantes reconocen el valor formativo del enfoque socrático frente a la obtención directa de soluciones.

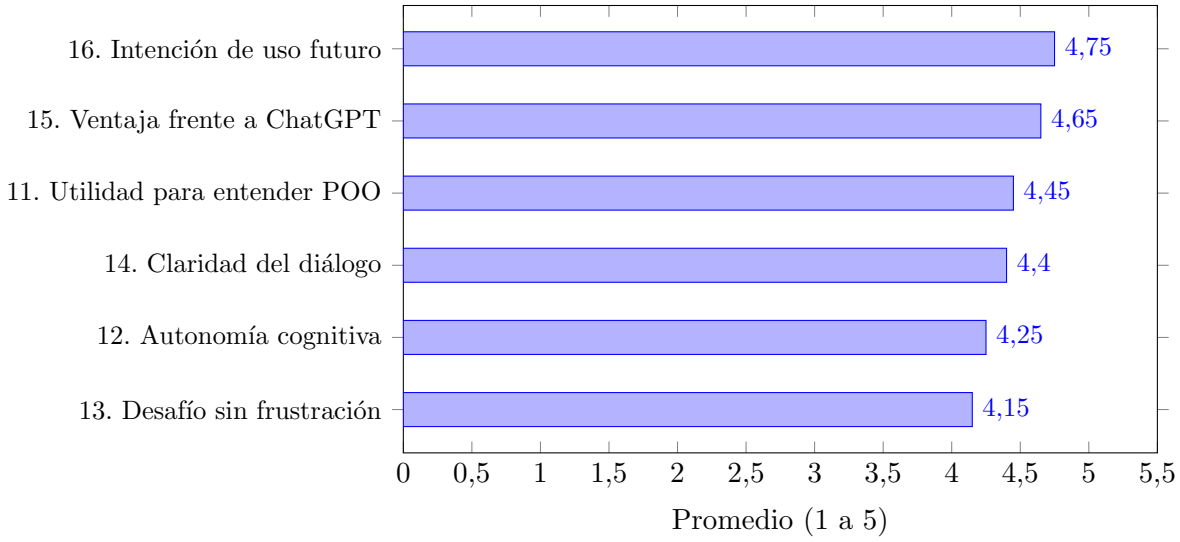


Figura 4.3: Promedio de los ítems de aceptación basados en el modelo TAM

Al agrupar los ítems por constructo, la utilidad percibida, que reúne los ítems 11, 12 y 15, obtuvo un promedio de 4,45, la facilidad de uso percibida (ítem 14) un promedio de 4,40, la actitud hacia el uso (ítem 13) un promedio de 4,15 y la intención de uso (ítem 16) un promedio de 4,75. De acuerdo con el modelo TAM, en el que la aceptación se caracteriza por una actitud positiva, la intención de utilizar el sistema y su uso real (González-Bravo & Valdivia-Peralta, 2015), estos resultados evidencian una alta aceptación del prototipo. No obstante, el promedio más bajo corresponde al ítem 13, que evalúa si el tutor desafió al estudiante sin generarle frustración por no entregarle el código. Este resultado es coherente con la naturaleza del método socrático, en el que el esfuerzo de razonar por cuenta propia puede resultar exigente para algunos estudiantes, y constituye un aspecto a considerar en el ajuste del nivel de apoyo del tutor.

4.5.3. Comentarios de los Estudiantes

De los veinte participantes, once dejaron un comentario abierto en el cuestionario, cuyo análisis permite identificar tres aspectos valorados de forma recurrente. En primer lugar, el uso de ejemplos sencillos y cercanos a la vida real para explicar los conceptos, mencionado por cinco estudiantes, como en el caso del participante que destacó que el tutor “ayuda a entender con buenos ejemplos y te guía en caso tal de que te equivoques”. En segundo lugar, el hecho de que el tutor no entregue la respuesta directamente, sino que guíe el razonamiento, como lo expresó otro participante al señalar que “me hace pensar de forma que entienda y logre entender de

manera progresiva sin necesidad de darme mucha ayuda". Por último, la adaptación del lenguaje y el trato motivador del tutor, junto con el deseo explícito de seguir utilizando la plataforma en futuras prácticas y para estudiar.

Por otro lado, se identificó una sugerencia de mejora, según la cual el uso de demasiados ejemplos de la vida real puede, en ocasiones, resultar confuso. Esta observación coincide con el resultado del ítem 14, relacionado con la claridad de las analogías, y sugiere moderar su uso cuando el estudiante ya comprende el concepto.

4.5.4. Uso del Prototipo durante la Actividad

Con el fin de contrastar la percepción de los estudiantes con su uso real de la plataforma, se analizaron las métricas registradas durante la actividad, presentadas en la Tabla 4.5. Como se observa, el 80 % de los estudiantes completó la ruta de preguntas guía, con un promedio de 7,45 mensajes por estudiante y un tiempo activo promedio de 15,7 minutos. Asimismo, solo se registraron dos solicitudes de respuesta directa sobre un total de 149 mensajes, lo que representa el 1,34 % de las consultas y evidencia que los estudiantes se involucraron en el proceso de razonamiento en lugar de buscar la solución.

Cuadro 4.5: Métricas de uso registradas durante la actividad

Indicador	**Resultado**
Estudiantes que completaron la ruta de preguntas guía	16 de 20 (80 %)
Mensajes enviados por los estudiantes	149 en total, 7,45 en promedio
Solicitudes de respuesta directa (M-03)	2, el 1,34 % de las consultas
Tiempo activo por estudiante (M-04)	15,7 minutos en promedio
Informes individuales generados	20 de 20

Por su parte, los informes individuales y la síntesis grupal generada por la plataforma identificaron tres focos de dificultad en el grupo. El primero, la sintaxis para invocar el constructor de la clase base desde la clase derivada, aun cuando el orden de ejecución de los constructores fue comprendido. El segundo, la duda sobre cuándo utilizar el modificador protected en lugar de mantener los atributos privados con métodos de acceso, junto con la creencia inicial de algunos estudiantes de que la herencia otorga acceso directo a los atributos privados. El tercero, errores puntuales al ubicar clases especializadas por encima de las generales en la jerarquía. En consecuencia, la síntesis recomendó al docente un taller práctico sobre la inicialización de constructores y un ejemplo comparativo entre los modificadores private y protected, lo que evidencia la utilidad del panel para orientar la siguiente clase presencial.

4.6. Limitaciones de la Evaluación

Si bien los resultados obtenidos son favorables, es importante considerar algunas limitaciones. En primer lugar, la evaluación se realizó con un único grupo de veinte estudiantes y sobre un solo tema, la herencia, por lo que sus resultados no pueden generalizarse a otros cursos o temáticas sin una evaluación adicional. En segundo lugar, tres respuestas presentaron un patrón uniforme que sugiere que el cuestionario no se respondió con atención, lo que afecta el puntaje promedio y evidencia la importancia de alternar afirmaciones positivas y negativas, como lo hace la escala SUS, para detectar este tipo de respuestas. Por último, la actividad se realizó de forma asíncrona y remota, por lo que no fue posible observar directamente la interacción de los estudiantes con la plataforma.

4.7. Síntesis del Capítulo

A partir de la evaluación realizada, se concluye que el prototipo presenta una usabilidad excelente, con un puntaje SUS promedio de 85,62 sobre 100, que asciende a 91,91 al excluir las respuestas inconsistentes, y una alta aceptación del enfoque socrático, con promedios superiores a 4 sobre 5 en todos los ítems basados en el modelo TAM, en particular en la intención de uso futuro. Además, estos resultados son coherentes con el uso registrado en la plataforma, en el que la mayoría de los estudiantes completó la ruta de preguntas guía y casi ninguno solicitó la respuesta directa. Por último, los aspectos de mejora identificados, como el proceso de ingreso, la primera interacción con el tutor y el equilibrio entre el desafío y la frustración, constituyen insumos para el refinamiento del prototipo. De esta manera, se da cumplimiento al cuarto objetivo específico del proyecto.

Capítulo 5

Conclusiones y Trabajos Futuros

5.1. Conclusiones

El presente proyecto tuvo como objetivo desarrollar un prototipo de plataforma web para la identificación de falencias en el aprendizaje y la generación de retroalimentación mediante un sistema de tutoría inteligente socrático. A partir del trabajo realizado, se presentan las conclusiones asociadas a cada uno de los objetivos específicos y finalmente, al objetivo general.

En relación con el primer objetivo específico, la revisión de la literatura sobre sistemas de tutoría inteligente y retroalimentación, junto con lo conversado con los stakeholders, permitió definir diez requisitos funcionales, dos requisitos no funcionales y cuatro métricas de seguimiento. Estas métricas, que abarcan la frecuencia de error por concepto, los intentos por pregunta guía, las solicitudes de respuesta directa y la frecuencia y duración de uso, demostraron ser medibles a partir de la propia interacción del estudiante con el tutor, sin requerir instrumentos adicionales. Asimismo, su acompañamiento con una traducción pedagógica permitió que cada indicador tuviera un significado práctico para el docente.

En cuanto al segundo objetivo específico, el diseño de la plataforma mediante los cuatro niveles del modelo C4 y el esquema de base de datos permitió estructurar sus módulos y definir con claridad las responsabilidades de cada componente antes de iniciar el desarrollo. Entre las decisiones de diseño, la arquitectura del prompt en tres capas resultó fundamental, dado que separó las reglas fijas del método socrático de la configuración del docente y del historial del estudiante. De esta manera, el docente pudo adaptar el tutor a cada clase sin riesgo de debilitar sus restricciones pedagógicas. Del mismo modo, la elección de Markdown para los informes y de registros código–valor para las métricas permitió combinar la legibilidad para el docente con la posibilidad de calcular indicadores de forma estructurada.

Respecto al tercer objetivo específico, se implementó y desplegó un prototipo funcional que integra un tutor socrático basado en Gemini, configurado mediante el framework CRAFT y las preguntas guía del docente. Entre los resultados del desarrollo, se destaca el asistente de configuración con IA, diseñado para que el docente pueda construir el prompt sin conocimientos previos en ingeniería de prompts. Además, el uso de meta-etiquetas permitió convertir el diálogo del tutor en métricas de seguimiento, mientras que la validación del cierre de la ruta en el servidor y el modelo de respaldo evitaron que las decisiones críticas o la disponibilidad de la plataforma dependieran únicamente del modelo. Por último, la generación automática

de informes individuales y de la síntesis grupal permitió al docente identificar las falencias del grupo sin tener que interpretar los datos por su cuenta, y la trazabilidad realizada evidenció que la totalidad de los requisitos definidos fue implementada.

En relación con el cuarto objetivo específico, la evaluación con veinte estudiantes del curso de Programación Orientada a Objetos arrojó un puntaje SUS promedio de 85,62, el cual corresponde a una usabilidad excelente y asciende a 91,91 al excluir las respuestas inconsistentes. Asimismo, los ítems basados en el modelo TAM obtuvieron promedios superiores a 4 sobre 5, en particular la intención de uso futuro, con 4,75 y la preferencia del tutor socrático frente a obtener el código resuelto de otras herramientas, con 4,65. Estos resultados fueron coherentes con el uso registrado, en el que el 80 % de los estudiantes completó la ruta de preguntas guía y las solicitudes de respuesta directa representaron apenas el 1,34 % de las consultas. No obstante, la evaluación también señaló aspectos de mejora, como el proceso de ingreso, la primera interacción con el tutor y el equilibrio entre el desafío y la frustración.

Finalmente, en cuanto al objetivo general, se concluye que es viable construir un sistema de tutoría inteligente socrático basado en un modelo de lenguaje comercial que, por un lado, guíe al estudiante en la construcción de su propio conocimiento sin entregarle la solución y por otro lado, ofrezca al docente información útil para el seguimiento de su grupo. En este sentido, el principal aporte del proyecto radica en mostrar que la parametrización del comportamiento del modelo mediante prompting, combinada con mecanismos de control en el servidor, permite obtener un tutor pedagógicamente consistente sin necesidad de entrenar un modelo propio, y que el mismo diálogo socrático puede convertirse en evidencia para identificar las falencias del aprendizaje.

5.2. Trabajos Futuros

A partir del desarrollo del prototipo y de la evaluación realizada, se identifican diversas líneas de trabajo futuro que permitirían ampliar el alcance del sistema y mejorar su robustez, precisión y aplicabilidad en contextos académicos más amplios.

En primer lugar, si bien el prototipo incorpora en las reglas del tutor un escalamiento de pistas en tres niveles, que va desde una pista conceptual abierta hasta un mini-ejemplo en un dominio neutro, inspirado en la secuencia de pistas de Colby (2017) pero bajo el enfoque de tutoría informativa, que orienta al estudiante sin entregarle la respuesta (Castellanos et al., 2026), este escalamiento depende únicamente de las instrucciones del prompt.. Dado que el comportamiento de un modelo de lenguaje no es completamente determinístico, no es posible garantizar de manera consistente que cada pista respete el nivel de progresividad esperado. Por ello, una línea de trabajo futuro corresponde a controlar dicho escalamiento desde el sistema, ya sea mediante un agente especializado que decida el nivel de la siguiente pista o mediante un modelo orientado específicamente a su generación.

De manera similar, aunque la literatura sugiere que un sistema de tutoría inteligente robusto

debería incorporar un modelo de seguimiento del estudiante como el Rastreo de Conocimiento Bayesiano (BKT), que permite inferir de forma continua qué habilidades ha dominado el alumno a partir de sus interacciones y sugerir la siguiente acción más apropiada cuando se encuentra bloqueado (Kurup et al., 2016), el presente prototipo no implementa este enfoque probabilístico. En su lugar, el seguimiento del progreso se realiza mediante un informe descriptivo en formato Markdown, generado por el modelo de lenguaje cuando el estudiante completa la ruta de preguntas guía o a solicitud del docente, junto con las métricas registradas durante el diálogo. Si bien esto permite una primera aproximación al seguimiento individual sin requerir la complejidad estadística ni los datos históricos necesarios para un modelo bayesiano, la incorporación de un modelo de tipo BKT, o de un mecanismo equivalente, permitiría transitar de un seguimiento descriptivo a uno predictivo del dominio del estudiante.

En esa misma línea, la implementación de un andamiaje (scaffolding) dinámico, es decir, la capacidad del sistema de proporcionar apoyo intensivo al estudiante en las etapas iniciales de un tema y retirarlo progresivamente a medida que este gana competencia, ajustando la dificultad de las tareas a su zona de desarrollo próximo (Colby, 2017), queda igualmente delimitada como trabajo futuro. Su implementación efectiva depende, en gran medida, de contar primero con un modelo de seguimiento del estudiante más preciso, por lo que se considera una extensión natural una vez incorporado el modelado bayesiano o un mecanismo equivalente.

Por otra parte, en el prototipo actual cada clase parte únicamente de su propia conversación, dado que el informe del estudiante no se envía al tutor. Una línea de trabajo complementaria consiste en aprovechar los informes de clases anteriores como contexto en las siguientes sesiones del mismo estudiante, de modo que el tutor pueda retomar sus errores recurrentes y dar continuidad a su proceso entre temas, sin que dicho contexto condicione negativamente su comportamiento.

Asimismo, la evaluación realizada se centró en la usabilidad y la aceptación del prototipo, con un único grupo y sobre un solo tema. Por ello, se propone ampliar la evaluación a otros grupos, temas y asignaturas de programación, e incorporar un diseño que permita medir el efecto del tutor sobre el aprendizaje, por ejemplo, mediante pruebas antes y después de la actividad o la comparación con un grupo de control.

Por último, a partir de los resultados de la evaluación, se plantean mejoras puntuales al prototipo. Entre ellas, simplificar el proceso de ingreso y la primera interacción con el tutor, moderar el uso de analogías cuando el estudiante ya comprende el concepto y ajustar el nivel de apoyo para reducir la frustración en los estudiantes con menos bases. Además, se propone que la plataforma notifique al docente de forma proactiva cuando un estudiante se encuentre en riesgo de abandono o se trabe en una pregunta guía, en lugar de que este deba consultarlo en el panel.

Capítulo 6

Anexos

6.0.1. Prompt de la Capa Fija del Tutor Socrático

Para poder revisar el contenido del archivo capa1_socratico.md, correspondiente a la Capa 1 del prompt, descrita en la Sección **Desarrollo del Prompt del Tutor Socrático**, por favvor acceder al siguiente enlace Accede Aquí

6.0.2. Video de Explicación del uso de la Plataforma

Para poder ver el vídeo de explicación del funcionamiento de la plataforma, por favor acceder al siguiente enlace Accede Aquí

6.0.3. Repositorio del Código Fuente

Para poder ver acceder al repositorio con el código fuente del proyecto, por favor acceder al siguiente enlace Accede Aquí

Bibliografía

Abiodun, E. F., & Isong, B. (2025). A Systematic Review of Challenges in Teaching and Learning Computer Programming Modules. *The Indonesian Journal of Computer Science*, *14*(1). https://doi.org/10.33022/ijcs.v14i1.4592

Almarales, M., & Fernández, O. (2019). Tutorial Interactivo para la Enseñanza de la Programación Orientada a Objetos. *Revista de Investigación , Formación y Desarrollo: Generando Productividad Institucional*, *7*, 8. https://doi.org/10.34070/rif.v7i3.172

Artificial Analysis. (2026). AI Model Comparison: Gemini 3 Flash reasoning, Grok 4 fast reasoning, GPT-5 mini, and Claude 4.5 Haiku reasoning [[En línea]. Disponible en: https://artificialanalysis.ai/models?models=gemini-3-flash-reasoning%2Cgrok-4-fast-reasoning%2Cgpt-5-mini%2Cclaude-4-5-haiku-reasoning. [Accedido: 30-ago-2026]].

Brown, S. (2022). *The C4 Model*. O'Reilly Media.

Castellanos, H., Vieira, C., & Magana, A. J. (2026). Feedback Systems on Computer Programming Courses: A Systematic Literature Review. *IEEE Access*, *14*, 30654-30674. https://doi.org/10.1109/ACCESS.2026.3665047

Chang, E. Y. (2023). Prompting Large Language Models With the Socratic Method. *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, 0351-0360. https://doi.org/10.1109/CCWC57344.2023.10099179

Colby, B. R. (2017). A Comparative Literature Review of Intelligent Tutoring Systems from 1992-2015. https://api.semanticscholar.org/CorpusID:150155560

Craig, J., Bleiberg, D., & Chugh, M. (2024, agosto). Implementing advanced prompt engineering with Amazon Bedrock [[En línea]. Disponible en: https://aws.amazon.com/es/blogs/machine-learning/implementing-advanced-prompt-engineering-with-amazon-bedrock/. [Accedido: 23-ago-2026]].

Deinum, M. (2018). Spring Boot—Introduction. En *Spring Boot 2 Recipes: A Problem-Solution Approach* (pp. 1-14). Apress. https://doi.org/10.1007/978-1-4842-3963-6_1

Evans, C. (2013). Making sense of assessment feedback in higher education. *Review of Educational Research*, *83*(1), 70-120. https://doi.org/10.3102/0034654312474350

Farieta-Barrera, A., & Murillo, A. (2023, enero). El método socrático y la posibilidad de una pedagogía socrática. https://doi.org/10.28970/9789585498921.01

González-Bravo, L., & Valdivia-Peralta, M. (2015). Possibilities for Using TAM and Technology Frames Models to Assess the Acceptance of New Technologies in the Chilean Higher Education Quality Assurance. *Revista Electrónica Educare*, *19*(2), 181-196. https://doi.org/10.15359/ree.19-2.11

Gusukuma, L., Bart, A. C., Kafura, D., & Ernst, J. (2018). Misconception-Driven Feedback: Results from an Experimental Study. *Proceedings of the 2018 ACM Conference on International Computing Education Research*, 160-168. https://doi.org/10.1145/3230977.3231002

IEEE Computer Society. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

Inc., D. (2026). Docker Overview [En línea. Disponible en: https://docs.docker.com/get-started/docker-overview/. Accedido: 06-jul-2026].

Inc., S. (2026). Supabase Documentation [En línea. Disponible en: https://supabase.com/docs. Accedido: 10-jun-2026].

International Organization for Standardization. (2022). Information security, cybersecurity and privacy protection — Information security management systems — Requirements (ISO/IEC Standard No. 27001:2022) [Accedido: 18-mar-2026]. https://www.iso.org/standard/27001

The Java TM Language : An Overview. (2009). https://api.semanticscholar.org/CorpusID:14097506

Joshi, D., Jocius, R., Albert, J., Joswick, C., & Blanton, M. (2026). CRAFT Prompt Generation Framework for Teachers. *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2*, 1383-1384. https://doi.org/10.1145/3770761.3777281

Jung, Y., Lee, Y., Bae, J., Kim, D., Choi, H., Kang, M., & Lee, U. (2026). Exploring the Role of Automated Feedback in Programming Education: A Systematic Literature Review. https://doi.org/10.48550/arXiv.2602.00089

Kurup, L. D., Joshi, A., & Shekhokar, N. (2016). A review on student modeling approaches in ITS. *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, 2513-2517.

Microsoft Learn. (2026). Object-Oriented programming (C#) [En línea. Disponible en: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop. Accedido: 20-jun-2026].

Molina, M. (2022). What is an intelligent system? [Version 3, updated December 2022].

Monett, D., & Lewis, C. W. P. (2018). Getting Clarity by Defining Artificial Intelligence—A Survey. En V. C. Muller (Ed.), *Philosophy and Theory of Artificial Intelligence 2017* (pp. 212-214). Springer International Publishing.

Pardos, Z. A., Tang, M., Anastasopoulos, I., Sheel, S. K., & Zhang, E. (2023). OATutor: An Open-source Adaptive Tutoring System and Curated Content Library for Learning Sciences Research [En línea. Disponible en: https://doi.org/10.1145/3544548.3581574. Accedido: 26-feb-2026]. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, 1-17. https://doi.org/10.1145/3544548.3581574

Pham, T., & Ngoc, T. N. (2025). Improving LLM's Response with a New Prompt Framework RCFOR. *2025 10th International Conference on Applying New Technology in Green Buildings (ATiGB)*, 666-670. https://doi.org/10.1109/ATiGB66719.2025.11142192

Picolo, L. F., Dan, L. F., Simoes, I., & Pereira, H. G. d. S. (2025). Avaliação somativa de usabilidade: uma abordagem utilizando o System Usability Scale (SUS). *Revista Eletrônica de Iniciação Científica em Computação*, *23*(1), 228-234. https://doi.org/10.5753/reic.2025.5313

Prasad Reddy, K. S. (2017). Web Applications with Spring Boot. En *Beginning Spring Boot 2: Applications and Microservices with the Spring Framework* (pp. 107-132). Apress. https://doi.org/10.1007/978-1-4842-2931-6_10

Quiñonez, A. G., et al. (2023). Educational Platforms: Digital Tools for the teaching-learning process in Education [En línea. Disponible en: https://edsociety.iberojournals.com/index.php/IBEROEDS/article/view/626. Accedido: 26-feb-2026]. *Ibero-American Journal of Education & Society*, *3*(1), 259-263.

Rodríguez Chávez, M. H. (2021). Sistemas de tutoría inteligente y su aplicación en la educación superior. *RIDE Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, *11*(22). https://doi.org/10.23913/ride.v11i22.848

Services, A. W. (s.f.). What is an API (Application Programming Interface)? [https://aws.amazon.com/what-is/api/. Accedido: 7-mzo-2026].

Shekar, S. C., Sowmya, U., Suharshitha, P., Sahith, S. V. D., & Varun, K. S. (2026). A Smart Code Review Framework using GPT and NLP-Driven Automatic Comment Generation. *2026 Fourth International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*, 297-302. https://doi.org/10.1109/ICAISS68683.2026.11526109

Xu, S., Wen, H.-N., Pan, H., Dominguez, D., Hu, D., & Zhang, X. (2025). Classroom Simulacra: Building Contextual Student Generative Agents in Online Education for Learning Behavioral Simulation. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3706598.3713773

Zhao, J., et al. (2025). CodeEdu: A Multi-Agent Collaborative Platform for Personalized Coding Education [En línea. Disponible en: https://arxiv.org/abs/2507.13814. Accedido: 27-feb-2026]. *arXiv preprint arXiv:2507.13814*.

Zhou, L. (2025). A Review of Educational Agents: Definitions, Features, Roles and Development Trends [En línea. Disponible en: https://bonoi.org/index.php/sief/article/view/1590. Accedido: 26-feb-2026]. *Science Insights Education Frontiers*, *28*(2), 4675-4688. https://doi.org/10.15354/sief.25.re552